

Applications linéaires, noyau ($\ker$), image (Im) et démonstration du théorème du rang

Charles EDOU NZE

2026-07-01

Table des matières

\chapter*{Jalon 8 : Applications linéaires, noyau (ker), image (Im) et démonstration du théorème du rang}

\section*{1. Présentation du concept clé

- \textbf{La Métaphore} : Imaginez une machine à étirer ou à faire tourner de la pâte à modeler. - L'\textbf{Application Linéaire}, c'est une machine respectueuse : si vous doublez la quantité de pâte au départ, vous aurez le double à l'arrivée. Si vous mettez deux morceaux de couleurs différentes, le résultat sera le même que si vous les aviez traités séparément puis mélangés. - Le \textbf{Noyau (ker)}, c'est la "poubelle" de la machine : ce sont tous les morceaux de pâte qui finissent écrasés en un tout petit point (zéro) après le passage dans la machine. - L'\textbf{Image (Im)}, c'est l'ensemble de toutes les formes que la machine est capable de fabriquer. - Le \textbf{Théorème du Rang}, c'est la loi de conservation : la complexité totale de votre pâte au départ est égale à la somme de ce qui a été écrasé (Noyau) et de ce qui a été transformé (Image). - \textbf{Le "Pourquoi on a inventé ça"} : La plupart des phénomènes physiques simples sont linéaires. En IA, transformer une image ou un texte revient souvent à appliquer une suite de ces machines. Savoir ce qui est "perdu" (le noyau) est vital. - \textbf{Visualisation} : Imaginez projeter l'ombre d'un cube 3D sur une feuille 2D. L'ombre est l'\textbf{Image (2D)}. La direction de la lumière qui "écrase" la profondeur est le \textbf{Noyau (1D)}. $3 = 1 + 2$. C'est le théorème du rang!

\section*{2. Formalisation

\subsection*{A. Définitions Formelles} Soient E et F deux $\mathbb{K}$ -espaces vectoriels. 1. \textbf{Application Linéaire} ($f : E \rightarrow F$) : Une application vérifiant : $\forall x, y \in E, f(x + y) = f(x) + f(y)$ (Additivité). - $\forall \lambda \in \mathbb{K}, \forall x \in E, f(\lambda \cdot x) = \lambda \cdot f(x)$ (Homogénéité). L'ensemble des applications linéaires de E dans F est noté $\mathcal{L}(E, F)$.

2. \textbf{Noyau (ker f)} : $\ker f = \{x \in E \mid f(x) = 0_F\}$. C'est un sous-espace vectoriel de E . 3. \textbf{Image (Im f)} : $\text{Im } f = f(E) = \{f(x) \mid x \in E\}$. C'est un sous-espace vectoriel de F . 4. \textbf{Rang (rg f)} : On appelle rang de f la dimension de son image : $\text{rg } f = \dim(\text{Im } f)$.

\subsection*{B. Théorèmes, Propositions & Lemmes} > \textbf{Caractérisation de l'injectivité} : f est injective $\iff \ker f = \{0_E\}$.

> \textbf{Théorème du Rang (Théorème Fondamental)} : > Soit E un espace vectoriel de dimension finie. Pour toute application linéaire $f : E \rightarrow F$, on a : >

$$\dim E = \dim(\ker f) + \text{rg}(f)$$

\section*{3. Démonstrations

\subsection*{Démonstration du Théorème Pivot : Théorème du Rang} Soit $f \in \mathcal{L}(E, F)$ avec $\dim E = n$.

1. \textbf{Initialisation / Cadre} : - Soit $p = \dim(\ker f)$. - Soit $(e_1, \dots, e_p)$ une base de $\ker f$. - D'après le théorème de la base incomplète, on peut compléter cette famille en une base de E : $\mathcal{B} = (e_1, \dots, e_p, e_{p+1}, \dots, e_n)$. - Nous voulons montrer que $(f(e_{p+1}), \dots, f(e_n))$ est une base de $\text{Im } f$. Si c'est le cas, alors $\dim(\text{Im } f) = n - p$, ce qui prouvera $n = p + (n - p)$.

2. \textbf{Étape 1} : Montrons que la famille est génératrice de $\text{Im } f$. Soit $y \in \text{Im } f$. Par définition, il existe $x \in E$ tel que $y = f(x)$. Comme $\mathcal{B}$ est une base de E , $x = \sum_{i=1}^n \lambda_i e_i$. Par linéarité de f : $y = f(\sum_{i=1}^n \lambda_i e_i) = \sum_{i=1}^n \lambda_i f(e_i)$. Or, pour $i \in \{1, \dots, p\}$, $e_i \in \ker f \implies f(e_i) = 0_F$. L'égalité devient : $y = \sum_{i=p+1}^n \lambda_i f(e_i)$. La famille est donc génératrice.

3. \textbf{Étape 2} : Montrons que la famille est libre. Supposons $\sum_{i=p+1}^n \mu_i f(e_i) = 0_F$. Par linéarité : $f(\sum_{i=p+1}^n \mu_i e_i) = 0_F$. Cela signifie que le vecteur $v = \sum_{i=p+1}^n \mu_i e_i$ appartient à $\ker f$. Comme $(e_1, \dots, e_p)$ est une base de $\ker f$, il existe des scalaires $(\alpha_1, \dots, \alpha_p)$ tels que : $\sum_{i=p+1}^n \mu_i e_i = \sum_{j=1}^p \alpha_j e_j$. En regroupant tout d'un côté : $\sum_{j=1}^p (-\alpha_j) e_j + \sum_{i=p+1}^n \mu_i e_i = 0_E$. Comme $\mathcal{B}$ est une base de E , elle est libre, donc tous les coefficients sont nuls. En particulier, $\forall i \in \{p+1, \dots, n\}, \mu_i = 0$. La famille est donc libre.

4. \textbf{Conclusion} : $(f(e_{p+1}), \dots, f(e_n))$ est une base de $\text{Im } f$. Son cardinal est $n - p$. Ainsi, $\text{rg}(f) = \dim E - \dim(\ker f)$.

\section*{4. Exercices d'Application

\subsection*{Exercice 1 : Application Directe (Noyau et Image)} \textbf{Énoncé} : Soit $f : \mathbb{R}^3 \rightarrow \mathbb{R}^2$ définie par $f(x, y, z) = (x + y, y + z)$. Déterminer $\ker f$ et $\text{rg } f$. \textbf{Correction Détaillée} : 1. \textbf{Noyau} : Résolvons $f(x, y, z) = (0, 0)$. - $x + y = 0 \implies x = -y$ - $y + z = 0 \implies z = -y$ - Donc $\ker f = \{(-y, y, -y) \mid y \in \mathbb{R}\} = \text{Vect}((-1, 1, -1))$. - $\dim(\ker f) = 1$. 2. \textbf{Théorème du rang} : $\dim \mathbb{R}^3 = \dim(\ker f) + \text{rg } f \implies 3 = 1 + \text{rg } f$. - $\text{rg } f = 2$. \textbf{Conclusion} : Le noyau est une droite vectorielle et l'image est l'espace $\mathbb{R}^2$ tout entier (car $\dim(\text{Im } f) = \dim \mathbb{R}^2$).

\subsection*{Exercice 2 : Niveau Avancé (Projecteurs)} \textbf{Énoncé} : Soit $p \in \mathcal{L}(E)$ tel que $p \circ p = p$. Démontrer que $E = \ker p \oplus \text{Im } p$. \textbf{Correction Détaillée} : 1. \textbf{Analyse} : On doit montrer que pour tout $x \in E$, il existe un unique couple $(y, z) \in \ker p \times \text{Im } p$ tel que $x = y + z$. 2. \textbf{Existence (Analyse-Synthèse)} : Écrivons $x = (x - p(x)) + p(x)$. - Posons $z = p(x)$. Par définition, $z \in \text{Im } p$. - Posons $y = x - p(x)$. Calculons $p(y) = p(x - p(x)) = p(x) - p(p(x)) = p(x) - p(x) = 0$. Donc $y \in \ker p$. - L'existence est prouvée. 3. \textbf{Somme directe (Intersection)} : Soit $u \in \ker p \cap \text{Im } p$. - $u \in \ker p \implies p(u) = 0$. - $u \in \text{Im } p \implies \exists v \in E, u = p(v)$. - Alors $p(u) = p(p(v)) = p(v) = u$. - Comme $p(u) = 0$, on en déduit $u = 0$. L'intersection est réduite à $\{0\}$. \textbf{Conclusion} : $E = \ker p \oplus \text{Im } p$.

\section*{5. Application en Intelligence Artificielle

- \textbf{Le Pont Théorique} : Les couches denses (Dense Layers / Linear Layers) des réseaux de neurones sont des applications affines (Linéaire + Translation). - \textbf{Exemple Concret} : Dans la \textbf{réduction de dimension} et la \textbf{Compression de Réseaux}, on cherche à savoir si une couche linéaire a un gros \textbf{Noyau}. Si $\dim(\ker f)$ est grand, cela signifie que beaucoup de combinaisons d'entrées sont "oubliées" par le réseau. Le \textbf{Rang} de la matrice de poids d'une couche détermine la "capacité expressive" de cette couche. Un réseau "Low-Rank" est plus rapide à entraîner et moins sujet au sur-apprentissage (Overfitting).

\section*{6. Liens Sémantiques - \textbf{Concepts Précédents requis} : Jalon 7 (Espaces vectoriels abstraits) - \textbf{Concepts Futurs dépendants} : Jalon-9, Jalon 30 (Trigonalisation d'endomorphismes et décomposition de Dunford.), Jalon 46 (Matrice jacobienne)

Exercices d'Application

\chapter*{Exercice 1 : Applications linéaires élémentaires (Difficulté : \{*)}

\section*{Énoncé du problème}

On considère l'application $f : \mathbb{R}^2 \rightarrow \mathbb{R}^3$ définie pour tout $(x, y) \in \mathbb{R}^2$ par :

$$f(x, y) = (x + y, x - y, 2x)$$

où $\mathbb{R}$ est le corps des nombres réels.

1. Démontrer que f est une application linéaire. 2. Déterminer le noyau de f , noté $\ker(f)$. En donner une base et préciser sa dimension. 3. Déterminer l'image de f , notée $\text{Im}(f)$. En donner une base et préciser sa dimension (qui est le rang de f). 4. Vérifier le théorème du rang pour l'application f .

\section*{Correction détaillée}

\subsection*{1. Démontrer que f est une application linéaire.

Pour qu'une application $f : E \rightarrow F$ soit linéaire, où E et F sont des espaces vectoriels sur un corps K , elle doit satisfaire deux propriétés : a) $f(\mathbf{u} + \mathbf{v}) = f(\mathbf{u}) + f(\mathbf{v})$ pour tous $\mathbf{u}, \mathbf{v} \in E$. b) $f(\lambda \mathbf{u}) = \lambda f(\mathbf{u})$ pour tout $\lambda \in K$ et tout $\mathbf{u} \in E$.

Soient $\mathbf{u} = (x_1, y_1)$ et $\mathbf{v} = (x_2, y_2)$ deux vecteurs de $\mathbb{R}^2$. Soit $\lambda \in \mathbb{R}$ un scalaire.

\subsubsection*{Propriété a) : Additivité $\mathbf{u} + \mathbf{v} = (x_1 + x_2, y_1 + y_2)$. Calculons $f(\mathbf{u} + \mathbf{v})$:

$$f(\mathbf{u} + \mathbf{v}) = f(x_1 + x_2, y_1 + y_2) = ((x_1 + x_2) + (y_1 + y_2), (x_1 + x_2) - (y_1 + y_2), 2(x_1 + x_2))$$

Réorganisons les termes :

$$f(\mathbf{u} + \mathbf{v}) = ((x_1 + y_1) + (x_2 + y_2), (x_1 - y_1) + (x_2 - y_2), 2x_1 + 2x_2)$$

Par ailleurs, calculons $f(\mathbf{u}) + f(\mathbf{v})$:

$$f(\mathbf{u}) = (x_1 + y_1, x_1 - y_1, 2x_1)$$

$$f(\mathbf{v}) = (x_2 + y_2, x_2 - y_2, 2x_2)$$

$$f(\mathbf{u}) + f(\mathbf{v}) = ((x_1 + y_1) + (x_2 + y_2), (x_1 - y_1) + (x_2 - y_2), (2x_1) + (2x_2))$$

Nous constatons que $f(\mathbf{u} + \mathbf{v}) = f(\mathbf{u}) + f(\mathbf{v})$. La première propriété est vérifiée.

\subsubsection*{Propriété b) : Homogénéité $\lambda \mathbf{u} = (\lambda x_1, \lambda y_1)$. Calculons $f(\lambda \mathbf{u})$:

$$f(\lambda \mathbf{u}) = f(\lambda x_1, \lambda y_1) = (\lambda x_1 + \lambda y_1, \lambda x_1 - \lambda y_1, 2\lambda x_1)$$

Factorisons λ dans chaque composante :

$$f(\lambda \mathbf{u}) = (\lambda(x_1 + y_1), \lambda(x_1 - y_1), \lambda(2x_1))$$

$$f(\lambda \mathbf{u}) = \lambda(x_1 + y_1, x_1 - y_1, 2x_1)$$

Par définition de $f(\mathbf{u})$, on a :

$$f(\lambda \mathbf{u}) = \lambda f(\mathbf{u})$$

La deuxième propriété est vérifiée.

Puisque les deux propriétés sont satisfaites, f est bien une application linéaire.

\subsection*{2. Déterminer le noyau de f , $\ker(f)$. En donner une base et sa dimension.

Le noyau de f , $\ker(f)$, est l'ensemble des vecteurs $\mathbf{u} \in \mathbb{R}^2$ tels que $f(\mathbf{u}) = \mathbf{0}_{\mathbb{R}^3}$, où $\mathbf{0}_{\mathbb{R}^3}$ est le vecteur nul de $\mathbb{R}^3$. Soit $\mathbf{u} = (x, y) \in \mathbb{R}^2$. $f(x, y) = (0, 0, 0)$ implique le système d'équations linéaires suivant : 1. $x + y = 0$ 2. $x - y = 0$ 3. $2x = 0$

À partir de l'équation (3), nous déduisons directement $x = 0$. Substituons $x = 0$ dans l'équation (1) : $0 + y = 0 \implies y = 0$. Vérifions ces valeurs dans l'équation (2) : $0 - 0 = 0$, ce qui est constant.

Ainsi, le seul vecteur (x, y) qui satisfait $f(x, y) = (0, 0, 0)$ est $(0, 0)$. Donc, $\ker(f) = \{(0, 0)\}$.

Une base de $\ker(f)$ est l'ensemble vide $\emptyset$, car le noyau ne contient que le vecteur nul. La dimension de $\ker(f)$, notée $\dim(\ker(f))$, est 0.

\{subsection*{3. Déterminer l'image de f , $\text{Im}(f)$. En donner une base et sa dimension.

L'image de f , $\text{Im}(f)$, est l'ensemble des vecteurs de $\mathbb{R}^3$ qui sont des images de vecteurs de $\mathbb{R}^2$ par f . $\text{Im}(f) = \{f(x, y) \mid (x, y) \in \mathbb{R}^2\}$. On peut écrire $f(x, y)$ comme une combinaison linéaire de vecteurs fixes :

$$f(x, y) = (x + y, x - y, 2x) = (x, x, 2x) + (y, -y, 0)$$

$$f(x, y) = x(1, 1, 2) + y(1, -1, 0)$$

Ceci signifie que l'image de f est l'espace vectoriel engendré par les vecteurs $v_1 = (1, 1, 2)$ et $v_2 = (1, -1, 0)$. $\text{Im}(f) = \text{Vect}((1, 1, 2), (1, -1, 0))$.

Pour trouver une base de $\text{Im}(f)$, nous devons vérifier si les vecteurs v_1 et v_2 sont linéairement indépendants. Soient $\lambda_1, \lambda_2 \in \mathbb{R}$ tels que $\lambda_1 v_1 + \lambda_2 v_2 = \mathbf{0}_{\mathbb{R}^3}$.

$$\lambda_1(1, 1, 2) + \lambda_2(1, -1, 0) = (0, 0, 0)$$

Ceci conduit au système d'équations : 1. $\lambda_1 + \lambda_2 = 0$ 2. $\lambda_1 - \lambda_2 = 0$ 3. $2\lambda_1 = 0$

De l'équation (3), nous obtenons $\lambda_1 = 0$. Substituons $\lambda_1 = 0$ dans l'équation (1) : $0 + \lambda_2 = 0 \implies \lambda_2 = 0$. Substituons $\lambda_1 = 0$ et $\lambda_2 = 0$ dans l'équation (2) : $0 - 0 = 0$, ce qui est consistant.

Puisque la seule solution est $\lambda_1 = 0$ et $\lambda_2 = 0$, les vecteurs $v_1 = (1, 1, 2)$ et $v_2 = (1, -1, 0)$ sont linéairement indépendants. Ils forment donc une base de $\text{Im}(f)$. Une base de $\text{Im}(f)$ est $\{(1, 1, 2), (1, -1, 0)\}$. La dimension de $\text{Im}(f)$, notée $\dim(\text{Im}(f))$, est le nombre de vecteurs dans sa base, soit 2. Le rang de f est $\text{rg}(f) = \dim(\text{Im}(f)) = 2$.

\{subsection*{4. Vérifier le théorème du rang.

Le théorème du rang stipule que pour une application linéaire $f : E \rightarrow F$, où E est un espace vectoriel de dimension finie, on a :

$$\dim(E) = \dim(\ker(f)) + \dim(\text{Im}(f))$$

Dans notre cas : L'espace de départ E est $\mathbb{R}^2$. Sa dimension est $\dim(\mathbb{R}^2) = 2$. Nous avons trouvé $\dim(\ker(f)) = 0$. Nous avons trouvé $\dim(\text{Im}(f)) = 2$.

Vérifions l'égalité : $\dim(\ker(f)) + \dim(\text{Im}(f)) = 0 + 2 = 2$. Cette somme est égale à la dimension de l'espace de départ $\mathbb{R}^2$.

$$2 = 2$$

Le théorème du rang est bien vérifié pour cette application linéaire.

\{\chapter\{\textit\{\{Exercice 2 : Application linéaire de dérivation (Difficulté :)

\{\section*\{Énoncé du problème

Soit $\mathbb{K}$ le corps des nombres réels, $\mathbb{K} = \mathbb{R}$. On considère l'espace vectoriel $E = \mathbb{R}_2[X]$ des polynômes à coefficients réels de degré au plus 2. Autrement dit, $E = \{a_0 + a_1X + a_2X^2 \mid a_0, a_1, a_2 \in \mathbb{R}\}$.

On définit l'application $D : E \rightarrow E$ par $D(P) = P'$, où P' désigne le polynôme dérivé de P .

1. Justifier que D est une application linéaire. 2. Déterminer le noyau de D , noté $\text{Ker}(D)$. En donner une base et sa dimension. 3. Déterminer l'image de D , notée $\text{Im}(D)$. En donner une base et sa dimension. 4. Vérifier le théorème du rang pour l'application D .

\{\section*\{Correction détaillée

\{\subsection*\{Question 1 : Justifier que D est une application linéaire.

Pour montrer que D est une application linéaire, nous devons vérifier deux propriétés : 1. Additivité : $D(P + Q) = D(P) + D(Q)$ pour tous $P, Q \in E$. 2. Homogénéité : $D(\lambda P) = \lambda D(P)$ pour tout $P \in E$ et tout scalaire $\lambda \in \mathbb{R}$.

Soient $P, Q \in E$. Par les propriétés de la dérivation des polynômes : * $D(P + Q) = (P + Q)' = P' + Q' = D(P) + D(Q)$. Soit $\lambda \in \mathbb{R}$. Par les propriétés de la dérivation des polynômes : * $D(\lambda P) = (\lambda P)' = \lambda P' = \lambda D(P)$.

Puisque ces deux propriétés sont satisfaites, l'application D est bien une application linéaire de E dans E .

\{\subsection*\{Question 2 : Déterminer le noyau de D , $\text{Ker}(D)$. En donner une base et sa dimension.

Par définition, le noyau de D est l'ensemble des polynômes $P \in E$ tels que $D(P) = 0_E$, où 0_E est le polynôme nul. $P \in \text{Ker}(D) \iff D(P) = 0_E \iff P' = 0_E$.

Soit un polynôme $P(X) \in E$, il s'écrit sous la forme $P(X) = a_0 + a_1X + a_2X^2$ avec $a_0, a_1, a_2 \in \mathbb{R}$. La dérivée de $P(X)$ est $P'(X) = a_1 + 2a_2X$.

Pour que $P'(X)$ soit le polynôme nul, il faut que tous ses coefficients soient nuls. Ainsi, $P'(X) = 0_E \iff a_1 = 0$ et $2a_2 = 0$. Cela implique $a_1 = 0$ et $a_2 = 0$.

Le polynôme $P(X)$ doit donc s'écrire $P(X) = a_0 + 0 \cdot X + 0 \cdot X^2 = a_0$, où a_0 est un réel quelconque. Le noyau de D est donc l'ensemble des polynômes constants : $\text{Ker}(D) = \{a_0 \mid a_0 \in \mathbb{R} = \mathbb{R}_0[X]\}$.

Pour trouver une base de $\text{Ker}(D)$, nous pouvons observer que tout polynôme constant $P(X) = a_0$ peut s'écrire $a_0 \cdot 1$. Le polynôme constant 1 est non nul et engendre $\text{Ker}(D)$. Une base de $\text{Ker}(D)$ est (1). La dimension de $\text{Ker}(D)$ est le nombre de vecteurs dans cette base, donc $\dim(\text{Ker}(D)) = 1$.

\{\subsection*\{Question 3 : Déterminer l'image de D , $\text{Im}(D)$. En donner une base et sa dimension.

Par définition, l'image de D est l'ensemble des polynômes $Q \in E$ pour lesquels il existe au moins un polynôme $P \in E$ tel que $D(P) = Q$. $Q \in \text{Im}(D) \iff \exists P \in E$ tel que $P' = Q$.

Soit un polynôme $P(X) = a_0 + a_1X + a_2X^2 \in E$. Son image par D est $D(P) = P'(X) = a_1 + 2a_2X$. Tout polynôme dans l'image est donc de la forme $a_1 + 2a_2X$. Ces polynômes sont de degré au plus 1. L'ensemble de ces polynômes est l'espace vectoriel $\mathbb{R}_1[X]$. Tout polynôme de $\mathbb{R}_1[X]$, disons $Q(X) = b_0 + b_1X$, peut être obtenu comme la dérivée d'un polynôme de $\mathbb{R}_2[X]$. Par exemple, $P(X) = b_0X + \frac{b_1}{2}X^2$ est dans $\mathbb{R}_2[X]$ et $P'(X) = b_0 + b_1X = Q(X)$. (Nous pouvons choisir $a_0 = 0$ et ajuster a_1, a_2 pour obtenir n'importe quel $b_0 + b_1X$).

Ainsi, $\text{Im}(D) = \{a_1 + 2a_2X \mid a_1, a_2 \in \mathbb{R} = \mathbb{R}_1[X]\}$.

Pour trouver une base de $\text{Im}(D)$, nous cherchons un ensemble de polynômes qui engendrent $\mathbb{R}_1[X]$ et sont linéairement indépendants. Les polynômes 1 et X forment une base standard de $\mathbb{R}_1[X]$. * 1 est la dérivée de X (qui est dans $\mathbb{R}_2[X]$). * X est la dérivée de $\frac{1}{2}X^2$ (qui est dans $\mathbb{R}_2[X]$). Une base de $\text{Im}(D)$ est (1, X). La dimension de $\text{Im}(D)$ est le nombre de vecteurs dans cette base, donc $\dim(\text{Im}(D)) = 2$.

\{\subsection*\{Question 4 : Vérifier le théorème du rang pour l'application D .

Le théorème du rang stipule que pour une application linéaire $D : E \rightarrow F$, on a la relation : $\dim(E) = \dim(\text{Ker}(D)) + \dim(\text{Im}(D))$.

Nous avons l'espace de départ $E = \mathbb{R}_2[X]$. Une base canonique de $\mathbb{R}_2[X]$ est (1, X, X^2). Le nombre d'éléments dans cette base est 3, donc $\dim(E) = 3$.

D'après la question 2, nous avons trouvé $\dim(\text{Ker}(D)) = 1$. D'après la question 3, nous avons trouvé $\dim(\text{Im}(D)) = 2$.

Calculons la somme de ces dimensions : $\dim(\text{Ker}(D)) + \dim(\text{Im}(D)) = 1 + 2 = 3$.

Nous observons que $\dim(E) = 3$ et $\dim(\text{Ker}(D)) + \dim(\text{Im}(D)) = 3$. L'égalité est vérifiée : $\dim(E) = \dim(\text{Ker}(D)) + \dim(\text{Im}(D))$. Le théorème du rang est bien vérifié pour l'application linéaire D .

\{\chapter\{\textit\{\{Exercice 3 : Noyau, Image, Théorème du Rang avec Paramètre (Difficulté : *)

Soit $\mathbb{K} = \mathbb{R}$ le corps des nombres réels. Soit $E = \mathbb{R}^3$ l'espace vectoriel des triplets de nombres réels, muni de sa base canonique $\mathcal{B} = (e_1, e_2, e_3)$, où $e_1 = (1, 0, 0)$, $e_2 = (0, 1, 0)$ et $e_3 = (0, 0, 1)$.

On considère l'application linéaire $f_a : E \rightarrow E$ définie, pour tout $a \in \mathbb{R}$, par sa matrice M_a dans la base canonique $\mathcal{B}$:

$$M_a = \begin{pmatrix} 1 & 1 & 1 \\ 1 & a & 1 \\ 1 & 1 & a \end{pmatrix}$$

1. Déterminer les valeurs du paramètre $a \in \mathbb{R}$ pour lesquelles l'application f_a est un automorphisme de E .
2. Pour les valeurs de a pour lesquelles f_a n'est pas un automorphisme, déterminer une base du noyau $\ker(f_a)$ et une base de l'image $\text{Im}(f_a)$.
3. Dans tous les cas (que f_a soit un automorphisme ou non), vérifier le théorème du rang pour l'application f_a .

\{\section*\{Correction détaillée

\{\subsection*\{Question 1 : Détermination des valeurs de a pour lesquelles f_a est un automorphisme

Une application linéaire $f : E \rightarrow E$ est un automorphisme si et seulement si sa matrice représentative dans une base donnée est inversible. Pour une matrice carrée $M_a \in \mathcal{M}_3(\mathbb{R})$, cela équivaut à $\det(M_a) \neq 0$.

Calculons le déterminant de M_a :

$$M_a = \begin{pmatrix} 1 & 1 & 1 \\ 1 & a & 1 \\ 1 & 1 & a \end{pmatrix}$$

Nous pouvons effectuer des opérations élémentaires sur les lignes qui ne modifient pas le déterminant. Pour simplifier le calcul, nous allons rendre nuls les éléments sous le pivot de la première colonne. Appliquons les opérations suivantes : $L_2 \leftarrow L_2 - L_1$ et $L_3 \leftarrow L_3 - L_1$.

$$\det(M_a) = \det \begin{pmatrix} 1 & 1 & 1 \\ 1-1 & a-1 & 1-1 \\ 1-1 & 1-1 & a-1 \end{pmatrix} = \det \begin{pmatrix} 1 & 1 & 1 \\ 0 & a-1 & 0 \\ 0 & 0 & a-1 \end{pmatrix}$$

Cette dernière matrice est triangulaire supérieure. Le déterminant d'une matrice triangulaire est égal au produit de ses éléments diagonaux.

$$\det(M_a) = 1 \cdot (a-1) \cdot (a-1) = (a-1)^2$$

L'application f_a est un automorphisme si et seulement si $\det(M_a) \neq 0$. L'équation $(a-1)^2 \neq 0$ est vérifiée si et seulement si $a-1 \neq 0$, ce qui implique $a \neq 1$.

Par conséquent, f_a est un automorphisme de E si et seulement si $a \neq 1$.

\{\subsection*\{Question 2 : Bases du noyau et de l'image pour f_a lorsque f_a n'est pas un automorphisme

D'après la question 1, f_a n'est pas un automorphisme si et seulement si $a = 1$. Nous allons donc étudier ce cas particulier.

Pour $a = 1$, la matrice de l'application f_1 est obtenue en substituant $a = 1$ dans M_a :

$$M_1 = \begin{pmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{pmatrix}$$

\{\subsubsection*\{Détermination d'une base du noyau $\ker(f_1)$

Le noyau $\ker(f_1)$ est l'ensemble des vecteurs $X = (x, y, z)^T \in \mathbb{R}^3$ tels que $f_1(X) = 0_E$. Cela est équivalent à résoudre le système linéaire homogène $M_1 X = 0$:

$$\begin{pmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{pmatrix} \begin{pmatrix} x \\ y \\ z \end{pmatrix} = \begin{pmatrix} 0 \\ 0 \\ 0 \end{pmatrix}$$

Ce système de trois équations à trois inconnues se réduit à une seule équation linéaire car toutes les lignes sont identiques :

$$x + y + z = 0$$

Les solutions de cette équation décrivent un plan dans $\mathbb{R}^3$. Nous pouvons exprimer une variable en fonction des deux autres, par exemple $z = -x - y$. Un vecteur $X \in \ker(f_1)$ s'écrit alors :

$$X = \begin{pmatrix} x \\ y \\ -x - y \end{pmatrix}$$

Nous pouvons décomposer ce vecteur comme une combinaison linéaire de vecteurs fixes :

$$X = x \begin{pmatrix} 1 \\ 0 \\ -1 \end{pmatrix} + y \begin{pmatrix} 0 \\ 1 \\ -1 \end{pmatrix}$$

Les vecteurs $v_1 = (1, 0, -1)^T$ et $v_2 = (0, 1, -1)^T$ engendrent le noyau $\ker(f_1)$. Pour vérifier s'ils forment une base, il faut montrer qu'ils sont linéairement indépendants. Supposons qu'il existe des scalaires $\alpha, \beta \in \mathbb{R}$ tels que $\alpha v_1 + \beta v_2 = 0_E$.

$$\alpha \begin{pmatrix} 1 \\ 0 \\ -1 \end{pmatrix} + \beta \begin{pmatrix} 0 \\ 1 \\ -1 \end{pmatrix} = \begin{pmatrix} \alpha \\ \beta \\ -\alpha - \beta \end{pmatrix} = \begin{pmatrix} 0 \\ 0 \\ 0 \end{pmatrix}$$

Ceci implique $\alpha = 0$, $\beta = 0$ et $-\alpha - \beta = 0$. Les deux premières équations suffisent à montrer que $\alpha = 0$ et $\beta = 0$. Les vecteurs v_1 et v_2 sont donc linéairement indépendants.

Ainsi, une base du noyau de f_1 est $\mathcal{B}_{\ker(f_1)} = \{(1, 0, -1), (0, 1, -1)\}$. La dimension du noyau est $\dim(\ker(f_1)) = 2$.

\{\subsubsection*{Détermination d'une base de l'image $\text{Im}(f_1)$ }

L'image $\text{Im}(f_1)$ est l'espace vectoriel engendré par les colonnes de la matrice M_1 . Les colonnes de M_1 sont :

$$C_1 = \begin{pmatrix} 1 \\ 1 \\ 1 \end{pmatrix}, \quad C_2 = \begin{pmatrix} 1 \\ 1 \\ 1 \end{pmatrix}, \quad C_3 = \begin{pmatrix} 1 \\ 1 \\ 1 \end{pmatrix}$$

Ces trois vecteurs colonnes sont identiques. Par conséquent, l'image est engendrée par n'importe lequel de ces vecteurs, puisque tous les autres sont des multiples de celui-ci. $\text{Im}(f_1) = \text{Vect} \left(\begin{pmatrix} 1 \\ 1 \\ 1 \end{pmatrix} \right)$. Le vecteur $w_1 = (1, 1, 1)^T$

est non nul, il forme donc à lui seul une base de l'image.

Ainsi, une base de l'image de f_1 est $\mathcal{B}_{\text{Im}(f_1)} = \{(1, 1, 1)\}$. La dimension de l'image est $\dim(\text{Im}(f_1)) = 1$.

\{\subsection*{Question 3 : Vérification du théorème du rang dans tous les cas}

Le théorème du rang stipule que pour toute application linéaire $f : E \rightarrow F$, on a la relation : $\dim(E) = \dim(\ker(f)) + \text{rang}(f)$, où $\text{rang}(f) = \dim(\text{Im}(f))$. Dans notre cas, l'espace de départ $E = \mathbb{R}^3$, donc $\dim(E) = 3$.

\{\subsubsection*{Cas 1 : $a = 1$ (où f_1 n'est pas un automorphisme)}

D'après les calculs de la question 2 : Nous avons trouvé $\dim(\ker(f_1)) = 2$. Nous avons trouvé $\dim(\text{Im}(f_1)) = 1$.

Vérifions le théorème du rang pour f_1 : $\dim(E) = \dim(\ker(f_1)) + \dim(\text{Im}(f_1))$ $3 = 2 + 1$ $3 = 3$ Le théorème du rang est bien vérifié pour $a = 1$.

\{\subsubsection*{Cas 2 : $a \neq 1$ (où f_a est un automorphisme)}

D'après la question 1, si $a \neq 1$, l'application f_a est un automorphisme de E . Par définition, un automorphisme est une application linéaire bijective. - Puisque f_a est injective, son noyau est réduit au seul vecteur nul de E : $\ker(f_a) = \{0_E\}$. Par conséquent, la dimension du noyau est $\dim(\ker(f_a)) = 0$. - Puisque f_a est surjective (et qu'elle va de E vers E), son image est égale à l'espace d'arrivée E : $\text{Im}(f_a) = E$. Par conséquent, la dimension de l'image est $\dim(\text{Im}(f_a)) = \dim(E) = 3$.

Vérifions le théorème du rang pour f_a lorsque $a \neq 1$: $\dim(E) = \dim(\ker(f_a)) + \dim(\text{Im}(f_a))$ $3 = 0 + 3$ $3 = 3$ Le théorème du rang est bien vérifié pour $a \neq 1$.

Le théorème du rang est donc vérifié pour toutes les valeurs de $a \in \mathbb{R}$.

\{\chapter\{\textit{\{Exercice 4 : Noyau, Image et Théorème du Rang (Difficulté : *)

\{\section*\{Énoncé du problème

Soit $\mathbb{K}$ le corps des nombres réels, $\mathbb{K} = \mathbb{R}$. On considère les $\mathbb{K}$ -espaces vectoriels $E = \mathbb{R}^3$ et $F = \mathbb{R}^2$. Soit f l'application de E vers F définie pour tout $\mathbf{v} = (x, y, z) \in E$ par :

$$f(x, y, z) = (x - y, y - z)$$

On admettra que f est une application linéaire.

1. Déterminer une base et la dimension du noyau de f , noté $\text{Ker}(f)$. 2. Déterminer une base et la dimension de l'image de f , notée $\text{Im}(f)$. 3. Vérifier le théorème du rang pour l'application linéaire f .

\{\section*\{Correction détaillée

\{\subsection*\{Question 1 : Détermination du noyau de f

Le noyau de f , $\text{Ker}(f)$, est l'ensemble des vecteurs $\mathbf{v} \in E$ tels que $f(\mathbf{v}) = \mathbf{0}_F$, où $\mathbf{0}_F$ est le vecteur nul de F . Soit $\mathbf{v} = (x, y, z) \in \mathbb{R}^3$. L'équation $f(x, y, z) = (0, 0)$ se traduit par le système d'équations linéaires suivant :

$$\begin{cases} x - y = 0 \\ y - z = 0 \end{cases}$$

De la première équation, nous déduisons $x = y$. De la seconde équation, nous déduisons $y = z$. Par conséquent, tout vecteur $\mathbf{v}$ appartenant à $\text{Ker}(f)$ doit satisfaire la condition $x = y = z$. Les vecteurs du noyau sont donc de la forme (x, x, x) pour tout $x \in \mathbb{R}$. On peut écrire $(x, x, x) = x(1, 1, 1)$. Le noyau est donc l'ensemble des multiples scalaires du vecteur $(1, 1, 1)$. Ainsi, $\text{Ker}(f) = \text{Vect}((1, 1, 1))$. Le vecteur $(1, 1, 1)$ est non nul et engendre $\text{Ker}(f)$, il forme donc une base de $\text{Ker}(f)$. Une base pour $\text{Ker}(f)$ est $B_{\text{Ker}(f)} = \{(1, 1, 1)\}$. La dimension du noyau de f est $\dim(\text{Ker}(f)) = 1$.

\{\subsection*\{Question 2 : Détermination de l'image de f

L'image de f , $\text{Im}(f)$, est l'ensemble des vecteurs $\mathbf{w} \in F$ pour lesquels il existe un vecteur $\mathbf{v} \in E$ tel que $f(\mathbf{v}) = \mathbf{w}$. L'image d'une application linéaire est engendrée par les images des vecteurs d'une base de l'espace de départ E . Considérons la base canonique de $E = \mathbb{R}^3$, notée $B_E = \{\mathbf{e}_1, \mathbf{e}_2, \mathbf{e}_3\}$, où $\mathbf{e}_1 = (1, 0, 0)$, $\mathbf{e}_2 = (0, 1, 0)$, $\mathbf{e}_3 = (0, 0, 1)$.

Calculons les images de ces vecteurs : * $f(\mathbf{e}_1) = f(1, 0, 0) = (1 - 0, 0 - 0) = (1, 0)$. * $f(\mathbf{e}_2) = f(0, 1, 0) = (0 - 1, 1 - 0) = (-1, 1)$. * $f(\mathbf{e}_3) = f(0, 0, 1) = (0 - 0, 0 - 1) = (0, -1)$.

L'image de f est donc l'espace vectoriel engendré par ces trois vecteurs :

$$\text{Im}(f) = \text{Vect}((1, 0), (-1, 1), (0, -1))$$

Ces vecteurs appartiennent à $F = \mathbb{R}^2$. L'espace $\mathbb{R}^2$ est de dimension 2, donc l'image de f ne peut pas avoir une dimension supérieure à 2. Nous devons extraire une base de l'ensemble de vecteurs $\{(1, 0), (-1, 1), (0, -1)\}$. Considérons les deux premiers vecteurs : $\mathbf{u}_1 = (1, 0)$ et $\mathbf{u}_2 = (-1, 1)$. Vérifions s'ils sont linéairement indépendants. Soient $\alpha, \beta \in \mathbb{R}$ tels que $\alpha\mathbf{u}_1 + \beta\mathbf{u}_2 = \mathbf{0}_F$:

$$\alpha(1, 0) + \beta(-1, 1) = (0, 0)$$

$$(\alpha - \beta, \beta) = (0, 0)$$

Ce qui implique $\beta = 0$ et $\alpha - \beta = 0 \implies \alpha = 0$. Les vecteurs $(1, 0)$ et $(-1, 1)$ sont donc linéairement indépendants. Puisqu'ils sont deux vecteurs linéairement indépendants dans l'espace de dimension 2, $F = \mathbb{R}^2$, ils forment une base de $\mathbb{R}^2$. Par conséquent, $\text{Im}(f) = \mathbb{R}^2$. Une base pour $\text{Im}(f)$ est $B_{\text{Im}(f)} = \{(1, 0), (-1, 1)\}$. La dimension de l'image de f est $\dim(\text{Im}(f)) = 2$. Le rang de f est $\text{rg}(f) = \dim(\text{Im}(f)) = 2$.

\{\subsection*\{Question 3 : Vérification du théorème du rang

Le théorème du rang stipule que pour toute application linéaire $f : E \rightarrow F$, la dimension de l'espace de départ E est égale à la somme de la dimension de son noyau et de la dimension de son image :

$$\dim(E) = \dim(\text{Ker}(f)) + \dim(\text{Im}(f))$$

Dans notre cas : * La dimension de l'espace de départ $E = \mathbb{R}^3$ est $\dim(E) = 3$. * D'après la Question 1, la dimension du noyau est $\dim(\text{Ker}(f)) = 1$. * D'après la Question 2, la dimension de l'image est $\dim(\text{Im}(f)) = 2$.

Vérifions l'égalité :

$$3 = 1 + 2$$

L'égalité est satisfaite. Le théorème du rang est vérifié pour l'application linéaire f .

\chapter{\textit{Exercice 5 : Noyau, Image et Rang de la composition d'applications linéaires (Difficulté : \textit{}}\textit{}}\textit{}

\textbf{Énoncé du Problème}

Soient K un corps commutatif. Soient E, F, G trois K -espaces vectoriels de dimensions finies. Soient $f : E \rightarrow F$ et $g : F \rightarrow G$ deux applications linéaires.

1. Démontrer que $\ker(f) \subseteq \ker(g \circ f)$ et que $\operatorname{Im}(g \circ f) \subseteq \operatorname{Im}(g)$. 2. On considère l'application linéaire restreinte $g_f : \operatorname{Im}(f) \rightarrow G$ définie par $g_f(y) = g(y)$ pour tout $y \in \operatorname{Im}(f)$. a) Démontrer que $\ker(g_f) = \operatorname{Im}(f) \cap \ker(g)$. b) Démontrer que $\operatorname{Im}(g_f) = \operatorname{Im}(g \circ f)$. 3. En utilisant le théorème du rang pour l'application g_f et l'application f , établir la relation suivante :

$$\dim(\ker(g \circ f)) = \dim(\ker(f)) + \dim(\operatorname{Im}(f) \cap \ker(g))$$

4. En déduire la relation liant les rangs des applications linéaires :

$$\operatorname{rg}(g \circ f) = \operatorname{rg}(f) - \dim(\operatorname{Im}(f) \cap \ker(g))$$

\section*{Correction détaillée}

1. \textbf{Démonstration de $\ker(f) \subseteq \ker(g \circ f)$ et $\operatorname{Im}(g \circ f) \subseteq \operatorname{Im}(g)$ }

* \textbf{Pour $\ker(f) \subseteq \ker(g \circ f)$:} Soit $x \in \ker(f)$. Par définition du noyau, cela signifie que $f(x) = 0_F$, où 0_F est le vecteur nul de F . Appliquons l'application g à $f(x)$: $(g \circ f)(x) = g(f(x)) = g(0_F)$. Puisque g est une application linéaire, elle envoie le vecteur nul sur le vecteur nul : $g(0_F) = 0_G$. Ainsi, $(g \circ f)(x) = 0_G$. Par définition du noyau, $x \in \ker(g \circ f)$. Donc, tout élément de $\ker(f)$ est aussi un élément de $\ker(g \circ f)$, ce qui prouve $\ker(f) \subseteq \ker(g \circ f)$.

* \textbf{Pour $\operatorname{Im}(g \circ f) \subseteq \operatorname{Im}(g)$:} Soit $z \in \operatorname{Im}(g \circ f)$. Par définition de l'image, cela signifie qu'il existe un vecteur $x \in E$ tel que $z = (g \circ f)(x)$. On peut réécrire $z = g(f(x))$. Posons $y = f(x)$. Alors, par définition de l'image de f , $y \in \operatorname{Im}(f)$. Nous avons $z = g(y)$. Puisque $y \in F$ et $z = g(y)$, par définition de l'image de g , $z \in \operatorname{Im}(g)$. Donc, tout élément de $\operatorname{Im}(g \circ f)$ est aussi un élément de $\operatorname{Im}(g)$, ce qui prouve $\operatorname{Im}(g \circ f) \subseteq \operatorname{Im}(g)$.

2. \textbf{Propriétés de l'application linéaire restreinte g_f }

L'application $g_f : \operatorname{Im}(f) \rightarrow G$ est définie par $g_f(y) = g(y)$ pour tout $y \in \operatorname{Im}(f)$. Il est important de noter que $\operatorname{Im}(f)$ est un sous-espace vectoriel de F , et g_f est la restriction de g à ce sous-espace.

a) \textbf{Démonstration de $\ker(g_f) = \operatorname{Im}(f) \cap \ker(g)$ }

* \textbf{Inclusion $\subseteq$:} Soit $y \in \ker(g_f)$. Par définition du noyau de g_f , cela signifie que $y \in \operatorname{Im}(f)$ et $g_f(y) = 0_G$. Par définition de g_f , on a $g_f(y) = g(y)$. Donc, $g(y) = 0_G$. Puisque $g(y) = 0_G$, par définition du noyau de g , on a $y \in \ker(g)$. Ainsi, y appartient à la fois à $\operatorname{Im}(f)$ et à $\ker(g)$. Donc $y \in \operatorname{Im}(f) \cap \ker(g)$. Ceci prouve $\ker(g_f) \subseteq \operatorname{Im}(f) \cap \ker(g)$.

* \textbf{Inclusion $\supseteq$:} Soit $y \in \operatorname{Im}(f) \cap \ker(g)$. Par définition de l'intersection, cela signifie que $y \in \operatorname{Im}(f)$ et $y \in \ker(g)$. Puisque $y \in \operatorname{Im}(f)$, l'application $g_f(y)$ est bien définie. Puisque $y \in \ker(g)$, par définition du noyau de g , on a $g(y) = 0_G$. Par définition de g_f , on a $g_f(y) = g(y)$. Donc, $g_f(y) = 0_G$. Ainsi, $y \in \ker(g_f)$. Ceci prouve $\operatorname{Im}(f) \cap \ker(g) \subseteq \ker(g_f)$.

Les deux inclusions étant établies, on conclut que $\ker(g_f) = \operatorname{Im}(f) \cap \ker(g)$.

b) \textbf{Démonstration de $\operatorname{Im}(g_f) = \operatorname{Im}(g \circ f)$ }

* \textbf{Inclusion $\subseteq$:} Soit $z \in \operatorname{Im}(g_f)$. Par définition de l'image de g_f , cela signifie qu'il existe un vecteur $y \in \operatorname{Im}(f)$ tel que $z = g_f(y)$. Par définition de g_f , on a $z = g(y)$. Puisque $y \in \operatorname{Im}(f)$, par définition de l'image de f , il existe un vecteur $x \in E$ tel que $y = f(x)$. En substituant y , nous obtenons $z = g(f(x))$. Par définition de la composition d'applications, $z = (g \circ f)(x)$. Puisque $x \in E$ et $z = (g \circ f)(x)$, par définition de l'image de $g \circ f$, on a $z \in \operatorname{Im}(g \circ f)$. Ceci prouve $\operatorname{Im}(g_f) \subseteq \operatorname{Im}(g \circ f)$.

* \textbf{Inclusion $\supseteq$:} Soit $z \in \operatorname{Im}(g \circ f)$. Par définition de l'image de $g \circ f$, cela signifie qu'il existe un vecteur $x \in E$ tel que $z = (g \circ f)(x)$. On peut réécrire $z = g(f(x))$. Posons $y = f(x)$. Alors, par définition de l'image de f , $y \in \operatorname{Im}(f)$. Puisque $y \in \operatorname{Im}(f)$, l'application $g_f(y)$ est bien définie, et par définition $g_f(y) = g(y)$. Ainsi, $z = g(f(x)) = g(y) = g_f(y)$. Puisque $y \in \operatorname{Im}(f)$ et $z = g_f(y)$, par définition de l'image de g_f , on a $z \in \operatorname{Im}(g_f)$. Ceci prouve $\operatorname{Im}(g \circ f) \subseteq \operatorname{Im}(g_f)$.

Les deux inclusions étant établies, on conclut que $\operatorname{Im}(g_f) = \operatorname{Im}(g \circ f)$.

3. \textbf{Établissement de la relation $\dim(\ker(g \circ f)) = \dim(\ker(f)) + \dim(\operatorname{Im}(f) \cap \ker(g))$ }

Nous allons appliquer le théorème du rang aux applications linéaires f et g_f .

* \textbf{Théorème du rang pour $g_f : \operatorname{Im}(f) \rightarrow G$:} Puisque $\operatorname{Im}(f)$ est un sous-espace vectoriel de F et est de dimension finie (car F est de dimension finie), le théorème du rang s'applique à g_f .

$$\dim(\operatorname{Im}(f)) = \dim(\ker(g_f)) + \dim(\operatorname{Im}(g_f))$$

En utilisant les résultats de la question 2a) et 2b) :

$$\dim(\operatorname{Im}(f)) = \dim(\operatorname{Im}(f) \cap \ker(g)) + \dim(\operatorname{Im}(g \circ f)) \quad (*)$$

* \textbf{Théorème du rang pour } $f : E \rightarrow F$:

$$\dim(E) = \dim(\ker(f)) + \dim(\operatorname{Im}(f))$$

On peut exprimer $\dim(\operatorname{Im}(f))$ comme :

$$\dim(\operatorname{Im}(f)) = \dim(E) - \dim(\ker(f)) \quad (**)$$

* \textbf{Théorème du rang pour } $g \circ f : E \rightarrow G$:

$$\dim(E) = \dim(\ker(g \circ f)) + \dim(\operatorname{Im}(g \circ f))$$

On peut exprimer $\dim(\operatorname{Im}(g \circ f))$ comme :

$$\dim(\operatorname{Im}(g \circ f)) = \dim(E) - \dim(\ker(g \circ f)) \quad (***)$$

Substituons (**) et (***) dans l'équation (*) :

$$(\dim(E) - \dim(\ker(f))) = \dim(\operatorname{Im}(f) \cap \ker(g)) + (\dim(E) - \dim(\ker(g \circ f)))$$

Simplifions en soustrayant $\dim(E)$ des deux côtés :

$$-\dim(\ker(f)) = \dim(\operatorname{Im}(f) \cap \ker(g)) - \dim(\ker(g \circ f))$$

Réarrangeons les termes pour isoler $\dim(\ker(g \circ f))$:

$$\dim(\ker(g \circ f)) = \dim(\ker(f)) + \dim(\operatorname{Im}(f) \cap \ker(g))$$

Ceci établit la relation demandée.

4. \textbf{Dédution de la relation } $\operatorname{rg}(g \circ f) = \operatorname{rg}(f) - \dim(\operatorname{Im}(f) \cap \ker(g))$

Par définition, le rang d'une application linéaire est la dimension de son image. Donc $\operatorname{rg}(f) = \dim(\operatorname{Im}(f))$ et $\operatorname{rg}(g \circ f) = \dim(\operatorname{Im}(g \circ f))$.

Reprenons l'équation (*) établie à la question 3 :

$$\dim(\operatorname{Im}(f)) = \dim(\operatorname{Im}(f) \cap \ker(g)) + \dim(\operatorname{Im}(g \circ f))$$

En substituant les termes par leurs rangs correspondants :

$$\operatorname{rg}(f) = \dim(\operatorname{Im}(f) \cap \ker(g)) + \operatorname{rg}(g \circ f)$$

Pour obtenir la relation souhaitée, il suffit de réarranger les termes :

$$\operatorname{rg}(g \circ f) = \operatorname{rg}(f) - \dim(\operatorname{Im}(f) \cap \ker(g))$$

Ceci conclut la déduction.

\chapter{\textit{Exercice 6 : L'opérateur de différence finie sur les polynômes (Difficulté :)}}{section*{Énoncé du problème}}

Soit $n \in \mathbb{N}$ un entier naturel. On considère l'espace vectoriel réel $E = \mathbb{R}_n[X]$, l'ensemble des polynômes à coefficients réels de degré inférieur ou égal à n . La base canonique de E est $B = (1, X, X^2, \dots, X^n)$.

On définit l'application $L : E \rightarrow E$ par :

$$L(P)(X) = P(X+1) - P(X)$$

pour tout polynôme $P \in E$.

1. Démontrer que L est une application linéaire. 2. Déterminer le noyau $\text{Ker}(L)$ de L . Donner une base de $\text{Ker}(L)$ et sa dimension. 3. Déterminer l'image $\text{Im}(L)$ de L . Donner une base de $\text{Im}(L)$ et sa dimension. 4. Calculer le rang de L et vérifier le théorème du rang. 5. Déterminer la matrice $M_B(L)$ de L dans la base canonique B de E .

—
\section*{Correction détaillée}

\subsection*{Question 1 : Démontrer que L est une application linéaire.}

Pour que L soit linéaire, elle doit satisfaire deux propriétés : a) $L(P+Q) = L(P) + L(Q)$ pour tous $P, Q \in E$.

b) $L(\lambda P) = \lambda L(P)$ pour tout $P \in E$ et tout scalaire $\lambda \in \mathbb{R}$.

Soient $P, Q \in E$ et $\lambda \in \mathbb{R}$.

a) Calculons $L(P+Q)(X)$:

$$L(P+Q)(X) = (P+Q)(X+1) - (P+Q)(X)$$

Par définition de l'addition de polynômes, $(P+Q)(X+1) = P(X+1) + Q(X+1)$ et $(P+Q)(X) = P(X) + Q(X)$. Donc,

$$L(P+Q)(X) = (P(X+1) + Q(X+1)) - (P(X) + Q(X))$$

Regroupons les termes :

$$L(P+Q)(X) = (P(X+1) - P(X)) + (Q(X+1) - Q(X))$$

Par définition de L , ceci est égal à $L(P)(X) + L(Q)(X)$. Ainsi, $L(P+Q) = L(P) + L(Q)$.

b) Calculons $L(\lambda P)(X)$:

$$L(\lambda P)(X) = (\lambda P)(X+1) - (\lambda P)(X)$$

Par définition de la multiplication d'un polynôme par un scalaire, $(\lambda P)(X+1) = \lambda P(X+1)$ et $(\lambda P)(X) = \lambda P(X)$. Donc,

$$L(\lambda P)(X) = \lambda P(X+1) - \lambda P(X)$$

Mettons λ en facteur :

$$L(\lambda P)(X) = \lambda(P(X+1) - P(X))$$

Par définition de L , ceci est égal à $\lambda L(P)(X)$. Ainsi, $L(\lambda P) = \lambda L(P)$.

Les deux propriétés étant vérifiées, L est bien une application linéaire. De plus, si $P \in \mathbb{R}_n[X]$, alors $P(X+1)$ est également un polynôme de degré au plus n . La différence $P(X+1) - P(X)$ est donc un polynôme de degré au plus n . L'application L est donc bien définie de E vers E .

\subsection*{Question 2 : Déterminer le noyau $\text{Ker}(L)$.}

Le noyau $\text{Ker}(L)$ est l'ensemble des polynômes $P \in E$ tels que $L(P) = 0$.

$$P \in \text{Ker}(L) \iff L(P)(X) = 0 \text{ pour tout } X \in \mathbb{R}$$

$$P(X+1) - P(X) = 0 \iff P(X+1) = P(X)$$

Un polynôme qui vérifie $P(X+1) = P(X)$ pour tout $X \in \mathbb{R}$ est un polynôme constant. Pour le prouver, soit $P(X) = a_k X^k + a_{k-1} X^{k-1} + \dots + a_1 X + a_0$ un polynôme de degré $k \leq n$, avec $a_k \neq 0$ si $k \geq 1$. Alors $P(X+1) = a_k (X+1)^k + a_{k-1} (X+1)^{k-1} + \dots + a_1 (X+1) + a_0$. Développons $(X+1)^k : (X+1)^k = X^k + kX^{k-1} + \binom{k}{2} X^{k-2} + \dots + 1$.

$$P(X+1) = a_k (X^k + kX^{k-1} + \dots) + a_{k-1} (X^{k-1} + \dots) + \dots + a_0$$

$$P(X+1) - P(X) = a_k k X^{k-1} + (\text{termes de degré inférieur})$$

Si $k \geq 1$, alors le polynôme $P(X+1) - P(X)$ a pour degré $k-1$ et son coefficient dominant est $a_k k$. Pour que $P(X+1) - P(X) = 0$, il faut que tous ses coefficients soient nuls. En particulier, $a_k k$ doit être nul. Puisque $a_k \neq 0$ (par hypothèse P est de degré k), il faut que $k = 0$. Si $k = 0$, le polynôme $P(X)$ est de degré 0, c'est-à-dire une constante $P(X) = a_0$. Dans ce cas, $L(a_0) = a_0 - a_0 = 0$. Donc, les seuls polynômes dans E qui

appartiennent à $\text{Ker}(L)$ sont les polynômes constants. L'ensemble des polynômes constants dans E est $\mathbb{R}_0[X]$. Une base pour $\mathbb{R}_0[X]$ est le polynôme constant 1. Par conséquent, $\text{Ker}(L) = \text{Vect}(1)$. La dimension de $\text{Ker}(L)$ est $\dim(\text{Ker}(L)) = 1$.

\{subsection*\{Question 3 : Déterminer l'image $\text{Im}(L)$.

L'image $\text{Im}(L)$ est l'ensemble des polynômes $Q \in E$ tels que $Q = L(P)$ pour un certain $P \in E$. D'après la question 2, si $P(X)$ est un polynôme de degré $k \geq 1$, alors $L(P)(X)$ est un polynôme de degré $k - 1$. Si $P(X)$ est un polynôme constant (de degré 0), $L(P)(X) = 0$. Puisque $P \in \mathbb{R}_n[X]$, le degré maximal de P est n . Si P est de degré n , $L(P)$ est de degré $n - 1$. Si P est de degré $k < n$, $L(P)$ est de degré $k - 1$ (si $k \geq 1$) ou 0 (si $k = 0$). Donc, tous les polynômes dans $\text{Im}(L)$ ont un degré inférieur ou égal à $n - 1$. Cela signifie que $\text{Im}(L) \subseteq \mathbb{R}_{n-1}[X]$.

Nous connaissons la dimension de l'espace de départ $E = \mathbb{R}_n[X]$, qui est $\dim(E) = n + 1$. Nous avons trouvé $\dim(\text{Ker}(L)) = 1$. D'après le théorème du rang (voir question 4), $\dim(\text{Im}(L)) = \dim(E) - \dim(\text{Ker}(L)) = (n + 1) - 1 = n$.

Nous avons donc $\text{Im}(L) \subseteq \mathbb{R}_{n-1}[X]$ et $\dim(\text{Im}(L)) = n$. Puisque $\dim(\mathbb{R}_{n-1}[X]) = n$, et que $\text{Im}(L)$ est un sous-espace de $\mathbb{R}_{n-1}[X]$ de même dimension, nous pouvons conclure que $\text{Im}(L) = \mathbb{R}_{n-1}[X]$.

Une base pour $\text{Im}(L) = \mathbb{R}_{n-1}[X]$ est la base canonique $(1, X, X^2, \dots, X^{n-1})$. La dimension de $\text{Im}(L)$ est n .

\{subsection*\{Question 4 : Calculer le rang de L et vérifier le théorème du rang.

Le rang d'une application linéaire L , noté $\text{rang}(L)$, est la dimension de son image $\text{Im}(L)$. D'après la question 3, $\dim(\text{Im}(L)) = n$. Donc, $\text{rang}(L) = n$.

Le théorème du rang stipule que pour une application linéaire $L : E \rightarrow F$, où E est un espace vectoriel de dimension finie, on a :

$$\dim(E) = \dim(\text{Ker}(L)) + \dim(\text{Im}(L))$$

Dans notre cas : * $\dim(E) = \dim(\mathbb{R}_n[X]) = n + 1$. * $\dim(\text{Ker}(L)) = 1$ (d'après la question 2). * $\dim(\text{Im}(L)) = n$ (d'après la question 3).

Vérifions le théorème du rang :

$$(n + 1) = 1 + n$$

L'égalité est bien vérifiée.

\{subsection*\{Question 5 : Déterminer la matrice $M_B(L)$ de L dans la base canonique B de E .

La base canonique de $E = \mathbb{R}_n[X]$ est $B = (P_0, P_1, \dots, P_n)$ où $P_k(X) = X^k$. Pour construire la matrice $M_B(L)$, nous devons calculer $L(P_k)$ pour chaque $k \in \{0, \dots, n\}$ et exprimer le résultat comme une combinaison linéaire des éléments de la base B . Le vecteur colonne j de la matrice sera constitué des coefficients de $L(P_j)$.

1. Pour $P_0(X) = 1 : L(1)(X) = (1) - (1) = 0$. Donc, $L(1) = 0 \cdot 1 + 0 \cdot X + \dots + 0 \cdot X^n$. La première colonne de $M_B(L)$ est $(0, 0, \dots, 0)^T$.

2. Pour $P_k(X) = X^k$, avec $k \geq 1 : L(X^k)(X) = (X + 1)^k - X^k$. Utilisons la formule du binôme de Newton : $(X + 1)^k = \sum_{j=0}^k \binom{k}{j} X^j = \binom{k}{0} X^0 + \binom{k}{1} X^1 + \dots + \binom{k}{k-1} X^{k-1} + \binom{k}{k} X^k$. $L(X^k)(X) = \left(\sum_{j=0}^k \binom{k}{j} X^j \right) - X^k$. Comme $\binom{k}{k} = 1$, le terme X^k s'annule : $L(X^k)(X) = \sum_{j=0}^{k-1} \binom{k}{j} X^j = \binom{k}{0} + \binom{k}{1} X + \binom{k}{2} X^2 + \dots + \binom{k}{k-1} X^{k-1}$. Ce polynôme est de degré $k - 1$.

Les colonnes de la matrice $M_B(L)$ sont les vecteurs de coordonnées de $L(X^j)$ dans la base $(1, X, \dots, X^n)$. * Colonne 0 (pour $X^0 = 1$) : $(0, 0, \dots, 0)^T$. * Colonne 1 (pour $X^1 = X$) : $L(X) = \binom{1}{0} = 1$. Coordonnées : $(1, 0, \dots, 0)^T$. * Colonne 2 (pour X^2) : $L(X^2) = \binom{2}{0} + \binom{2}{1} X = 1 + 2X$. Coordonnées : $(1, 2, 0, \dots, 0)^T$. * Colonne 3 (pour X^3) : $L(X^3) = \binom{3}{0} + \binom{3}{1} X + \binom{3}{2} X^2 = 1 + 3X + 3X^2$. Coordonnées : $(1, 3, 3, 0, \dots, 0)^T$. * Généralement, pour la colonne j (correspondant à X^j) : les coefficients sont $\binom{j}{0}, \binom{j}{1}, \dots, \binom{j}{j-1}$, suivis de zéros.

La matrice $M_B(L)$ est de taille $(n + 1) \times (n + 1)$. Ses coefficients $(M_B(L))_{i,j}$ (où i est l'indice de ligne et j l'indice de colonne, tous deux allant de 0 à n) sont donnés par :

$$(M_B(L))_{i,j} = \begin{cases} \binom{j}{i} & \text{si } i < j \\ 0 & \text{si } i \geq j \end{cases}$$

(Notez que $\binom{j}{i} = 0$ si $i > j$ ou $i < 0$, donc la condition $i < j$ est suffisante).

Exemple pour $n = 3$, $E = \mathbb{R}_3[X]$, base $(1, X, X^2, X^3) : L(1) = 0 \quad L(X) = 1 \quad L(X^2) = 1 + 2X \quad L(X^3) = 1 + 3X + 3X^2$

La matrice $M_B(L)$ est :

$$M_B(L) = \begin{pmatrix} (M_B(L))_{0,0} & (M_B(L))_{0,1} & (M_B(L))_{0,2} & (M_B(L))_{0,3} \\ (M_B(L))_{1,0} & (M_B(L))_{1,1} & (M_B(L))_{1,2} & (M_B(L))_{1,3} \\ (M_B(L))_{2,0} & (M_B(L))_{2,1} & (M_B(L))_{2,2} & (M_B(L))_{2,3} \\ (M_B(L))_{3,0} & (M_B(L))_{3,1} & (M_B(L))_{3,2} & (M_B(L))_{3,3} \end{pmatrix} = \begin{pmatrix} 0 & \binom{1}{0} & \binom{2}{0} & \binom{3}{0} \\ 0 & 0 & \binom{2}{1} & \binom{3}{1} \\ 0 & 0 & 0 & \binom{3}{2} \\ 0 & 0 & 0 & 0 \end{pmatrix} = \begin{pmatrix} 0 & 1 & 1 & 1 \\ 0 & 0 & 2 & 3 \\ 0 & 0 & 0 & 3 \\ 0 & 0 & 0 & 0 \end{pmatrix}$$

Cette matrice est strictement triangulaire supérieure. Son rang est n , comme attendu.

\chapter*{Exercice 7 : Endomorphismes idempotents et stabilité (Difficulté : \textbf{)}}{}

Soit $\mathbb{K}$ un corps commutatif. Soit E un espace vectoriel de dimension finie $n \geq 1$ sur $\mathbb{K}$. Soit $u \in \mathcal{L}(E)$ un endomorphisme de E tel que $u \circ u = u$, c'est-à-dire $u^2 = u$. Un tel endomorphisme est appelé un projecteur ou un endomorphisme idempotent.

1. Démontrer que $E = \text{Ker}(u) \oplus \text{Im}(u)$. \textit{(Indication : Considérer l'expression $x = (x - u(x)) + u(x)$ pour tout $x \in E$.)}

2. Démontrer que $\text{Im}(u) = \{y \in E \mid u(y) = y\}$. \textit{(Autrement dit, $\text{Im}(u) = \text{Ker}(u - \text{Id}_E)$.)}

3. Soit $v \in \mathcal{L}(E)$ un autre endomorphisme de E . On suppose que u et v commutent, c'est-à-dire $u \circ v = v \circ u$. Démontrer que $\text{Ker}(u)$ est stable par v , et que $\text{Im}(u)$ est stable par v . \textit{(Un sous-espace F de E est stable par v si $v(F) \subset F$.)}

4. En déduire que si u et v commutent, alors la restriction de v à $\text{Im}(u)$, notée $v|_{\text{Im}(u)}$, est un endomorphisme de $\text{Im}(u)$, et la restriction de v à $\text{Ker}(u)$, notée $v|_{\text{Ker}(u)}$, est un endomorphisme de $\text{Ker}(u)$.

\section*{Correction détaillée}

1. \textbf{Démontrer que $E = \text{Ker}(u) \oplus \text{Im}(u)$.

Pour établir qu'un espace vectoriel E est la somme directe de deux sous-espaces vectoriels F_1 et F_2 , il faut montrer que $F_1 \cap F_2 = \{0_E\}$ (l'intersection est triviale) et que $E = F_1 + F_2$ (la somme est génératrice).

a) \textbf{Montrons que $\text{Ker}(u) \cap \text{Im}(u) = \{0_E\}$. Soit y un vecteur appartenant à l'intersection $\text{Ker}(u) \cap \text{Im}(u)$. Puisque $y \in \text{Ker}(u)$, la définition du noyau implique que $u(y) = 0_E$. Puisque $y \in \text{Im}(u)$, la définition de l'image implique qu'il existe un vecteur $x \in E$ tel que $y = u(x)$. Appliquons l'endomorphisme u à l'égalité $y = u(x)$: $u(y) = u(u(x)) = u^2(x)$. Par hypothèse, nous savons que $u^2 = u$. Donc $u^2(x) = u(x)$. Ainsi, nous avons $u(y) = u(x)$. En substituant $u(x)$ par y , nous obtenons $u(y) = y$. Nous avons donc deux expressions pour $u(y)$: $u(y) = 0_E$ (car $y \in \text{Ker}(u)$) et $u(y) = y$. En égalisant ces deux expressions, nous obtenons $y = 0_E$. Par conséquent, le seul vecteur commun à $\text{Ker}(u)$ et $\text{Im}(u)$ est le vecteur nul, ce qui prouve que $\text{Ker}(u) \cap \text{Im}(u) = \{0_E\}$.

b) \textbf{Montrons que $E = \text{Ker}(u) + \text{Im}(u)$. Soit x un vecteur quelconque de E . Nous voulons montrer qu'il peut s'écrire comme la somme d'un vecteur de $\text{Ker}(u)$ et d'un vecteur de $\text{Im}(u)$. Considérons la décomposition suggérée : $x = (x - u(x)) + u(x)$. Analysons le premier terme, $x - u(x)$: Appliquons u à ce terme : $u(x - u(x)) = u(x) - u(u(x))$. Ceci peut s'écrire $u(x) - u^2(x)$. Puisque $u^2 = u$ par hypothèse, nous avons $u^2(x) = u(x)$. Donc, $u(x - u(x)) = u(x) - u(x) = 0_E$. Par définition du noyau, cela signifie que le vecteur $x - u(x)$ appartient à $\text{Ker}(u)$. Analysons le second terme, $u(x)$: Par définition même de l'image d'un endomorphisme, tout vecteur de la forme $u(z)$ pour un certain $z \in E$ appartient à $\text{Im}(u)$. Ici, $u(x)$ appartient clairement à $\text{Im}(u)$. Ainsi, tout vecteur $x \in E$ peut s'écrire comme la somme d'un vecteur de $\text{Ker}(u)$ (à savoir $x - u(x)$) et d'un vecteur de $\text{Im}(u)$ (à savoir $u(x)$). Ceci prouve que $E = \text{Ker}(u) + \text{Im}(u)$.

En combinant les résultats de a) et b), nous concluons que $E = \text{Ker}(u) \oplus \text{Im}(u)$.

2. \textbf{Démontrer que $\text{Im}(u) = \{y \in E \mid u(y) = y\}$.

Notons $F = \{y \in E \mid u(y) = y\}$. Nous devons montrer l'égalité des ensembles $\text{Im}(u)$ et F . Pour cela, nous démontrons la double inclusion.

a) \textbf{Montrons que $\text{Im}(u) \subset F$. Soit $y \in \text{Im}(u)$. Par définition, il existe un vecteur $x \in E$ tel que $y = u(x)$. Appliquons u au vecteur y : $u(y) = u(u(x)) = u^2(x)$. Puisque $u^2 = u$ par hypothèse, nous avons $u^2(x) = u(x)$. En substituant, $u(y) = u(x)$. Comme $y = u(x)$, on obtient $u(y) = y$. Par conséquent, y satisfait la condition $u(y) = y$, ce qui signifie $y \in F$. Donc, tout élément de $\text{Im}(u)$ est un élément de F , d'où $\text{Im}(u) \subset F$.

b) \textbf{Montrons que $F \subset \text{Im}(u)$. Soit $y \in F$. Par définition de F , nous avons $u(y) = y$. L'expression $y = u(y)$ montre que y est l'image d'un certain vecteur par u (en l'occurrence, y est l'image de lui-même par u). Par définition de l'image, tout vecteur de la forme $u(z)$ est dans $\text{Im}(u)$. Donc, $y \in \text{Im}(u)$. Ainsi, tout élément de F est un élément de $\text{Im}(u)$, d'où $F \subset \text{Im}(u)$.

Des inclusions a) et b), nous concluons que $\text{Im}(u) = \{y \in E \mid u(y) = y\}$. Cette condition $u(y) = y$ est équivalente à $u(y) - y = 0_E$, ce qui peut s'écrire $(u - \text{Id}_E)(y) = 0_E$. C'est précisément la définition du noyau de l'endomorphisme $(u - \text{Id}_E)$, donc $\text{Im}(u) = \text{Ker}(u - \text{Id}_E)$.

3. \textbf{Soit $v \in \mathcal{L}(E)$ tel que $u \circ v = v \circ u$. Démontrer que $\text{Ker}(u)$ est stable par v , et que $\text{Im}(u)$ est stable par v .

a) \textbf{Stabilité de $\text{Ker}(u)$ par v . Un sous-espace F est stable par v si pour tout $x \in F$, $v(x) \in F$. Soit $x \in \text{Ker}(u)$. Par définition, $u(x) = 0_E$. Nous voulons montrer que $v(x) \in \text{Ker}(u)$, c'est-à-dire que $u(v(x)) = 0_E$. Calculons $u(v(x))$. Puisque u et v commutent (c'est-à-dire $u \circ v = v \circ u$), nous avons : $u(v(x)) = (u \circ v)(x) = (v \circ u)(x) = v(u(x))$. Comme $x \in \text{Ker}(u)$, nous savons que $u(x) = 0_E$. Donc, $u(v(x)) = v(0_E)$. Puisque v est un endomorphisme linéaire, il transforme le vecteur nul en le vecteur nul : $v(0_E) = 0_E$. Ainsi, $u(v(x)) = 0_E$. Cela signifie que $v(x)$ appartient à $\text{Ker}(u)$. Par conséquent, $\text{Ker}(u)$ est stable par v .

b) \textbf{Stabilité de $\text{Im}(u)$ par v . Un sous-espace F est stable par v si pour tout $y \in F$, $v(y) \in F$. Soit $y \in \text{Im}(u)$. Par définition, il existe un vecteur $x \in E$ tel que $y = u(x)$. Nous voulons montrer que $v(y) \in \text{Im}(u)$. Calculons $v(y)$: $v(y) = v(u(x))$. Puisque u et v commutent ($v \circ u = u \circ v$), nous pouvons écrire : $v(u(x)) = (v \circ u)(x) = (u \circ v)(x) = u(v(x))$. Donc, $v(y) = u(v(x))$. Par définition, tout vecteur qui est l'image d'un autre vecteur (ici $v(x)$) par u appartient à $\text{Im}(u)$. Ainsi, $v(y) \in \text{Im}(u)$. Par conséquent, $\text{Im}(u)$ est stable par v .

4. \textbf{En déduire que si u et v commutent, alors la restriction de v à $\text{Im}(u)$, notée $v|_{\text{Im}(u)}$, est un endomorphisme de $\text{Im}(u)$, et la restriction de v à $\text{Ker}(u)$, notée $v|_{\text{Ker}(u)}$, est un endomorphisme de $\text{Ker}(u)$.

Par définition, une application linéaire $w : F \rightarrow E'$ est un endomorphisme de F si $E' = F$. C'est-à-dire que w doit être une application linéaire de F dans F . Pour qu'une restriction $v|_F : F \rightarrow E$ devienne un endomorphisme de F , il est nécessaire et suffisant que F soit stable par v (car la linéarité de v implique la linéarité de sa restriction).

a) \textbf{Pour } $F = \text{Ker}(u)$: D'après la question 3a), si u et v commutent, alors $\text{Ker}(u)$ est stable par v . Cela signifie que pour tout $x \in \text{Ker}(u)$, $v(x) \in \text{Ker}(u)$. Nous pouvons donc définir une application $v|_{\text{Ker}(u)} : \text{Ker}(u) \rightarrow \text{Ker}(u)$ qui associe à $x \in \text{Ker}(u)$ le vecteur $v(x)$. Puisque v est un endomorphisme linéaire de E , sa restriction à un sous-espace, $v|_{\text{Ker}(u)}$, est également linéaire. Par conséquent, $v|_{\text{Ker}(u)}$ est un endomorphisme de $\text{Ker}(u)$.

b) \textbf{Pour } $F = \text{Im}(u)$: D'après la question 3b), si u et v commutent, alors $\text{Im}(u)$ est stable par v . Cela signifie que pour tout $y \in \text{Im}(u)$, $v(y) \in \text{Im}(u)$. Nous pouvons donc définir une application $v|_{\text{Im}(u)} : \text{Im}(u) \rightarrow \text{Im}(u)$ qui associe à $y \in \text{Im}(u)$ le vecteur $v(y)$. Puisque v est un endomorphisme linéaire de E , sa restriction à un sous-espace, $v|_{\text{Im}(u)}$, est également linéaire. Par conséquent, $v|_{\text{Im}(u)}$ est un endomorphisme de $\text{Im}(u)$.

\chapter*{Exercice 8 : Indice de Fitting et Décomposition de Fitting (Difficulté : \textbf{})}

Soit $\mathbb{K}$ un corps commutatif. Soit E un espace vectoriel sur $\mathbb{K}$ de dimension finie $n \geq 1$. Soit $f \in \mathcal{L}(E)$ un endomorphisme de E . On définit l'application $f^0 = \text{Id}_E$ (l'application identité sur E), et pour tout entier $k \in \mathbb{N}$, $f^{k+1} = f \circ f^k$.

\section*{Correction détaillée}

1. \textbf{Analyse des Noyaux}

a. Pour montrer que $\ker(f^k) \subseteq \ker(f^{k+1})$ pour tout $k \in \mathbb{N}$: Soit $x \in \ker(f^k)$. Par définition du noyau d'une application linéaire, cela signifie que $f^k(x) = 0_E$. Appliquons l'endomorphisme f à cette égalité. Puisque f est linéaire, $f(0_E) = 0_E$. Nous obtenons $f(f^k(x)) = f(0_E)$, ce qui se simplifie en $f^{k+1}(x) = 0_E$ par définition de la composition des applications. L'égalité $f^{k+1}(x) = 0_E$ signifie que x appartient au noyau de f^{k+1} , c'est-à-dire $x \in \ker(f^{k+1})$. Par conséquent, tout élément de $\ker(f^k)$ est aussi un élément de $\ker(f^{k+1})$, d'où l'inclusion $\ker(f^k) \subseteq \ker(f^{k+1})$ pour tout $k \in \mathbb{N}$.

b. La suite $(\ker(f^k))_{k \in \mathbb{N}}$ est une suite croissante de sous-espaces vectoriels de E . Puisque E est un espace vectoriel de dimension finie n , la dimension de chaque sous-espace vectoriel $\ker(f^k)$ est un entier naturel. Cette dimension est bornée supérieurement par la dimension de E , c'est-à-dire n . Nous avons la suite d'inégalités de dimensions : $0 \leq \dim(\ker(f^0)) \leq \dim(\ker(f^1)) \leq \dim(\ker(f^2)) \leq \dots \leq \dim(E) = n$. Une suite croissante d'entiers naturels qui est bornée supérieurement converge et donc doit nécessairement stabiliser à partir d'un certain rang. Si la dimension $\dim(\ker(f^k))$ est égale à $\dim(\ker(f^{k+1}))$, et comme nous avons déjà établi l'inclusion $\ker(f^k) \subseteq \ker(f^{k+1})$, l'égalité des dimensions implique que les deux sous-espaces vectoriels sont identiques : $\ker(f^k) = \ker(f^{k+1})$. Ainsi, la suite des noyaux doit stabiliser. Par le principe du bon ordre des entiers naturels, il existe un plus petit entier naturel d_0 tel que $\ker(f^{d_0}) = \ker(f^{d_0+1})$.

c. Nous montrons par récurrence sur k que pour tout $k \geq d_0$, on a $\ker(f^k) = \ker(f^{d_0})$. * \textbf{Initialisation} ($k = d_0$) : La propriété est vraie par définition de d_0 , puisque $\ker(f^{d_0}) = \ker(f^{d_0+1})$. * \textbf{Hypothèse de récurrence} : Supposons que pour un certain entier $k \geq d_0$, on ait $\ker(f^k) = \ker(f^{d_0})$. * \textbf{Hérédité} : Démontrons que $\ker(f^{k+1}) = \ker(f^{d_0})$. Nous savons d'après 1.a que $\ker(f^k) \subseteq \ker(f^{k+1})$. Par transitivité, $\ker(f^{d_0}) = \ker(f^k) \subseteq \ker(f^{k+1})$. Il reste à prouver l'inclusion inverse : $\ker(f^{k+1}) \subseteq \ker(f^{d_0})$. Soit $x \in \ker(f^{k+1})$. Par définition, $f^{k+1}(x) = 0_E$. On peut écrire $f^{k+1}(x) = f^k(f(x))$. Donc $f^k(f(x)) = 0_E$. Ceci signifie que $f(x) \in \ker(f^k)$. D'après l'hypothèse de récurrence, $\ker(f^k) = \ker(f^{d_0})$. Par conséquent, $f(x) \in \ker(f^{d_0})$. Cela implique que $f^{d_0}(f(x)) = 0_E$, ce qui est équivalent à $f^{d_0+1}(x) = 0_E$. Puisque, par définition de d_0 , nous avons $\ker(f^{d_0+1}) = \ker(f^{d_0})$, l'égalité $f^{d_0+1}(x) = 0_E$ signifie que $x \in \ker(f^{d_0})$. Ainsi, $\ker(f^{k+1}) \subseteq \ker(f^{d_0})$. En combinant les inclusions, nous avons $\ker(f^{k+1}) = \ker(f^{d_0})$. Par le principe de récurrence, pour tout $k \geq d_0$, l'égalité $\ker(f^k) = \ker(f^{d_0})$ est établie.

2. \textbf{Analyse des Images}

a. Pour montrer que $\text{Im}(f^{k+1}) \subseteq \text{Im}(f^k)$ pour tout $k \in \mathbb{N}$: Soit $y \in \text{Im}(f^{k+1})$. Par définition de l'image, il existe un vecteur $x \in E$ tel que $y = f^{k+1}(x)$. On peut réécrire $f^{k+1}(x)$ comme $f^k(f(x))$. Posons $x' = f(x)$. Alors x' est un vecteur de E . L'égalité devient $y = f^k(x')$. Ceci signifie que y appartient à l'image de f^k , c'est-à-dire $y \in \text{Im}(f^k)$. Par conséquent, $\text{Im}(f^{k+1}) \subseteq \text{Im}(f^k)$ pour tout $k \in \mathbb{N}$.

b. La suite $(\text{Im}(f^k))_{k \in \mathbb{N}}$ est une suite décroissante de sous-espaces vectoriels de E . Puisque E est un espace vectoriel de dimension finie n , la dimension de chaque sous-espace vectoriel $\text{Im}(f^k)$ est un entier naturel, bornée inférieurement par 0. Nous avons la suite d'inégalités de dimensions : $n = \dim(E) \geq \dim(\text{Im}(f^0)) \geq \dim(\text{Im}(f^1)) \geq \dim(\text{Im}(f^2)) \geq \dots \geq 0$. Une suite décroissante d'entiers naturels qui est bornée inférieurement doit nécessairement stabiliser à partir d'un certain rang. Si la dimension $\dim(\text{Im}(f^k))$ est égale à $\dim(\text{Im}(f^{k+1}))$, et comme nous avons déjà établi l'inclusion $\text{Im}(f^{k+1}) \subseteq \text{Im}(f^k)$, l'égalité des dimensions implique que les deux sous-espaces vectoriels sont identiques : $\text{Im}(f^k) = \text{Im}(f^{k+1})$. Ainsi, la suite des images doit stabiliser. Par le principe du bon ordre des entiers naturels, il existe un plus petit entier naturel p_0 tel que $\text{Im}(f^{p_0}) = \text{Im}(f^{p_0+1})$.

c. Nous devons montrer que pour tout $k \geq p_0$, on a $\text{Im}(f^k) = \text{Im}(f^{p_0})$. Par définition de p_0 , nous avons $\text{Im}(f^{p_0}) = \text{Im}(f^{p_0+1})$. Considérons la restriction de f à $\text{Im}(f^{p_0})$. Notons cette application $g = f|_{\text{Im}(f^{p_0})} : \text{Im}(f^{p_0}) \rightarrow E$. L'image de cette application est $g(\text{Im}(f^{p_0})) = f(\text{Im}(f^{p_0})) = f(f^{p_0}(E)) = f^{p_0+1}(E) = \text{Im}(f^{p_0+1})$. Puisque $\text{Im}(f^{p_0}) = \text{Im}(f^{p_0+1})$, cela signifie que l'image de g est $\text{Im}(f^{p_0})$. Ainsi, $g : \text{Im}(f^{p_0}) \rightarrow \text{Im}(f^{p_0})$ est un endomorphisme de l'espace $\text{Im}(f^{p_0})$. De plus, puisque l'image de g est $\text{Im}(f^{p_0})$ lui-même, cet endomorphisme g est surjectif. Comme $\text{Im}(f^{p_0})$ est un sous-espace vectoriel de E , il est de dimension finie. Pour un espace vectoriel de dimension finie, un endomorphisme surjectif est aussi injectif, et donc un automorphisme. Par conséquent, $f|_{\text{Im}(f^{p_0})} : \text{Im}(f^{p_0}) \rightarrow \text{Im}(f^{p_0})$ est un automorphisme. Cela signifie que pour tout entier $k \geq 1$, la k -ième composition de $f|_{\text{Im}(f^{p_0})}$ sur lui-même, notée $(f|_{\text{Im}(f^{p_0})})^k$, est également un automorphisme de $\text{Im}(f^{p_0})$. En particulier, $(f|_{\text{Im}(f^{p_0})})^k$ est surjective, ce qui implique que $f^k(\text{Im}(f^{p_0})) = \text{Im}(f^{p_0})$ pour tout $k \geq 1$. Par définition, $f^k(\text{Im}(f^{p_0})) = f^k(f^{p_0}(E)) = f^{k+p_0}(E) = \text{Im}(f^{k+p_0})$. En combinant ces résultats, nous obtenons $\text{Im}(f^{p_0+k}) = \text{Im}(f^{p_0})$ pour tout $k \geq 1$. Ceci démontre que pour tout $k \geq p_0$, on a $\text{Im}(f^k) = \text{Im}(f^{p_0})$.

3. \textbf{Lien entre Noyau et Image (Théorème du Rang)}

a. Le théorème du rang affirme que pour toute application linéaire $L : V \rightarrow W$ où V est de dimension finie, on a $\dim(V) = \dim(\ker(L)) + \dim(\text{Im}(L))$. Appliquons ce théorème à l'endomorphisme $f^k : E \rightarrow E$ pour tout

$k \in \mathbb{N}$. L'espace E est de dimension finie n . Donc, $n = \dim(\ker(f^k)) + \dim(\operatorname{Im}(f^k))$.

* Montrons que $p_0 \leq d_0$: Par définition de d_0 , nous avons $\ker(f^{d_0}) = \ker(f^{d_0+1})$. Ceci implique que $\dim(\ker(f^{d_0})) = \dim(\ker(f^{d_0+1}))$. En utilisant le théorème du rang pour f^{d_0} et f^{d_0+1} : $\dim(\operatorname{Im}(f^{d_0})) = n - \dim(\ker(f^{d_0}))$ et $\dim(\operatorname{Im}(f^{d_0+1})) = n - \dim(\ker(f^{d_0+1}))$. Puisque $\dim(\ker(f^{d_0})) = \dim(\ker(f^{d_0+1}))$, il s'ensuit que $\dim(\operatorname{Im}(f^{d_0})) = \dim(\operatorname{Im}(f^{d_0+1}))$. Comme nous savons que $\operatorname{Im}(f^{d_0+1}) \subseteq \operatorname{Im}(f^{d_0})$ (d'après 2.a), l'égalité des dimensions implique que $\operatorname{Im}(f^{d_0}) = \operatorname{Im}(f^{d_0+1})$. Par définition, p_0 est le plus petit entier tel que $\operatorname{Im}(f^{p_0}) = \operatorname{Im}(f^{p_0+1})$. Puisque cette égalité est vérifiée pour d_0 , nous devons avoir $p_0 \leq d_0$.

* Montrons que $d_0 \leq p_0$: Par définition de p_0 , nous avons $\operatorname{Im}(f^{p_0}) = \operatorname{Im}(f^{p_0+1})$. Ceci implique que $\dim(\operatorname{Im}(f^{p_0})) = \dim(\operatorname{Im}(f^{p_0+1}))$. En utilisant le théorème du rang pour f^{p_0} et f^{p_0+1} : $\dim(\ker(f^{p_0})) = n - \dim(\operatorname{Im}(f^{p_0}))$ et $\dim(\ker(f^{p_0+1})) = n - \dim(\operatorname{Im}(f^{p_0+1}))$. Puisque $\dim(\operatorname{Im}(f^{p_0})) = \dim(\operatorname{Im}(f^{p_0+1}))$, il s'ensuit que $\dim(\ker(f^{p_0})) = \dim(\ker(f^{p_0+1}))$. Comme nous savons que $\ker(f^{p_0}) \subseteq \ker(f^{p_0+1})$ (d'après 1.a), l'égalité des dimensions implique que $\ker(f^{p_0}) = \ker(f^{p_0+1})$. Par définition, d_0 est le plus petit entier tel que $\ker(f^{d_0}) = \ker(f^{d_0+1})$. Puisque cette égalité est vérifiée pour p_0 , nous devons avoir $d_0 \leq p_0$.

Des deux inégalités $p_0 \leq d_0$ et $d_0 \leq p_0$, nous concluons que $d_0 = p_0$. Nous noterons cette valeur commune k_0 .

b. D'après la question 2.c, la condition $\operatorname{Im}(f^{p_0}) = \operatorname{Im}(f^{p_0+1})$ implique que la restriction de f à $\operatorname{Im}(f^{p_0})$, soit $f|_{\operatorname{Im}(f^{p_0})}$, est un automorphisme de $\operatorname{Im}(f^{p_0})$. En utilisant $k_0 = p_0$, cela signifie que $f|_{\operatorname{Im}(f^{k_0})} : \operatorname{Im}(f^{k_0}) \rightarrow \operatorname{Im}(f^{k_0})$ est un endomorphisme dont l'image est $\operatorname{Im}(f^{k_0+1})$, qui est égale à $\operatorname{Im}(f^{k_0})$. Par conséquent, $f|_{\operatorname{Im}(f^{k_0})}$ est un endomorphisme surjectif de $\operatorname{Im}(f^{k_0})$. Puisque $\operatorname{Im}(f^{k_0})$ est un sous-espace vectoriel de E , il est de dimension finie. Dans un espace vectoriel de dimension finie, un endomorphisme surjectif est nécessairement bijectif (et donc injectif). Ainsi, $f|_{\operatorname{Im}(f^{k_0})}$ est un isomorphisme (un automorphisme) de $\operatorname{Im}(f^{k_0})$ sur lui-même.

4. \textbf{Décomposition de Fitting}

a. Nous devons montrer que $\ker(f^{k_0}) \cap \operatorname{Im}(f^{k_0}) = \{0_E\}$. Soit $x \in \ker(f^{k_0}) \cap \operatorname{Im}(f^{k_0})$. Puisque $x \in \ker(f^{k_0})$, par définition, $f^{k_0}(x) = 0_E$. Puisque $x \in \operatorname{Im}(f^{k_0})$, par définition, il existe un vecteur $y \in E$ tel que $x = f^{k_0}(y)$. Substituons cette expression de x dans la première égalité : $f^{k_0}(f^{k_0}(y)) = 0_E$. Ceci s'écrit $f^{2k_0}(y) = 0_E$. Cela implique que $y \in \ker(f^{2k_0})$. D'après la question 1.c, nous savons que pour tout $k \geq d_0$, $\ker(f^k) = \ker(f^{d_0})$. Puisque $k_0 = d_0$, cette propriété s'applique à k_0 . En particulier, puisque $2k_0 \geq k_0$ (tout en supposant $k_0 \geq 0$; si $k_0 = 0$, $\ker(f^0) = \{0_E\}$, alors $\ker(f^{2 \cdot 0}) = \ker(f^0)$), nous avons $\ker(f^{2k_0}) = \ker(f^{k_0})$. Par conséquent, $y \in \ker(f^{k_0})$. Cela signifie que $f^{k_0}(y) = 0_E$. Puisque nous avons posé $x = f^{k_0}(y)$, nous en déduisons que $x = 0_E$. Ainsi, le seul élément de l'intersection de $\ker(f^{k_0})$ et $\operatorname{Im}(f^{k_0})$ est le vecteur nul, donc $\ker(f^{k_0}) \cap \operatorname{Im}(f^{k_0}) = \{0_E\}$.

b. Pour prouver que $E = \ker(f^{k_0}) \oplus \operatorname{Im}(f^{k_0})$, il suffit de montrer deux conditions pour les sous-espaces vectoriels $U = \ker(f^{k_0})$ et $V = \operatorname{Im}(f^{k_0})$: i. Leur intersection est triviale : $U \cap V = \{0_E\}$. Cette condition a été démontrée à la question 4.a. ii. La somme de leurs dimensions est égale à la dimension de l'espace ambiant : $\dim(U) + \dim(V) = \dim(E)$. Appliquons le théorème du rang à l'endomorphisme $f^{k_0} : E \rightarrow E$: $\dim(E) = \dim(\ker(f^{k_0})) + \dim(\operatorname{Im}(f^{k_0}))$. Cette condition est également vérifiée.

Étant donné que ces deux conditions sont satisfaites, nous pouvons conclure que l'espace vectoriel E est la somme directe de $\ker(f^{k_0})$ et $\operatorname{Im}(f^{k_0})$. Ainsi, $E = \ker(f^{k_0}) \oplus \operatorname{Im}(f^{k_0})$.

\chapter{\textit{Exercice 9 : Décomposition de Fitting et endomorphismes nilpotents/inversibles (Difficulté : \textbf{)}}{}

Soit K un corps commutatif quelconque. Soit E un K -espace vectoriel de dimension finie $n \geq 1$. Soit $u \in L(E)$ un endomorphisme de E .

Pour tout entier naturel $k \geq 0$, on définit les sous-espaces vectoriels : - $K_k = \ker(u^k)$, le noyau de u^k . - $I_k = \text{Im}(u^k)$, l'image de u^k . Par convention, $u^0 = \text{id}_E$, donc $K_0 = \{0\}$ et $I_0 = E$.

1. Démontrer que la suite $(K_k)_{k \geq 0}$ est une suite croissante de sous-espaces vectoriels de E (pour l'inclusion), et que la suite $(I_k)_{k \geq 0}$ est une suite décroissante de sous-espaces vectoriels de E . 2. Démontrer qu'il existe un plus petit entier $p \in \{0, \dots, n\}$ tel que $K_p = K_{p+1}$. Démontrer alors que pour tout $k \geq p$, on a $K_k = K_p$. Démontrer un résultat analogue pour la suite $(I_k)_{k \geq 0}$, c'est-à-dire $I_p = I_{p+1}$ et $I_k = I_p$ pour tout $k \geq p$. Cet entier p est appelé l'indice de Fitting de l'endomorphisme u . 3. On se place avec l'entier p défini à la question précédente. Démontrer que $E = K_p \oplus I_p$. 4. Démontrer que les sous-espaces K_p et I_p sont stables par u . On note u_K la restriction de u à K_p (vue comme endomorphisme de K_p) et u_I la restriction de u à I_p (vue comme endomorphisme de I_p). Démontrer que u_K est un endomorphisme nilpotent et que u_I est un automorphisme.

\section*{Correction détaillée}

1. \textbf{Monotonie des suites $(K_k)_{k \geq 0}$ et $(I_k)_{k \geq 0}$ }

* \textbf{Pour $(K_k)_{k \geq 0}$:} Soit $k \geq 0$. Nous voulons montrer que $K_k \subseteq K_{k+1}$. Soit $x \in K_k$. Par définition, $u^k(x) = 0$. Alors $u^{k+1}(x) = u(u^k(x)) = u(0) = 0$. Donc $x \in K_{k+1}$. Ceci prouve que $K_k \subseteq K_{k+1}$ pour tout $k \geq 0$. La suite $(K_k)_{k \geq 0}$ est donc croissante pour l'inclusion.

* \textbf{Pour $(I_k)_{k \geq 0}$:} Soit $k \geq 0$. Nous voulons montrer que $I_{k+1} \subseteq I_k$. Soit $y \in I_{k+1}$. Par définition, il existe $x \in E$ tel que $y = u^{k+1}(x)$. On peut écrire $y = u^k(u(x))$. Puisque $u(x) \in E$, y est l'image d'un élément de E par u^k . Donc $y \in I_k$. Ceci prouve que $I_{k+1} \subseteq I_k$ pour tout $k \geq 0$. La suite $(I_k)_{k \geq 0}$ est donc décroissante pour l'inclusion.

2. \textbf{Stabilisation des suites $(K_k)_{k \geq 0}$ et $(I_k)_{k \geq 0}$ }

* \textbf{Pour $(K_k)_{k \geq 0}$:} La suite $(K_k)_{k \geq 0}$ est une suite croissante de sous-espaces vectoriels de E . De plus, E est de dimension finie n . Cela implique que la suite des dimensions $\dim(K_k)$ est une suite croissante d'entiers naturels, majorée par n . Une telle suite doit nécessairement stationner. C'est-à-dire qu'il existe un entier p_0 tel que $\dim(K_{p_0}) = \dim(K_{p_0+1})$. Puisque $K_{p_0} \subseteq K_{p_0+1}$ et qu'ils ont la même dimension, on en déduit que $K_{p_0} = K_{p_0+1}$. Soit p le plus petit entier tel que $K_p = K_{p+1}$. Un tel p existe car la suite stationne et $K_0 \subseteq K_1 \subseteq \dots \subseteq K_n = E$ est la chaîne maximale de sous-espaces strictement croissants. Si $K_i \subsetneq K_{i+1}$ pour tout i , alors on aurait $\dim(K_{i+1}) \geq \dim(K_i) + 1$, ce qui impliquerait $\dim(K_n) \geq \dim(K_0) + n = n$, atteignant la dimension de E au plus en n étapes. Nous allons montrer par récurrence que pour tout $k \geq p$, $K_k = K_p$. L'initialisation ($k = p$) est triviale : $K_p = K_p$. L'étape de récurrence : Supposons que $K_k = K_p$ pour un $k \geq p$. Nous voulons montrer que $K_{k+1} = K_p$. Nous savons déjà que $K_p \subseteq K_{k+1}$ (car la suite est croissante). Il reste à montrer que $K_{k+1} \subseteq K_p$. Soit $x \in K_{k+1}$. Alors $u^{k+1}(x) = 0$. Nous avons $u^p(x) \in E$. Alors $u^{k+1-p}(u^p(x)) = u^{k+1}(x) = 0$. Puisque $k \geq p$, $k+1-p \geq 1$. Considérons l'application u restreinte à K_{j+1} pour un certain j . Si $y \in K_{k+1}$, alors $u^{k+1}(y) = 0$. Si $K_p = K_{p+1}$, montrons $K_{p+1} = K_{p+2}$. Soit $x \in K_{p+2}$. Alors $u^{p+2}(x) = 0$. On peut écrire $u^{p+1}(u(x)) = 0$. Donc $u(x) \in K_{p+1}$. Puisque $K_{p+1} = K_p$, on a $u(x) \in K_p$, ce qui signifie $u^p(u(x)) = 0$, donc $u^{p+1}(x) = 0$. Par conséquent $x \in K_{p+1}$. Comme $K_{p+1} = K_p$, on a $x \in K_p$. Donc $K_{p+2} \subseteq K_p$. Puisque $K_p \subseteq K_{p+2}$ (par croissance), on a $K_{p+2} = K_p$. En répétant ce raisonnement, on montre par récurrence que $K_k = K_p$ pour tout $k \geq p$.

* \textbf{Pour $(I_k)_{k \geq 0}$:} La suite $(I_k)_{k \geq 0}$ est une suite décroissante de sous-espaces vectoriels de E . Comme précédemment, la suite des dimensions $\dim(I_k)$ est une suite décroissante d'entiers naturels, minorée par 0. Elle doit donc stationner. Il existe un entier p' tel que $\dim(I_{p'}) = \dim(I_{p'+1})$. Puisque $I_{p'+1} \subseteq I_{p'}$ et qu'ils ont la même dimension, on en déduit que $I_{p'} = I_{p'+1}$. Le théorème du rang stipule que $\dim(E) = \dim(\ker(u^k)) + \dim(\text{Im}(u^k))$, soit $n = \dim(K_k) + \dim(I_k)$. Si $K_p = K_{p+1}$, alors $\dim(K_p) = \dim(K_{p+1})$. Donc $n - \dim(I_p) = n - \dim(I_{p+1})$, ce qui implique $\dim(I_p) = \dim(I_{p+1})$. Puisque $I_{p+1} \subseteq I_p$ et qu'ils ont la même dimension, on a $I_p = I_{p+1}$. Ainsi, l'entier p est le même pour la stabilisation de (K_k) et (I_k) . Pour montrer $I_k = I_p$ pour tout $k \geq p$: Nous savons déjà que $I_k \subseteq I_p$ (car la suite est décroissante). Il reste à montrer $I_p \subseteq I_k$. Puisque $I_p = I_{p+1}$, cela signifie que pour tout $y \in I_p$, $y \in I_{p+1}$, donc $y = u^{p+1}(x')$ pour un certain $x' \in E$. Plus généralement, si $I_j = I_{j+1}$, alors l'application u restreinte à I_j est surjective de I_j dans I_j . En effet, $u(I_j) = u(\text{Im}(u^j)) = \text{Im}(u^{j+1}) = I_{j+1} = I_j$. Étant surjective sur un espace de dimension finie, elle est un automorphisme. Donc $u|_{I_p} : I_p \rightarrow I_p$ est un automorphisme. Ceci signifie que $u(I_p) = I_p$. Par conséquent $u^2(I_p) = u(I_p) = I_p$, et par récurrence $u^k(I_p) = I_p$ pour tout $k \geq 0$. En particulier, $u^k(E) = u^k(I_0) \supseteq u^k(I_p)$. Soit $y \in I_p$. Alors $y = u^p(x)$ pour un certain $x \in E$. Puisque $I_p = I_k$ pour $k \geq p$, et on sait que $I_k \subseteq I_p$. Si $y \in I_p$, alors $y = u^p(x)$. Mais on a montré que $u|_{I_p}$ est un automorphisme. Donc $u^k|_{I_p}$ est aussi un automorphisme pour tout $k \geq 0$. Conséquence : $\text{Im}(u^k|_{I_p}) = I_p$. Mais $\text{Im}(u^k|_{I_p})$ est l'ensemble des éléments de la forme $u^k(z)$ où $z \in I_p$. C'est donc un sous-ensemble de I_k . Non, c'est $u^k(I_p)$. Et nous avons montré que $u^k(I_p) = I_p$. Donc $I_p \subseteq I_k$ pour $k \geq p$. Puisque nous avons aussi $I_k \subseteq I_p$, il s'ensuit que $I_k = I_p$ pour tout $k \geq p$.

3. \textbf{Décomposition $E = K_p \oplus I_p$ }

Pour montrer que $E = K_p \oplus I_p$, nous devons montrer deux choses : a. $K_p \cap I_p = \{0\}$ b. $K_p + I_p = E$ (ou de manière équivalente $\dim(K_p) + \dim(I_p) = \dim(E)$ si $K_p \cap I_p = \{0\}$)

* \textbf{a. $K_p \cap I_p = \{0\}$:} Soit $y \in K_p \cap I_p$. Puisque $y \in K_p$, on a $u^p(y) = 0$. Puisque $y \in I_p$, il existe un $x \in E$ tel que $y = u^p(x)$. En substituant, nous obtenons $u^p(u^p(x)) = 0$, c'est-à-dire $u^{2p}(x) = 0$. Cela signifie que $x \in K_{2p}$. Or, d'après la question 2, puisque p est l'indice de stabilisation, nous avons $K_{2p} = K_p$ (car $2p \geq p$). Donc $x \in K_p$. Par conséquent, $u^p(x) = 0$. Puisque $y = u^p(x)$, on en déduit $y = 0$. Donc $K_p \cap I_p = \{0\}$.

* \textbf{b. $K_p + I_p = E$:} D'après le théorème du rang appliqué à u^p : $\dim(E) = \dim(\ker(u^p)) + \dim(\text{Im}(u^p))$ $\dim(E) = \dim(K_p) + \dim(I_p)$. Puisque $K_p \cap I_p = \{0\}$, la somme $K_p + I_p$ est directe, et sa dimension est $\dim(K_p \oplus I_p) = \dim(K_p) + \dim(I_p)$. Donc $\dim(K_p \oplus I_p) = \dim(E)$. Puisque $K_p \oplus I_p$ est un sous-espace vectoriel de E et qu'il a la même dimension que E , il est égal à E . Ainsi, $E = K_p \oplus I_p$.

4. \textbf{Stabilité par u , nilpotence et automorphisme}

* \textbf{Stabilité des sous-espaces K_p et I_p par u :} * \textbf{Pour K_p :} Soit $x \in K_p$. Alors $u^p(x) = 0$. Nous devons montrer que $u(x) \in K_p$. Calculons $u^p(u(x))$. C'est $u^{p+1}(x)$. D'après la question 2, $K_p = K_{p+1}$ (par définition de p). Puisque $x \in K_p$, $u^p(x) = 0$. On peut dire que $x \in K_{p+1}$ également. Or $K_p = K_{p+1}$ signifie que si $u^{p+1}(x) = 0$, alors $u^p(x) = 0$. Ah, wait. $u(x) \in K_p$ requires $u^p(u(x)) = 0$. We know $u^p(x) = 0$. So $x \in K_p$. We want to check if $u(x) \in K_p$. This means $u^p(u(x)) = 0$, i.e. $u^{p+1}(x) = 0$. Since $x \in K_p$ and $K_p = K_{p+1}$, it implies that $x \in K_{p+1}$, so $u^{p+1}(x) = 0$ holds. Therefore, $u(x) \in K_p$. Ceci prouve que K_p est stable par u .

* \textbf{Pour I_p :} Soit $y \in I_p$. Alors il existe $x \in E$ tel que $y = u^p(x)$. Nous devons montrer que $u(y) \in I_p$. Calculons $u(y) = u(u^p(x)) = u^{p+1}(x)$. Par définition, $u^{p+1}(x) \in \text{Im}(u^{p+1}) = I_{p+1}$. D'après la question 2, $I_p = I_{p+1}$ (car p est l'indice de stabilisation). Donc $u(y) \in I_p$. Ceci prouve que I_p est stable par u .

* \textbf{Nature des endomorphismes u_K et u_I :} * \textbf{Pour $u_K : K_p \rightarrow K_p$:} Par définition, $u_K(x) = u(x)$ pour $x \in K_p$. Pour tout $x \in K_p$, nous avons $u^p(x) = 0$. Donc $u_K^p(x) = u^p(x) = 0$. Ceci signifie que u_K^p est l'endomorphisme nul sur K_p . Par conséquent, u_K est un endomorphisme nilpotent.

* \textbf{Pour $u_I : I_p \rightarrow I_p$:} Par définition, $u_I(y) = u(y)$ pour $y \in I_p$. Nous savons que u_I est un endomorphisme de I_p . Pour montrer que u_I est un automorphisme, il suffit de montrer qu'il est injectif ou surjectif, car I_p est de dimension finie. Montrons que u_I est surjectif. L'image de u_I est $\text{Im}(u_I) = \{u(y) \mid y \in I_p\}$. Puisque $y \in I_p = \text{Im}(u^p)$, il existe $x \in E$ tel que $y = u^p(x)$. Alors $u(y) = u(u^p(x)) = u^{p+1}(x)$. Ainsi, $\text{Im}(u_I) = \{u^{p+1}(x) \mid x \in E\} = \text{Im}(u^{p+1}) = I_{p+1}$. D'après la question 2, nous savons que $I_{p+1} = I_p$. Donc $\text{Im}(u_I) = I_p$. Ceci signifie que u_I est surjectif. Puisque u_I est un endomorphisme d'un espace de dimension finie I_p et qu'il est surjectif, il est également injectif. Par conséquent, u_I est un automorphisme de I_p .

Cette décomposition $E = K_p \oplus I_p$ est connue sous le nom de \textbf{Décomposition de Fitting}, et elle est fondamentale dans l'étude des endomorphismes en dimension finie, notamment pour la forme de Jordan ou la décomposition de Dunford.

\chapter{\textit{Exercice 10 : Noyau et Image d'une Composée d'Applications Linéaires (Difficulté : \textbf{}}\textbf{Énoncé}}

Soient $\mathbb{K}$ un corps commutatif et E, F, G des $\mathbb{K}$ -espaces vectoriels. Soient $f \in \mathcal{L}(E, F)$ et $g \in \mathcal{L}(F, G)$ deux applications linéaires. On note $\ker(\cdot)$ le noyau et $\text{Im}(\cdot)$ l'image d'une application linéaire.

1. Démontrer que $\ker(g \circ f) = f^{-1}(\ker g \cap \text{Im } f)$. (Rappel : $f^{-1}(A) = \{x \in E \mid f(x) \in A\}$ pour un sous-ensemble $A \subseteq F$).

2. Prouver que l'application $\hat{f} : \ker(g \circ f) \rightarrow \ker g \cap \text{Im } f$ définie par $\hat{f}(x) = f(x)$ est une application linéaire surjective dont le noyau est $\ker f$. En déduire l'isomorphisme de $\mathbb{K}$ -espaces vectoriels : $\ker(g \circ f)/\ker f \cong \ker g \cap \text{Im } f$.

3. Prouver que l'application $\bar{g} : \text{Im } f/(\ker g \cap \text{Im } f) \rightarrow \text{Im}(g \circ f)$ définie par $\bar{g}(y + (\ker g \cap \text{Im } f)) = g(y)$ pour tout $y \in \text{Im } f$ est un isomorphisme de $\mathbb{K}$ -espaces vectoriels. (On vérifiera d'abord que le sous-espace $\ker g \cap \text{Im } f$ est bien un sous-espace de $\text{Im } f$ et que $\bar{g}$ est bien définie, linéaire et injective/surjective).

4. On suppose désormais que E, F, G sont des $\mathbb{K}$ -espaces vectoriels de dimensions finies. On note $\dim(V)$ la dimension de V et $\text{rang}(h) = \dim(\text{Im } h)$ le rang d'une application linéaire h . À partir des résultats des questions 2 et 3, établir les égalités suivantes : a) $\dim(\ker(g \circ f)) = \dim(\ker f) + \dim(\ker g \cap \text{Im } f)$. b) $\text{rang}(g \circ f) = \text{rang}(f) - \dim(\ker g \cap \text{Im } f)$.

5. Utiliser les résultats précédents et le théorème du rang pour f et g pour démontrer l'inégalité de Sylvester : $\text{rang}(f) + \text{rang}(g) - \dim(F) \leq \text{rang}(g \circ f) \leq \min(\text{rang } f, \text{rang } g)$.

\section*{Correction détaillée}

1. \textbf{Démonstration de } $\ker(g \circ f) = f^{-1}(\ker g \cap \text{Im } f)$

Pour prouver l'égalité de deux ensembles, nous allons démontrer l'inclusion dans les deux sens.

* \textbf{Inclusion } $\ker(g \circ f) \subseteq f^{-1}(\ker g \cap \text{Im } f)$: Soit $x \in \ker(g \circ f)$. Par définition du noyau, cela signifie que $(g \circ f)(x) = 0_G$. Par définition de la composition, $g(f(x)) = 0_G$. Cela implique que $f(x) \in \ker g$. Par ailleurs, par définition de l'image, $f(x)$ est un élément de $\text{Im } f$. Donc, $f(x)$ appartient à l'intersection $\ker g \cap \text{Im } f$. Par définition de l'image réciproque, $x \in f^{-1}(\ker g \cap \text{Im } f)$. L'inclusion est démontrée.

* \textbf{Inclusion } $f^{-1}(\ker g \cap \text{Im } f) \subseteq \ker(g \circ f)$: Soit $x \in f^{-1}(\ker g \cap \text{Im } f)$. Par définition de l'image réciproque, cela signifie que $f(x) \in \ker g \cap \text{Im } f$. Puisque $f(x) \in \ker g$, par définition du noyau, nous avons $g(f(x)) = 0_G$. Par définition de la composition, $(g \circ f)(x) = 0_G$. Cela signifie que $x \in \ker(g \circ f)$. L'inclusion est démontrée.

Ayant démontré les deux inclusions, nous concluons que $\ker(g \circ f) = f^{-1}(\ker g \cap \text{Im } f)$.

2. \textbf{Analyse de l'application } $\hat{f}$ \textbf{et isomorphisme de quotient}

Considérons l'application $\hat{f} : \ker(g \circ f) \rightarrow \ker g \cap \text{Im } f$ définie par $\hat{f}(x) = f(x)$.

* \textbf{ } $\hat{f}$ \textbf{est bien définie et linéaire} : Le domaine de $\hat{f}$ est $\ker(g \circ f)$, qui est un sous-espace vectoriel de E . Pour tout $x \in \ker(g \circ f)$, nous avons montré à la question 1 que $f(x) \in \ker g \cap \text{Im } f$. Donc le codomaine est correct. L'application f est linéaire, et $\hat{f}$ est simplement la restriction de f à un sous-espace vectoriel, avec un codomaine restreint. Par conséquent, $\hat{f}$ est linéaire.

* \textbf{ } $\hat{f}$ \textbf{est surjective} : Soit $y \in \ker g \cap \text{Im } f$. Puisque $y \in \text{Im } f$, il existe un $x \in E$ tel que $f(x) = y$. Puisque $y \in \ker g$, nous avons $g(y) = 0_G$. Donc, $g(f(x)) = 0_G$, ce qui signifie $(g \circ f)(x) = 0_G$. Par conséquent, $x \in \ker(g \circ f)$. Nous avons trouvé un élément x dans le domaine de $\hat{f}$ tel que $\hat{f}(x) = f(x) = y$. Donc, $\hat{f}$ est surjective sur son codomaine $\ker g \cap \text{Im } f$.

* \textbf{Détermination du noyau de } $\hat{f}$: Par définition, $\ker \hat{f} = \{x \in \ker(g \circ f) \mid \hat{f}(x) = 0_F\}$. Cela signifie $\ker \hat{f} = \{x \in \ker(g \circ f) \mid f(x) = 0_F\}$. L'ensemble $\{x \in E \mid f(x) = 0_F\}$ est $\ker f$. Donc, $\ker \hat{f} = \ker(g \circ f) \cap \ker f$. Nous savons que si $x \in \ker f$, alors $f(x) = 0_F$. Par suite, $g(f(x)) = g(0_F) = 0_G$, ce qui implique $x \in \ker(g \circ f)$. Ainsi, $\ker f \subseteq \ker(g \circ f)$. Par conséquent, l'intersection $\ker(g \circ f) \cap \ker f$ se réduit à $\ker f$. Donc, $\ker \hat{f} = \ker f$.

* \textbf{Application du premier théorème d'isomorphisme} : Puisque $\hat{f} : \ker(g \circ f) \rightarrow \ker g \cap \text{Im } f$ est une application linéaire surjective de noyau $\ker f$, le premier théorème d'isomorphisme établit que $\ker(g \circ f)/\ker f \cong \text{Im}(\hat{f})$. Comme $\hat{f}$ est surjective sur son codomaine, $\text{Im}(\hat{f}) = \ker g \cap \text{Im } f$. Nous en déduisons l'isomorphisme : $\ker(g \circ f)/\ker f \cong \ker g \cap \text{Im } f$.

3. \textbf{Analyse de l'application } $\bar{g}$ \textbf{et isomorphisme de quotient}

Considérons l'application $\bar{g} : \text{Im } f/(\ker g \cap \text{Im } f) \rightarrow \text{Im}(g \circ f)$ définie par $\bar{g}(y + (\ker g \cap \text{Im } f)) = g(y)$.

* \textbf{ } $\ker g \cap \text{Im } f$ \textbf{est un sous-espace de } $\text{Im } f$: $\ker g$ est un sous-espace de F car c'est le noyau d'une application linéaire. $\text{Im } f$ est un sous-espace de F car c'est l'image d'une application linéaire. L'intersection de deux sous-espaces vectoriels est toujours un sous-espace vectoriel. Puisque $\ker g \cap \text{Im } f$ est contenu dans $\text{Im } f$, il est bien un sous-espace de $\text{Im } f$. Le quotient $\text{Im } f/(\ker g \cap \text{Im } f)$ est donc bien défini.

* \textbf{ } $\bar{g}$ \textbf{est bien définie} : Soient $Y_1 = y_1 + (\ker g \cap \text{Im } f)$ et $Y_2 = y_2 + (\ker g \cap \text{Im } f)$ deux représentants de la même classe dans le quotient. Cela signifie que $y_1 - y_2 \in \ker g \cap \text{Im } f$. Puisque $y_1 - y_2 \in \ker g$, nous avons $g(y_1 - y_2) = 0_G$. Par linéarité de g , $g(y_1) - g(y_2) = 0_G$, d'où $g(y_1) = g(y_2)$. Ainsi, $\bar{g}(Y_1) = g(y_1) = g(y_2) = \bar{g}(Y_2)$. L'application $\bar{g}$ est donc bien définie.

* \{\textbf{\textit{}}\} $\bar{g}$ est linéaire : Soient $Y_1 = y_1 + (\ker g \cap \text{Im } f)$ et $Y_2 = y_2 + (\ker g \cap \text{Im } f)$ deux éléments du quotient, et $\lambda \in \mathbb{K}$. $\bar{g}(Y_1 + Y_2) = \bar{g}((y_1 + y_2) + (\ker g \cap \text{Im } f)) = g(y_1 + y_2)$. Par linéarité de g , $g(y_1 + y_2) = g(y_1) + g(y_2) = \bar{g}(Y_1) + \bar{g}(Y_2)$. $\bar{g}(\lambda Y_1) = \bar{g}(\lambda y_1 + (\ker g \cap \text{Im } f)) = g(\lambda y_1)$. Par linéarité de g , $g(\lambda y_1) = \lambda g(y_1) = \lambda \bar{g}(Y_1)$. L'application $\bar{g}$ est donc linéaire.

* \{\textbf{\textit{}}\} $\bar{g}$ est surjective : Soit $z \in \text{Im}(g \circ f)$. Par définition, il existe $x \in E$ tel que $z = (g \circ f)(x)$. Cela signifie $z = g(f(x))$. Posons $y = f(x)$. Alors $y \in \text{Im } f$. L'élément $y + (\ker g \cap \text{Im } f)$ est un élément du domaine de $\bar{g}$. Alors $\bar{g}(y + (\ker g \cap \text{Im } f)) = g(y) = g(f(x)) = z$. Donc, pour tout $z \in \text{Im}(g \circ f)$, il existe un antécédent dans le domaine de $\bar{g}$. L'application $\bar{g}$ est surjective.

* \{\textbf{\textit{}}\} $\bar{g}$ est injective : Soit $Y = y + (\ker g \cap \text{Im } f)$ un élément du noyau de $\bar{g}$. Cela signifie $\bar{g}(Y) = 0_G$, c'est-à-dire $g(y) = 0_G$. Puisque $g(y) = 0_G$, y appartient à $\ker g$. Par ailleurs, par la définition du domaine du quotient, y appartient à $\text{Im } f$. Donc, $y \in \ker g \cap \text{Im } f$. Par conséquent, la classe $Y = y + (\ker g \cap \text{Im } f)$ est la classe zéro du quotient, i.e., $Y = \ker g \cap \text{Im } f$. Le noyau de $\bar{g}$ est réduit à l'élément neutre du quotient. L'application $\bar{g}$ est injective.

* \{\textbf{\textit{}}\} Conclusion : Puisque $\bar{g}$ est linéaire, bien définie, surjective et injective, c'est un isomorphisme de $\mathbb{K}$ -espaces vectoriels. Ainsi, $\text{Im } f / (\ker g \cap \text{Im } f) \cong \text{Im}(g \circ f)$.

4. \{\textbf{\textit{}}\} Application aux dimensions finies

Nous supposons E, F, G de dimensions finies.

a) \{\textbf{\textit{}}\} Égalité pour le noyau : D'après la question 2, nous avons l'isomorphisme $\ker(g \circ f) / \ker f \cong \ker g \cap \text{Im } f$. Pour des espaces vectoriels de dimensions finies, si $V_1 \cong V_2$, alors $\dim(V_1) = \dim(V_2)$. De plus, pour un quotient V/W , $\dim(V/W) = \dim(V) - \dim(W)$. Donc, $\dim(\ker(g \circ f) / \ker f) = \dim(\ker(g \circ f)) - \dim(\ker f)$. Par conséquent, $\dim(\ker(g \circ f)) - \dim(\ker f) = \dim(\ker g \cap \text{Im } f)$. Ce qui donne l'égalité souhaitée : $\dim(\ker(g \circ f)) = \dim(\ker f) + \dim(\ker g \cap \text{Im } f)$.

b) \{\textbf{\textit{}}\} Égalité pour l'image (rang) : D'après la question 3, nous avons l'isomorphisme $\text{Im } f / (\ker g \cap \text{Im } f) \cong \text{Im}(g \circ f)$. En appliquant le même principe des dimensions finies : $\dim(\text{Im } f / (\ker g \cap \text{Im } f)) = \dim(\text{Im } f) - \dim(\ker g \cap \text{Im } f)$. Et $\dim(\text{Im}(g \circ f)) = \text{rang}(g \circ f)$. De plus, $\dim(\text{Im } f) = \text{rang}(f)$. Par conséquent, $\text{rang}(g \circ f) = \text{rang}(f) - \dim(\ker g \cap \text{Im } f)$.

5. \{\textbf{\textit{}}\} Démonstration de l'inégalité de Sylvester

L'inégalité de Sylvester est de la forme $\text{rang}(f) + \text{rang}(g) - \dim(F) \leq \text{rang}(g \circ f) \leq \min(\text{rang } f, \text{rang } g)$. Nous allons prouver les deux parties de l'inégalité.

* \{\textbf{\textit{}}\} Borne supérieure : $\text{rang}(g \circ f) \leq \min(\text{rang } f, \text{rang } g)$: 1. \{\textbf{\textit{}}\} $\text{rang}(g \circ f) \leq \text{rang}(f)$: D'après la question 4b, $\text{rang}(g \circ f) = \text{rang}(f) - \dim(\ker g \cap \text{Im } f)$. Comme $\ker g \cap \text{Im } f$ est un sous-espace vectoriel, sa dimension est non-négative : $\dim(\ker g \cap \text{Im } f) \geq 0$. Par conséquent, $\text{rang}(g \circ f) \leq \text{rang}(f)$. On peut aussi observer que $\text{Im}(g \circ f) = g(\text{Im } f)$. L'image d'un sous-espace par une application linéaire a une dimension inférieure ou égale à la dimension de ce sous-espace. Ainsi, $\dim(g(\text{Im } f)) \leq \dim(\text{Im } f)$, ce qui signifie $\text{rang}(g \circ f) \leq \text{rang}(f)$.

2. \{\textbf{\textit{}}\} $\text{rang}(g \circ f) \leq \text{rang}(g)$: Comme précédemment, $\text{Im}(g \circ f) = g(\text{Im } f)$. Puisque $\text{Im } f \subseteq F$, tout vecteur dans $g(\text{Im } f)$ est aussi un vecteur dans $g(F)$, c'est-à-dire dans $\text{Im } g$. Donc $\text{Im}(g \circ f)$ est un sous-espace de $\text{Im } g$. Par conséquent, $\dim(\text{Im}(g \circ f)) \leq \dim(\text{Im } g)$, ce qui signifie $\text{rang}(g \circ f) \leq \text{rang}(g)$.

Ces deux inégalités prouvent que $\text{rang}(g \circ f) \leq \min(\text{rang } f, \text{rang } g)$.

* \{\textbf{\textit{}}\} Borne inférieure : $\text{rang}(f) + \text{rang}(g) - \dim(F) \leq \text{rang}(g \circ f)$: D'après la question 4b, nous avons $\text{rang}(g \circ f) = \text{rang}(f) - \dim(\ker g \cap \text{Im } f)$. Nous devons maintenant borner $\dim(\ker g \cap \text{Im } f)$. Le sous-espace $\ker g \cap \text{Im } f$ est un sous-espace de $\ker g$. Donc, $\dim(\ker g \cap \text{Im } f) \leq \dim(\ker g)$. En utilisant cette inégalité dans l'expression de $\text{rang}(g \circ f)$: $\text{rang}(g \circ f) \geq \text{rang}(f) - \dim(\ker g)$.

Maintenant, nous utilisons le théorème du rang pour l'application linéaire $g : F \rightarrow G$. Le théorème du rang stipule que $\dim(F) = \dim(\ker g) + \dim(\text{Im } g)$. Puisque $\dim(\text{Im } g) = \text{rang}(g)$, nous avons $\dim(F) = \dim(\ker g) + \text{rang}(g)$. D'où $\dim(\ker g) = \dim(F) - \text{rang}(g)$.

Substituons cette expression de $\dim(\ker g)$ dans l'inégalité pour $\text{rang}(g \circ f)$: $\text{rang}(g \circ f) \geq \text{rang}(f) - (\dim(F) - \text{rang}(g))$. En réarrangeant les termes, nous obtenons : $\text{rang}(g \circ f) \geq \text{rang}(f) + \text{rang}(g) - \dim(F)$.

Nous avons ainsi démontré les deux parties de l'inégalité de Sylvester.

Travaux Pratiques

\{\chapter*{TP 1 : Fondamentaux de l'Algèbre Linéaire pour l'IA : Applications, Noyau, Image et Théorème du Rang "From Scratch"}

\{\section*{Objectif}

Ce TP a pour but de vous faire implémenter, à partir de zéro et sans aucune librairie externe (comme NumPy ou SciPy), les concepts fondamentaux de l'algèbre linéaire que sont les applications linéaires, leur noyau (kernel), leur image (range) et le théorème du rang.

\{\subsection*{Objectif Mathématique} * Comprendre et représenter les vecteurs et matrices en Python pur (listes de nombres et listes de listes). * Implémenter les opérations de base de l'algèbre linéaire : addition de vecteurs, multiplication scalaire-vecteur, produit scalaire, et surtout la multiplication matrice-vecteur. * Saisir la définition formelle et l'implémentation d'une application linéaire comme une transformation représentée par une multiplication matricielle ($f(x) = Ax$). * Définir et vérifier l'appartenance d'un vecteur au noyau d'une application linéaire. * Définir et vérifier l'appartenance d'un vecteur à l'image d'une application linéaire (avec une méthode adaptée aux contraintes "from scratch"). * Comprendre et illustrer le Théorème du Rang qui établit une relation fondamentale entre la dimension de l'espace de départ, la dimension du noyau (nullité) et la dimension de l'image (rang).

\{\subsection*{Objectif IA} Les concepts d'algèbre linéaire sont des piliers de nombreux algorithmes en Intelligence Artificielle et en Machine Learning. * \{\textbf{Applications Linéaires} : Elles sont le cœur des couches "Dense" ou "Linéaires" des réseaux de neurones. Comprendre $f(x) = Ax$ est crucial pour saisir comment les données d'entrée sont transformées et projetées dans des espaces de caractéristiques de plus grande ou plus petite dimension. * \{\textbf{Noyau (Ker(f))} : Le noyau représente l'ensemble des entrées qui sont "écrasées" ou projetées sur le vecteur nul par l'application linéaire. En IA, cela peut être interprété comme une perte d'information, des caractéristiques redondantes, ou des directions dans l'espace d'entrée qui n'influencent pas la sortie. Il est lié aux concepts de réduction de dimension et de compression de données. * \{\textbf{Image (Im(f))} : L'image est l'ensemble de toutes les sorties possibles de l'application linéaire. Elle définit l'espace de "capacité expressive" d'un modèle linéaire. Comprendre l'image permet de savoir quelles transformations et quelles sorties un modèle peut effectivement produire, et quelle est la "richesse" de son espace de sortie. * \{\textbf{Théorème du Rang} : Ce théorème ($\dim(\text{Espace de Départ}) = \dim(\text{Noyau}) + \dim(\text{Image})$) fournit une relation fondamentale entre la dimension de l'espace d'entrée, la dimension de l'information "perdue" (noyau) et la dimension de l'information "transformée" (image). Il est essentiel pour l'analyse de la complexité, de la redondance et de la capacité de représentation des modèles d'apprentissage automatique. Une faible nullité (grand rang) signifie que l'application préserve beaucoup d'information, tandis qu'une grande nullité (petit rang) indique une forte compression ou perte d'information.

\{\section*{Implémentation en Python "From Scratch"}

Nous allons implémenter les fonctions nécessaires et les valider avec des assertions mathématiques pour prouver leur correction.

```
import math

# --- Fonctions Utilitaires de Base pour l'Algèbre Linéaire ---

def are_close(val1, val2, tolerance=1e-9):
    """
    Vérifie si deux valeurs flottantes sont considérées comme égales
    compte tenu des imprécisions de la virgule flottante.
    """
    return abs(val1 - val2) < tolerance

def is_zero_vector(v, tolerance=1e-9):
    """
    Vérifie si un vecteur est le vecteur nul (toutes ses composantes sont proches de
    zéro).
```

```

    """
    return all(are_close(x, 0.0, tolerance) for x in v)

def vector_add(v1, v2):
    """
    Additionne deux vecteurs composante par composante.
    Les vecteurs doivent avoir la même dimension.
    """
    if len(v1) != len(v2):
        raise ValueError("Les vecteurs doivent avoir la même dimension pour être additionnés.")
    return [v1[i] + v2[i] for i in range(len(v1))]

def scalar_vector_multiply(scalar, v):
    """
    Multiplie un vecteur par un scalaire.
    """
    return [scalar * x for x in v]

def dot_product(v1, v2):
    """
    Calcule le produit scalaire (produit interne) de deux vecteurs.
    Les vecteurs doivent avoir la même dimension.
    """
    if len(v1) != len(v2):
        raise ValueError("Les vecteurs doivent avoir la même dimension pour le produit scalaire.")
    return sum(v1[i] * v2[i] for i in range(len(v1)))

def matrix_vector_multiply(matrix, vector):
    """
    Multiplie une matrice par un vecteur.
    Cette opération est la représentation standard d'une application linéaire  $f(x) = Ax$ .

    Args:
        matrix (list of list of float): La matrice A, représentée comme une liste de lignes.
        vector (list of float): Le vecteur x.

    Returns:
        list of float: Le vecteur résultant  $y = Ax$ .
    """
    num_rows = len(matrix)
    if num_rows == 0:
        if len(vector) != 0:
            raise ValueError("La matrice est vide mais le vecteur ne l'est pas.")
        return [] # Matrice vide, vecteur vide -> résultat vide

    num_cols = len(matrix[0])
    if num_cols == 0 and num_rows > 0:
        raise ValueError("La matrice a des lignes mais pas de colonnes (vide).")

    if len(vector) != num_cols:
        raise ValueError(
            f"Le nombre de colonnes de la matrice ({num_cols}) "
            f"doit être égal à la dimension du vecteur ({len(vector)})."
        )

    result_vector = [0.0] * num_rows
    for i in range(num_rows):
        # Chaque composante du vecteur résultat est le produit scalaire de la i-ème
        # ligne de la matrice
        # avec le vecteur d'entrée.
        result_vector[i] = dot_product(matrix[i], vector)

```



```

    return result_vector

# --- Vérification de l'implémentation des utilitaires ---
print("--- Test des Fonctions Utilitaires de Base ---")
v_a = [1.0, 2.0, 3.0]
v_b = [4.0, 5.0, 6.0]
s = 2.0
matrix_A_test = [[1.0, 2.0, 3.0],
                  [4.0, 5.0, 6.0]]
vector_x_test = [1.0, 0.5, 0.1]

assert vector_add(v_a, v_b) == [5.0, 7.0, 9.0], "Test vector_add échoué"
assert scalar_vector_multiply(s, v_a) == [2.0, 4.0, 6.0], "Test
    scalar_vector_multiply échoué"
assert are_close(dot_product(v_a, v_b), 32.0), "Test dot_product échoué"

expected_mv_result = [1.0*1.0 + 2.0*0.5 + 3.0*0.1, 4.0*1.0 + 5.0*0.5 + 6.0*0.1] # [1
    + 1 + 0.3, 4 + 2.5 + 0.6] = [2.3, 7.1]
mv_result = matrix_vector_multiply(matrix_A_test, vector_x_test)
assert len(mv_result) == len(expected_mv_result) and \
    all(are_close(val, expected) for val, expected in zip(mv_result,
        expected_mv_result)), \
    f"Test matrix_vector_multiply échoué. Attendu {expected_mv_result}, Obtenu {
        mv_result}"

assert is_zero_vector([0.0, 0.0, 0.0]), "Test is_zero_vector pour zéro échoué"
assert not is_zero_vector([0.1, 0.0, 0.0]), "Test is_zero_vector pour non-zéro échoué"
print("Tests des Fonctions Utilitaires réussis !")

# --- 1. Application Linéaire ---

def linear_application(matrix, vector):
    """
    Définit une application linéaire  $f(x) = Ax$ .
    C'est conceptuellement un alias pour matrix_vector_multiply pour la clarté.
    """
    return matrix_vector_multiply(matrix, vector)

print("\n--- 1. Application Linéaire ---")

# Matrice pour notre application linéaire,  $\mathbb{R}^2 \rightarrow \mathbb{R}^2$ 
A_linear = [[1.0, 2.0],
            [3.0, 4.0]]

# Vecteurs d'entrée pour tester les propriétés de linéarité
x1 = [1.0, 0.0]
x2 = [0.0, 1.0]
x_sum = vector_add(x1, x2) # [1.0, 1.0]
c = 2.0
x_scaled = scalar_vector_multiply(c, x1) # [2.0, 0.0]

# Appliquer la transformation
y1 = linear_application(A_linear, x1) # [1.0, 3.0]
y2 = linear_application(A_linear, x2) # [2.0, 4.0]
y_sum = linear_application(A_linear, x_sum) # [3.0, 7.0]
y_scaled = linear_application(A_linear, x_scaled) # [2.0, 6.0]

# Test de la propriété d'additivité :  $f(x1 + x2) = f(x1) + f(x2)$ 
expected_y_sum = vector_add(y1, y2) # [1+2, 3+4] = [3.0, 7.0]
assert len(y_sum) == len(expected_y_sum) and \
    all(are_close(val, expected) for val, expected in zip(y_sum, expected_y_sum)),
    \
    f"Test d'additivité de l'application linéaire échoué.  $f(x1+x2)={y\_sum}$ ,  $f(x1)+f($ 
         $x2)={expected\_y\_sum}$ "
print(f"f({x1} + {x2}) = {y_sum}")

```

```

print(f"f({x1}) + f({x2}) = {expected_y_sum}")
print("Test d'additivité de l'application linéaire réussi.")

# Test de la propriété d'homogénéité : f(c * x1) = c * f(x1)
expected_y_scaled = scalar_vector_multiply(c, y1) # 2 * [1.0, 3.0] = [2.0, 6.0]
assert len(y_scaled) == len(expected_y_scaled) and \
    all(are_close(val, expected) for val, expected in zip(y_scaled,
        expected_y_scaled)), \
    f"Test d'homogénéité de l'application linéaire échoué. f(c*x1)={y_scaled}, c*f(x1)={expected_y_scaled}"
print(f"f({c} * {x1}) = {y_scaled}")
print(f"{c} * f({x1}) = {expected_y_scaled}")
print("Test d'homogénéité de l'application linéaire réussi.")

print("L'implémentation de l'application linéaire est correcte et respecte les propriétés de linéarité.")

# --- 2. Noyau (Kernel) ---

def is_in_kernel(matrix, vector, tolerance=1e-9):
    """
    Vérifie si un vecteur appartient au noyau d'une application linéaire définie par une matrice.
    Un vecteur v est dans le noyau de A si Av = 0.
    """
    result = matrix_vector_multiply(matrix, vector)
    return is_zero_vector(result, tolerance)

print("\n--- 2. Noyau (Kernel) ---")

# Exemple 1 : Matrice identité (le noyau est trivial : seulement le vecteur nul)
I_2x2 = [[1.0, 0.0],
         [0.0, 1.0]]
v_zero = [0.0, 0.0]
v_non_zero = [1.0, 2.0]

assert is_in_kernel(I_2x2, v_zero), "Le vecteur nul doit être dans le noyau de la matrice identité."
assert not is_in_kernel(I_2x2, v_non_zero), "Un vecteur non nul ne doit pas être dans le noyau de la matrice identité."
print("Tests pour la matrice identité réussis : le noyau est trivial.")

# Exemple 2 : Matrice avec un noyau non trivial
# Matrice A_kernel : R^2 -> R^2, où la deuxième colonne est un multiple de la première.
# Cela signifie que les colonnes ne sont pas linéairement indépendantes, donc il y a un noyau non trivial.
A_kernel = [[1.0, 2.0],
            [2.0, 4.0]]
# Pour cette matrice, tout vecteur de la forme k * [-2, 1] est dans le noyau.
# Par exemple, si k=1, x_k = [-2, 1].
# A_kernel @ x_k = [1*(-2) + 2*1, 2*(-2) + 4*1] = [-2+2, -4+4] = [0, 0]
x_kernel_candidate = [-2.0, 1.0]

assert is_in_kernel(A_kernel, x_kernel_candidate), \
    f"Le vecteur {x_kernel_candidate} devrait être dans le noyau de A_kernel."
print(f"Vérification du noyau pour A_kernel: {A_kernel}")
print(f"A_kernel @ {x_kernel_candidate} = {matrix_vector_multiply(A_kernel, x_kernel_candidate)}")
print(f"Le vecteur {x_kernel_candidate} est bien dans le noyau de la matrice A_kernel.")

# Un autre vecteur dans le noyau (un multiple du premier)
x_kernel_candidate_2 = [-4.0, 2.0] # 2 * [-2, 1]
assert is_in_kernel(A_kernel, x_kernel_candidate_2), \
    f"Le vecteur {x_kernel_candidate_2} (multiple) devrait être dans le noyau de

```

```

        A_kernel."
print(f"Le vecteur {x_kernel_candidate_2} est bien dans le noyau de la matrice
      A_kernel.")

# Un vecteur qui n'est PAS dans le noyau
x_not_kernel = [1.0, 1.0]
assert not is_in_kernel(A_kernel, x_not_kernel), \
    f"Le vecteur {x_not_kernel} ne devrait PAS être dans le noyau de A_kernel."
print(f"A_kernel @ {x_not_kernel} = {matrix_vector_multiply(A_kernel, x_not_kernel)}")
print(f"Le vecteur {x_not_kernel} n'est PAS dans le noyau de la matrice A_kernel.")

print("Implémentation et tests du noyau réussis.")

# --- 3. Image (Range) ---

def is_in_image(matrix, vector_y, input_vector_x=None, tolerance=1e-9):
    """
    Vérifie si un vecteur y appartient à l'image d'une application linéaire A.
    Ceci est vérifié si  $y = Ax$  pour un certain vecteur x.

    Pour une implémentation "from scratch" sans résolution de système linéaire ( $Ax = y$ ),
    nous testons si un 'input_vector_x' (qui est censé produire 'vector_y') est
    fourni.
    Si 'input_vector_x' n'est pas fourni, cette fonction ne peut pas déterminer l'
    appartenance
    pour un vecteur 'y' arbitraire sans résoudre  $Ax = y$ , ce qui est une tâche plus
    complexe
    (nécessitant l'élimination de Gauss ou des factorisations matricielles) et dé
    passe
    le cadre de ce TP 1.

    Args:
        matrix (list of list of float): La matrice A de l'application linéaire.
        vector_y (list of float): Le vecteur à vérifier s'il est dans l'image.
        input_vector_x (list of float, optional): Un vecteur x tel que  $A @ x = y$ .
            Obligatoire pour cette implé
            mentation simple.
        tolerance (float): La tolérance pour la comparaison des flottants.

    Returns:
        bool: True si  $A @ \text{input\_vector\_x}$  est égal à vector_y, False sinon.
    """
    if input_vector_x is None:
        raise ValueError(
            "Pour vérifier si un vecteur est dans l'image 'from scratch' sans solveur
            , "
            "vous devez fournir un 'input_vector_x' tel que  $A @ x = y$ . "
            "La résolution générale de  $Ax=y$  n'est pas implémentée ici."
        )

    result_y_computed = matrix_vector_multiply(matrix, input_vector_x)

    if len(result_y_computed) != len(vector_y):
        return False # Les dimensions ne correspondent pas

    return all(are_close(val_res, val_expected, tolerance) for val_res, val_expected
                in zip(result_y_computed, vector_y))

print("\n--- 3. Image (Range) ---")

# Matrice pour l'image : A_image :  $\mathbb{R}^2 \rightarrow \mathbb{R}^3$ 
# Les colonnes de A_image sont [1,0,1] et [0,1,1]. L'image est le plan engendré par
ces deux vecteurs dans  $\mathbb{R}^3$ .

```

```

A_image = [[1.0, 0.0],
            [0.0, 1.0],
            [1.0, 1.0]]

# Vecteur d'entrée qui génère un vecteur dans l'image
x_image_input = [1.0, 2.0] # Un vecteur de  $\mathbb{R}^2$ 
y_in_image = linear_application(A_image, x_image_input) #  $y = A\_image @ x\_image\_input$ 
                = [1, 2, 3]

# Vérifier que y_in_image est bien dans l'image en utilisant l'input_vector_x
assert is_in_image(A_image, y_in_image, x_image_input), \
    f"Le vecteur {y_in_image} devrait être dans l'image de A_image avec l'entrée {
        x_image_input}."
print(f"Matrice A_image: {A_image}")
print(f"Vecteur d'entrée x_image_input: {x_image_input}")
print(f"Vecteur de sortie y_in_image (A_image @ x_image_input): {y_in_image}")
print(f"Le vecteur {y_in_image} est bien dans l'image de A_image.")

# Un autre vecteur dans l'image (obtenu avec une autre entrée)
x_image_input_2 = [3.0, -1.0]
y_in_image_2 = linear_application(A_image, x_image_input_2) #  $y = [3, -1, 2]$ 
assert is_in_image(A_image, y_in_image_2, x_image_input_2), \
    f"Le vecteur {y_in_image_2} devrait être dans l'image de A_image avec l'entrée {
        x_image_input_2}."
print(f"Un autre vecteur {y_in_image_2} est aussi dans l'image de A_image.")

# Un vecteur qui n'est PAS dans l'image (par exemple, non sur le plan généré par les
    colonnes de A_image)
# L'image de A_image est le plan défini par les vecteurs [1,0,1] et [0,1,1].
# Un vecteur [a,b,c] est dans l'image si  $c = a + b$ .
y_not_in_image = [1.0, 1.0, 3.0] # Ici,  $3 \neq 1 + 1$ . Ce vecteur n'est pas sur le plan.

print(f"\nVecteur y_not_in_image (conceptuellement non dans l'image): {y_not_in_image
    }")
print("Sans un solveur pour  $Ax=y$ , il est difficile de vérifier qu'un vecteur N'EST
    PAS dans l'image 'from scratch'.")
print("Notre fonction 'is_in_image' est conçue pour la *vérification* étant donné un
    x.")
try:
    is_in_image(A_image, y_not_in_image) # Cela doit lever une erreur car
        input_vector_x est manquant
except ValueError as e:
    print(f"Erreur attendue (pas d'input_vector_x fourni pour la vérification né
        gative) : {e}")

print("Implémentation et tests de l'image (avec x fourni) réussis.")

# --- 4. Théorème du Rang ---

# Le Théorème du Rang :  $\dim(\text{Espace de Départ}) = \dim(\text{Noyau}) + \dim(\text{Image})$ 
# (où  $\dim(\text{Noyau})$  est la nullité et  $\dim(\text{Image})$  est le rang)

# Comme l'implémentation robuste de la détermination du rang et de la nullité
# pour une matrice arbitraire (qui nécessite la réduction de Gauss-Jordan)
# est très complexe "from scratch" et dépasse le cadre de ce TP 1,
# nous allons illustrer le théorème avec des matrices où le rang et la nullité
# sont connus par inspection ou par construction simple.

print("\n--- 4. Théorème du Rang ---")

# Exemple 1 : Matrice identité 2x2
# Application de  $\mathbb{R}^2 \rightarrow \mathbb{R}^2$ 
I_2x2 = [[1.0, 0.0],
          [0.0, 1.0]]
dim_domain_I = 2 # Dimension de l'espace de départ ( $\mathbb{R}^2$ )

```

```

rank_I = 2                # Deux colonnes linéairement indépendantes => image de
    dimension 2
nullity_I = 0             # Seul le vecteur nul est dans le noyau => noyau de dimension
    0

print("\nExemple 1 : Matrice Identité 2x2 (Transformation qui ne perd pas d'
    information)")
print(f"Matrice: {I_2x2}")
print(f"Dimension de l'espace de départ ( $\mathbb{R}^2$ ): {dim_domain_I}")
print(f"Rang (dimension de l'image): {rank_I}")
print(f"Nullité (dimension du noyau): {nullity_I}")

assert dim_domain_I == rank_I + nullity_I, "Le Théorème du Rang ne tient pas pour la
    matrice identité."
print(f"Vérification du Théorème du Rang : {dim_domain_I} == {rank_I} + {nullity_I} (
    Vrai)")

# Exemple 2 : Matrice de projection 2x2 sur un axe (image de dimension 1, perte d'
    information)
# Application de  $\mathbb{R}^2 \rightarrow \mathbb{R}^2$ 
# Projette sur l'axe des x
P_x = [[1.0, 0.0],
        [0.0, 0.0]]
dim_domain_P = 2          # Dimension de l'espace de départ ( $\mathbb{R}^2$ )
rank_P = 1                # L'image est l'axe des x (engendrée par [1,0]), dimension 1
# Le noyau est l'axe des y (vecteurs de la forme [0, k]), dimension 1.
# Car  $P_x \cdot [0, k] = [0, 0]$ 
nullity_P = 1

print("\nExemple 2 : Matrice de Projection 2x2 (Perte d'information le long de l'axe
    y)")
print(f"Matrice: {P_x}")
print(f"Dimension de l'espace de départ ( $\mathbb{R}^2$ ): {dim_domain_P}")
print(f"Rang (dimension de l'image): {rank_P}")
print(f"Nullité (dimension du noyau): {nullity_P}")

assert dim_domain_P == rank_P + nullity_P, "Le Théorème du Rang ne tient pas pour la
    matrice de projection."
print(f"Vérification du Théorème du Rang : {dim_domain_P} == {rank_P} + {nullity_P} (
    Vrai)")

# Exemple 3 : Matrice 3x2 avec rang 2 (pas de perte de dimension si l'espace de dé
    part est  $\mathbb{R}^2$ )
# Application de  $\mathbb{R}^2 \rightarrow \mathbb{R}^3$  (injection si le noyau est trivial)
# C'est notre A_image précédente
A_rank_3x2 = [[1.0, 0.0],
               [0.0, 1.0],
               [1.0, 1.0]]
dim_domain_A_3x2 = 2      # L'espace de départ est  $\mathbb{R}^2$ 
rank_A_3x2 = 2            # Les deux colonnes sont linéairement indépendantes dans  $\mathbb{R}^3$ .
    L'image est un plan dans  $\mathbb{R}^3$ .
nullity_A_3x2 = 0         # Le noyau est trivial, seul le vecteur nul de  $\mathbb{R}^2$  donne 0 en
     $\mathbb{R}^3$ .

print("\nExemple 3 : Matrice 3x2 ( $\mathbb{R}^2 \rightarrow \mathbb{R}^3$ , pas de perte d'information de l'espace
    de départ)")
print(f"Matrice: {A_rank_3x2}")
print(f"Dimension de l'espace de départ ( $\mathbb{R}^2$ ): {dim_domain_A_3x2}")
print(f"Rang (dimension de l'image, un plan dans  $\mathbb{R}^3$ ): {rank_A_3x2}")
print(f"Nullité (dimension du noyau, seulement le vecteur nul de  $\mathbb{R}^2$ ): {nullity_A_3x2
    }")

assert dim_domain_A_3x2 == rank_A_3x2 + nullity_A_3x2, "Le Théorème du Rang ne tient
    pas pour la matrice 3x2."

```

```

print(f"Vérification du Théorème du Rang : {dim_domain_A_3x2} == {rank_A_3x2} + {
    nullity_A_3x2} (Vrai)")

# Exemple 4 : Matrice 2x3 de rang 1 (réduit fortement la dimension, compression)
# Application de  $\mathbb{R}^3 \rightarrow \mathbb{R}^2$ 
# Toutes les lignes sont des multiples l'une de l'autre
A_rank_2x3 = [[1.0, 2.0, 3.0],
               [2.0, 4.0, 6.0]]
dim_domain_A_2x3 = 3      # L'espace de départ est  $\mathbb{R}^3$ 
rank_A_2x3 = 1            # L'image est une droite dans  $\mathbb{R}^2$  (engendrée par [1,2]).
    Toutes les colonnes sont multiples de la première.
# La nullité est  $\dim(\text{domain}) - \text{rank} = 3 - 1 = 2$ .
# Le noyau est un plan dans  $\mathbb{R}^3$ . Par exemple, [-2, 1, 0] et [-3, 0, 1] forment une
    base du noyau.
nullity_A_2x3 = 2

print("\nExemple 4 : Matrice 2x3 ( $\mathbb{R}^3 \rightarrow \mathbb{R}^2$ , compression forte, perte d'information)
    ")
print(f"Matrice: {A_rank_2x3}")
print(f"Dimension de l'espace de départ ( $\mathbb{R}^3$ ): {dim_domain_A_2x3}")
print(f"Rang (dimension de l'image, une droite dans  $\mathbb{R}^2$ ): {rank_A_2x3}")
print(f"Nullité (dimension du noyau, un plan dans  $\mathbb{R}^3$ ): {nullity_A_2x3}")

assert dim_domain_A_2x3 == rank_A_2x3 + nullity_A_2x3, "Le Théorème du Rang ne tient
    pas pour la matrice 2x3."
print(f"Vérification du Théorème du Rang : {dim_domain_A_2x3} == {rank_A_2x3} + {
    nullity_A_2x3} (Vrai)")

print("\nConclusion des tests du Théorème du Rang : les relations dimensionnelles
    sont vérifiées pour ces exemples.")
print("\nFélicitations ! Tous les tests pour le TP 1 ont été réussis. Vous avez implé
    menté des concepts clés de l'algèbre linéaire \"from scratch\"!")

```

\{\chapter*{TP 2 : Applications Linéaires, Noyau, Image et Théorème du Rang

\{\section*{Objectifs du TP

Ce TP a pour objectif d'implémenter de manière "from scratch" (sans l'aide de bibliothèques tierces comme NumPy) les concepts fondamentaux d'algèbre linéaire que sont les applications linéaires, leur noyau (null space), leur image (column space) et le théorème du rang.

\{\subsection*{Objectifs Mathématiques

* Comprendre et coder les opérations de base sur les vecteurs et les matrices (addition, multiplication scalaire, multiplication matrice-vecteur). * Implémenter l'algorithme d'élimination de Gauss-Jordan pour réduire une matrice à sa forme échelonnée réduite par lignes (RREF). * À partir de la RREF, déterminer une base pour le noyau d'une application linéaire (espace nul de la matrice associée). * À partir de la RREF, déterminer une base pour l'image d'une application linéaire (espace colonne de la matrice associée). * Calculer le rang d'une matrice (dimension de l'image) et la nullité (dimension du noyau). * Vérifier le théorème du rang ($\text{rang}(A) + \text{nullité}(A) = \text{dimension de l'espace de départ}$).

\{\subsection*{Objectifs IA

Les concepts d'applications linéaires, de noyau et d'image sont omniprésents en IA : * \{\textbf{Applications linéaires} modélisent les couches fully-connected des réseaux de neurones, les transformations de caractéristiques, les modèles de régression linéaire, ou les opérateurs de convolution. * \{\textbf{L'image d'une application linéaire} représente l'espace des sorties possibles, c'est-à-dire les "informations transformées" ou les "caractéristiques extraites" par un modèle. Sa dimension (le rang) indique la complexité, la capacité de transformation, ou les degrés de liberté d'un modèle. En PCA, l'espace des composantes principales est l'image d'une projection linéaire. * \{\textbf{Le noyau} représente les entrées qui sont "écrasées" ou "rendues invisibles" par la transformation, c'est-à-dire les "informations perdues" ou les "invariances" du modèle. Comprendre le noyau aide à déceler les limitations, les redondances ou les biais d'un modèle. Par exemple, dans une tâche de classification d'images, le noyau pourrait représenter des transformations de l'image qui ne changent pas la prédiction du modèle. * Le \{\textbf{théorème du rang} est fondamental pour la réduction de dimensionnalité, l'analyse des architectures de réseaux de neurones (où un rang faible peut indiquer une capacité limitée), et la compréhension des relations entre les données d'entrée et de sortie d'un système.

\{\section*{Implémentation en Python Pur

Nous allons définir des fonctions pour gérer les vecteurs et les matrices, et implémenter toutes les opérations nécessaires "from scratch".

```
import math

# --- Configuration pour les comparaisons de flottants ---
EPSILON = 1e-9 # Petite valeur pour gérer les imprécisions des flottants

# --- Fonctions utilitaires de base sur les vecteurs et matrices ---

def are_almost_equal(a: float, b: float) -> bool:
    """Vérifie si deux flottants sont presque égaux."""
    return abs(a - b) < EPSILON

def vec_add(v1: list[float], v2: list[float]) -> list[float]:
    """Additionne deux vecteurs."""
    assert len(v1) == len(v2), "Les vecteurs doivent avoir la même dimension pour l'addition."
    return [v1[i] + v2[i] for i in range(len(v1))]

def vec_sub(v1: list[float], v2: list[float]) -> list[float]:
    """Soustrait deux vecteurs."""
    assert len(v1) == len(v2), "Les vecteurs doivent avoir la même dimension pour la soustraction."
    return [v1[i] - v2[i] for i in range(len(v1))]

def scalar_vec_mul(scalar: float, v: list[float]) -> list[float]:
    """Multiplie un vecteur par un scalaire."""
    return [scalar * x for x in v]

def dot_product(v1: list[float], v2: list[float]) -> float:
    """Calcule le produit scalaire de deux vecteurs."""
    assert len(v1) == len(v2), "Les vecteurs doivent avoir la même dimension pour le produit scalaire."
    return sum(v1[i] * v2[i] for i in range(len(v1)))
```

```

def vec_norm_squared(v: list[float]) -> float:
    """Calcule la norme euclidienne au carré d'un vecteur."""
    return sum(x*x for x in v)

def vec_norm(v: list[float]) -> float:
    """Calcule la norme euclidienne d'un vecteur."""
    return math.sqrt(vec_norm_squared(v))

def is_zero_vector(v: list[float]) -> bool:
    """Vérifie si un vecteur est le vecteur nul."""
    return all(are_almost_equal(x, 0.0) for x in v)

def get_row(matrix: list[list[float]], row_idx: int) -> list[float]:
    """Récupère une ligne de la matrice."""
    return matrix[row_idx]

def get_col(matrix: list[list[float]], col_idx: int) -> list[float]:
    """Récupère une colonne de la matrice."""
    return [matrix[i][col_idx] for i in range(len(matrix))]

def matrix_transpose(matrix: list[list[float]]) -> list[list[float]]:
    """Calcule la transposée d'une matrice."""
    if not matrix or not matrix[0]:
        return []
    num_rows = len(matrix)
    num_cols = len(matrix[0])
    transposed = [[0.0 for _ in range(num_rows)] for _ in range(num_cols)]
    for i in range(num_rows):
        for j in range(num_cols):
            transposed[j][i] = matrix[i][j]
    return transposed

def mat_vec_mul(matrix: list[list[float]], vector: list[float]) -> list[float]:
    """Multiplie une matrice par un vecteur."""
    if not matrix:
        return []
    num_rows = len(matrix)
    num_cols = len(matrix[0])
    assert num_cols == len(vector), "Le nombre de colonnes de la matrice doit correspondre à la dimension du vecteur."
    result = [0.0] * num_rows
    for i in range(num_rows):
        result[i] = dot_product(matrix[i], vector)
    return result

# --- Implémentation de l'élimination de Gauss-Jordan (RREF) ---

def swap_rows(matrix: list[list[float]], r1: int, r2: int):
    """Échange deux lignes d'une matrice."""
    matrix[r1], matrix[r2] = matrix[r2], matrix[r1]

def multiply_row(matrix: list[list[float]], r: int, scalar: float):
    """Multiplie une ligne d'une matrice par un scalaire."""
    for j in range(len(matrix[0])):
        matrix[r][j] *= scalar

def add_multiple_of_row(matrix: list[list[float]], target_row: int, source_row: int, scalar: float):
    """Ajoute un multiple d'une ligne à une autre ligne."""
    for j in range(len(matrix[0])):
        matrix[target_row][j] += scalar * matrix[source_row][j]

def get_rref(matrix_in: list[list[float]]) -> list[list[float]]:
    """
    Convertit une matrice en sa forme échelonnée réduite par lignes (RREF).
    """

```



```

Implémente l'algorithme de Gauss-Jordan.
"""
matrix = [row[:] for row in matrix_in] # Créer une copie pour ne pas modifier l'
original

if not matrix or not matrix[0]:
    return []

num_rows = len(matrix)
num_cols = len(matrix[0])

pivot_row = 0
for col in range(num_cols):
    if pivot_row >= num_rows:
        break

    # 1. Trouver une ligne avec un élément non nul dans la colonne courante (
    pivot)
    pivot_candidate_row = pivot_row
    while pivot_candidate_row < num_rows and are_almost_equal(matrix[
        pivot_candidate_row][col], 0.0):
        pivot_candidate_row += 1

    if pivot_candidate_row == num_rows:
        # Aucune ligne non nulle trouvée pour cette colonne, passer à la suivante
        continue

    # 2. Échanger la ligne trouvée avec la ligne pivot_row
    swap_rows(matrix, pivot_row, pivot_candidate_row)

    # 3. Normaliser la ligne pivot pour que le pivot soit 1
    pivot_val = matrix[pivot_row][col]
    # Éviter la division par zéro si un pivot devient ~0 à cause d'erreurs
    flottantes intermédiaires
    if are_almost_equal(pivot_val, 0.0):
        continue # Si le pivot est presque zéro, il ne peut pas être un pivot
        valide pour cette colonne.
    multiply_row(matrix, pivot_row, 1.0 / pivot_val)

    # 4. Éliminer les autres éléments de la colonne (rendre les autres éléments
    0)
    for r in range(num_rows):
        if r != pivot_row:
            factor = -matrix[r][col]
            add_multiple_of_row(matrix, r, pivot_row, factor)

    pivot_row += 1

return matrix

# --- Fonctions pour le noyau, l'image et le rang ---

def get_pivot_columns(rref_matrix: list[list[float]]) -> list[int]:
    """
    Détermine les indices des colonnes pivots d'une matrice en RREF.
    Une colonne est pivot si elle contient un 1 qui est le premier élément non nul de
    sa ligne.
    """
    if not rref_matrix or not rref_matrix[0]:
        return []

    pivot_cols = []
    num_rows = len(rref_matrix)
    num_cols = len(rref_matrix[0])

    # Track the last pivot column found to ensure we only look to the right for the

```

```

        next pivot
    current_pivot_col = -1
    for r in range(num_rows):
        found_pivot_in_row = False
        for c in range(current_pivot_col + 1, num_cols):
            if are_almost_equal(rref_matrix[r][c], 1.0):
                # Check if it's the first non-zero element in its row up to this
                point
                if all(are_almost_equal(rref_matrix[r][k], 0.0) for k in range(c)):
                    pivot_cols.append(c)
                    current_pivot_col = c
                    found_pivot_in_row = True
                    break
        if not found_pivot_in_row: # Si pas de pivot dans cette ligne, elle est nulle
            ou dépendante
            continue
    return pivot_cols

def get_rank(rref_matrix: list[list[float]]) -> int:
    """Calcule le rang de la matrice (dimension de l'image)."""
    return len(get_pivot_columns(rref_matrix))

def get_basis_image(original_matrix: list[list[float]], rref_matrix: list[list[float]]) -> list[list[float]]:
    """
    Détermine une base pour l'image (espace colonne) de la matrice.
    Les vecteurs de base sont les colonnes de la matrice originale correspondant aux
    colonnes pivots de la RREF.
    """
    if not original_matrix or not original_matrix[0]:
        return []
    pivot_cols = get_pivot_columns(rref_matrix)
    basis = [get_col(original_matrix, col_idx) for col_idx in pivot_cols]
    return basis

def get_basis_kernel(rref_matrix: list[list[float]]) -> list[list[float]]:
    """
    Détermine une base pour le noyau (null space) de la matrice.
    Résout  $A \cdot x = 0$  en exprimant les variables de base en fonction des variables
    libres.
    """
    if not rref_matrix or not rref_matrix[0]:
        return []

    num_rows = len(rref_matrix)
    num_cols = len(rref_matrix[0])

    pivot_cols = get_pivot_columns(rref_matrix)
    free_vars = [c for c in range(num_cols) if c not in pivot_cols]

    if not free_vars: # Si pas de variables libres, le noyau est trivial (seulement
        le vecteur nul)
        return []

    basis_kernel = []

    # Dictionnaire pour associer l'index de colonne pivot à l'index de ligne où il
    est pivot
    pivot_col_to_row = {}
    current_pivot_col_idx = 0
    for r in range(num_rows):
        if current_pivot_col_idx < len(pivot_cols):
            col_idx = pivot_cols[current_pivot_col_idx]
            if are_almost_equal(rref_matrix[r][col_idx], 1.0) and all(
                are_almost_equal(rref_matrix[r][k], 0.0) for k in range(col_idx)):
                pivot_col_to_row[col_idx] = r
            current_pivot_col_idx += 1

```

```

        current_pivot_col_idx += 1

# Pour chaque variable libre, construire un vecteur de base
for free_var_idx in free_vars:
    basis_vector = [0.0] * num_cols
    basis_vector[free_var_idx] = 1.0 # La variable libre courante est 1

    # Remplir les valeurs des variables de base
    # Pour chaque variable de base x_j (où j est un pivot_col), trouver son é
    # quation
    # et exprimer x_j en fonction des variables libres.
    for basic_var_col_idx in pivot_cols:
        # Trouver la ligne où basic_var_col_idx est un pivot
        pivot_r = pivot_col_to_row[basic_var_col_idx]

        # L'équation de cette ligne (dans RREF) est:
        # 1 * x_basic_var_col_idx + sum(coeff * x_free) = 0
        # Donc, x_basic_var_col_idx = -sum(coeff * x_free)

        sum_free_coeffs = 0.0
        for fv in free_vars:
            # Seule la variable libre courante (free_var_idx) est 1 dans
            # basis_vector
            sum_free_coeffs += rref_matrix[pivot_r][fv] * basis_vector[fv] #
            # basis_vector[fv] est 1 si fv==free_var_idx, 0 sinon

        basis_vector[basic_var_col_idx] = -sum_free_coeffs

    basis_kernel.append(basis_vector)

return basis_kernel

def get_nullity(rref_matrix: list[list[float]]) -> int:
    """Calcule la nullité de la matrice (dimension du noyau)."""
    if not rref_matrix:
        return 0
    num_cols = len(rref_matrix[0]) if rref_matrix else 0
    return num_cols - get_rank(rref_matrix) # Théorème du rang : nullité =
    dim_domaine - rang

# --- Tests et assertions de validation ---

print("--- Début des tests ---")

# Matrice A pour les tests (3x4)
A = [
    [1.0, 2.0, 3.0, 4.0],
    [5.0, 6.0, 7.0, 8.0],
    [9.0, 10.0, 11.0, 12.0]
]

# Matrice B (plus simple, 3x3, rang 2)
B = [
    [1.0, 1.0, 2.0],
    [2.0, 2.0, 4.0],
    [0.0, 1.0, 1.0]
]

# Matrice C (3x3, rang 1)
C = [
    [1.0, 2.0, 1.0],
    [2.0, 4.0, 2.0],
    [3.0, 6.0, 3.0]
]

```

```

# Matrice D (identité, 3x3, rang 3)
D = [
    [1.0, 0.0, 0.0],
    [0.0, 1.0, 0.0],
    [0.0, 0.0, 1.0]
]

# Matrice E (nulle, 2x3, rang 0)
E = [
    [0.0, 0.0, 0.0],
    [0.0, 0.0, 0.0]
]

# --- Test des opérations de base ---
print("\n# Tests des opérations de base #")
v1 = [1.0, 2.0, 3.0]
v2 = [4.0, 5.0, 6.0]
assert vec_add(v1, v2) == [5.0, 7.0, 9.0]
assert scalar_vec_mul(2.0, v1) == [2.0, 4.0, 6.0]
assert dot_product(v1, v2) == 32.0 # 1*4 + 2*5 + 3*6 = 4 + 10 + 18 = 32
assert are_almost_equal(vec_norm(v1), math.sqrt(14.0)) # 1^2 + 2^2 + 3^2 = 1 + 4 + 9 = 14

mat_test_mv = [
    [1.0, 2.0],
    [3.0, 4.0]
]
vec_test_mv = [5.0, 6.0]
# [1*5 + 2*6, 3*5 + 4*6] = [5+12, 15+24] = [17, 39]
assert mat_vec_mul(mat_test_mv, vec_test_mv) == [17.0, 39.0]
print("Opérations de base : OK")

# --- Test get_rref ---
print("\n# Tests de get_rref #")
rref_A = get_rref(A)
expected_rref_A = [
    [1.0, 0.0, -1.0, -2.0],
    [0.0, 1.0, 2.0, 3.0],
    [0.0, 0.0, 0.0, 0.0]
]
for r_idx in range(len(rref_A)):
    for c_idx in range(len(rref_A[0])):
        assert are_almost_equal(rref_A[r_idx][c_idx], expected_rref_A[r_idx][c_idx]),
            f"RREF de A - Erreur à ({r_idx},{c_idx}): Attendu {expected_rref_A[r_idx][c_idx]}, Obtenu {rref_A[r_idx][c_idx]}"
print("RREF de A : OK")

rref_B = get_rref(B)
expected_rref_B = [
    [1.0, 0.0, 1.0],
    [0.0, 1.0, 1.0],
    [0.0, 0.0, 0.0]
]
for r_idx in range(len(rref_B)):
    for c_idx in range(len(rref_B[0])):
        assert are_almost_equal(rref_B[r_idx][c_idx], expected_rref_B[r_idx][c_idx]),
            f"RREF de B - Erreur à ({r_idx},{c_idx}): Attendu {expected_rref_B[r_idx][c_idx]}, Obtenu {rref_B[r_idx][c_idx]}"
print("RREF de B : OK")

rref_C = get_rref(C)
expected_rref_C = [
    [1.0, 2.0, 1.0],

```

```

    [0.0, 0.0, 0.0],
    [0.0, 0.0, 0.0]
]
for r_idx in range(len(rref_C)):
    for c_idx in range(len(rref_C[0])):
        assert are_almost_equal(rref_C[r_idx][c_idx], expected_rref_C[r_idx][c_idx]),
            \
            f"RREF de C - Erreur à ({r_idx},{c_idx}): Attendu {expected_rref_C[r_idx]
              ][c_idx]}, Obtenu {rref_C[r_idx][c_idx]}"
print("RREF de C : OK")

rref_D = get_rref(D)
expected_rref_D = [
    [1.0, 0.0, 0.0],
    [0.0, 1.0, 0.0],
    [0.0, 0.0, 1.0]
]
for r_idx in range(len(rref_D)):
    for c_idx in range(len(rref_D[0])):
        assert are_almost_equal(rref_D[r_idx][c_idx], expected_rref_D[r_idx][c_idx]),
            \
            f"RREF de D - Erreur à ({r_idx},{c_idx}): Attendu {expected_rref_D[r_idx]
              ][c_idx]}, Obtenu {rref_D[r_idx][c_idx]}"
print("RREF de D : OK")

rref_E = get_rref(E)
expected_rref_E = [
    [0.0, 0.0, 0.0],
    [0.0, 0.0, 0.0]
]
for r_idx in range(len(rref_E)):
    for c_idx in range(len(rref_E[0])):
        assert are_almost_equal(rref_E[r_idx][c_idx], expected_rref_E[r_idx][c_idx]),
            \
            f"RREF de E - Erreur à ({r_idx},{c_idx}): Attendu {expected_rref_E[r_idx]
              ][c_idx]}, Obtenu {rref_E[r_idx][c_idx]}"
print("RREF de E : OK")

# --- Test get_pivot_columns et get_rank ---
print("\n# Tests de get_pivot_columns et get_rank #")
assert get_pivot_columns(rref_A) == [0, 1]
assert get_rank(rref_A) == 2
print("Pivot columns et rang de A : OK")

assert get_pivot_columns(rref_B) == [0, 1]
assert get_rank(rref_B) == 2
print("Pivot columns et rang de B : OK")

assert get_pivot_columns(rref_C) == [0]
assert get_rank(rref_C) == 1
print("Pivot columns et rang de C : OK")

assert get_pivot_columns(rref_D) == [0, 1, 2]
assert get_rank(rref_D) == 3
print("Pivot columns et rang de D : OK")

assert get_pivot_columns(rref_E) == []
assert get_rank(rref_E) == 0
print("Pivot columns et rang de E : OK")

# --- Test get_basis_image ---
print("\n# Tests de get_basis_image #")
basis_image_A = get_basis_image(A, rref_A)
# Pivot columns for A are 0 and 1. So basis for image should be A[:,0] and A[:,1]
expected_basis_image_A = [[1.0, 5.0, 9.0], [2.0, 6.0, 10.0]]
assert len(basis_image_A) == len(expected_basis_image_A)

```

```

for i in range(len(basis_image_A)):
    for j in range(len(basis_image_A[0])):
        assert are_almost_equal(basis_image_A[i][j], expected_basis_image_A[i][j]), \
            f"Base image de A - Erreur à ({i},{j}): Attendu {expected_basis_image_A[i][j]}, Obtenu {basis_image_A[i][j]}"
print("Base de l'image de A : OK")

basis_image_B = get_basis_image(B, rref_B)
# Pivot columns for B are 0 and 1.
expected_basis_image_B = [[1.0, 2.0, 0.0], [1.0, 2.0, 1.0]]
assert len(basis_image_B) == len(expected_basis_image_B)
for i in range(len(basis_image_B)):
    for j in range(len(basis_image_B[0])):
        assert are_almost_equal(basis_image_B[i][j], expected_basis_image_B[i][j]), \
            f"Base image de B - Erreur à ({i},{j}): Attendu {expected_basis_image_B[i][j]}, Obtenu {basis_image_B[i][j]}"
print("Base de l'image de B : OK")

basis_image_C = get_basis_image(C, rref_C)
# Pivot column for C is 0.
expected_basis_image_C = [[1.0, 2.0, 3.0]]
assert len(basis_image_C) == len(expected_basis_image_C)
for i in range(len(basis_image_C)):
    for j in range(len(basis_image_C[0])):
        assert are_almost_equal(basis_image_C[i][j], expected_basis_image_C[i][j]), \
            f"Base image de C - Erreur à ({i},{j}): Attendu {expected_basis_image_C[i][j]}, Obtenu {basis_image_C[i][j]}"
print("Base de l'image de C : OK")

# --- Test get_basis_kernel et get_nullity ---
print("\n# Tests de get_basis_kernel et get_nullity #")
basis_kernel_A = get_basis_kernel(rref_A)
# rref_A:
# [1.0, 0.0, -1.0, -2.0]
# [0.0, 1.0, 2.0, 3.0]
# [0.0, 0.0, 0.0, 0.0]
# Pivot vars: x0, x1. Free vars: x2, x3.
#  $x_0 - x_2 - 2x_3 = 0 \Rightarrow x_0 = x_2 + 2x_3$ 
#  $x_1 + 2x_2 + 3x_3 = 0 \Rightarrow x_1 = -2x_2 - 3x_3$ 
# For  $x_2=1, x_3=0$ :  $x = [1, -2, 1, 0]$ 
# For  $x_2=0, x_3=1$ :  $x = [2, -3, 0, 1]$ 
expected_basis_kernel_A = [[1.0, -2.0, 1.0, 0.0], [2.0, -3.0, 0.0, 1.0]]
assert len(basis_kernel_A) == len(expected_basis_kernel_A)
# Sort to make comparison robust to order of free vars iteration
basis_kernel_A.sort(key=lambda v: [round(x, 5) for x in v])
expected_basis_kernel_A.sort(key=lambda v: [round(x, 5) for x in v])
for i in range(len(basis_kernel_A)):
    for j in range(len(basis_kernel_A[0])):
        assert are_almost_equal(basis_kernel_A[i][j], expected_basis_kernel_A[i][j]), \
            f"Base noyau de A - Erreur à ({i},{j}): Attendu {expected_basis_kernel_A[i][j]}, Obtenu {basis_kernel_A[i][j]}"
print("Base du noyau de A : OK")
assert get_nullity(rref_A) == 2
print("Nullité de A : OK")

# Vérifier que les vecteurs du noyau sont bien annulés par la matrice originale A
for vec_k in basis_kernel_A:
    result = mat_vec_mul(A, vec_k)
    assert is_zero_vector(result), f"Vecteur du noyau de A non annulé: {vec_k}, Résultat: {result}"
print("Vérification des vecteurs du noyau de A : OK")

basis_kernel_B = get_basis_kernel(rref_B)
# rref_B:

```

```

# [1.0, 0.0, 1.0]
# [0.0, 1.0, 1.0]
# [0.0, 0.0, 0.0]
# Pivot vars: x0, x1. Free var: x2.
#  $x_0 + x_2 = 0 \Rightarrow x_0 = -x_2$ 
#  $x_1 + x_2 = 0 \Rightarrow x_1 = -x_2$ 
# For  $x_2=1$ :  $x = [-1, -1, 1]$ 
expected_basis_kernel_B = [[-1.0, -1.0, 1.0]]
assert len(basis_kernel_B) == len(expected_basis_kernel_B)
for i in range(len(basis_kernel_B)):
    for j in range(len(basis_kernel_B[0])):
        assert are_almost_equal(basis_kernel_B[i][j], expected_basis_kernel_B[i][j]),
            \
            f"Base noyau de B - Erreur à ({i},{j}): Attendu {expected_basis_kernel_B[i][j]}, Obtenu {basis_kernel_B[i][j]}"
print("Base du noyau de B : OK")
assert get_nullity(rref_B) == 1
print("Nullité de B : OK")
for vec_k in basis_kernel_B:
    result = mat_vec_mul(B, vec_k)
    assert is_zero_vector(result), f"Vecteur du noyau de B non annulé: {vec_k}, Résultat: {result}"
print("Vérification des vecteurs du noyau de B : OK")

basis_kernel_C = get_basis_kernel(rref_C)
# rref_C:
# [1.0, 2.0, 1.0]
# [0.0, 0.0, 0.0]
# [0.0, 0.0, 0.0]
# Pivot var: x0. Free vars: x1, x2.
#  $x_0 + 2x_1 + x_2 = 0 \Rightarrow x_0 = -2x_1 - x_2$ 
# For  $x_1=1, x_2=0$ :  $x = [-2, 1, 0]$ 
# For  $x_1=0, x_2=1$ :  $x = [-1, 0, 1]$ 
expected_basis_kernel_C = [[-2.0, 1.0, 0.0], [-1.0, 0.0, 1.0]]
assert len(basis_kernel_C) == len(expected_basis_kernel_C)
basis_kernel_C.sort(key=lambda v: [round(x, 5) for x in v])
expected_basis_kernel_C.sort(key=lambda v: [round(x, 5) for x in v])
for i in range(len(basis_kernel_C)):
    for j in range(len(basis_kernel_C[0])):
        assert are_almost_equal(basis_kernel_C[i][j], expected_basis_kernel_C[i][j]),
            \
            f"Base noyau de C - Erreur à ({i},{j}): Attendu {expected_basis_kernel_C[i][j]}, Obtenu {basis_kernel_C[i][j]}"
print("Base du noyau de C : OK")
assert get_nullity(rref_C) == 2
print("Nullité de C : OK")
for vec_k in basis_kernel_C:
    result = mat_vec_mul(C, vec_k)
    assert is_zero_vector(result), f"Vecteur du noyau de C non annulé: {vec_k}, Résultat: {result}"
print("Vérification des vecteurs du noyau de C : OK")

basis_kernel_D = get_basis_kernel(rref_D)
assert basis_kernel_D == [] # Matrice identité, noyau trivial
print("Base du noyau de D : OK")
assert get_nullity(rref_D) == 0
print("Nullité de D : OK")

basis_kernel_E = get_basis_kernel(rref_E)
# rref_E:
# [0.0, 0.0, 0.0]
# [0.0, 0.0, 0.0]
# No pivot vars. Free vars: x0, x1, x2.

```

```

# For x0=1, x1=0, x2=0: x = [1, 0, 0]
# For x0=0, x1=1, x2=0: x = [0, 1, 0]
# For x0=0, x1=0, x2=1: x = [0, 0, 1]
expected_basis_kernel_E = [[1.0, 0.0, 0.0], [0.0, 1.0, 0.0], [0.0, 0.0, 1.0]]
assert len(basis_kernel_E) == len(expected_basis_kernel_E)
basis_kernel_E.sort(key=lambda v: [round(x, 5) for x in v])
expected_basis_kernel_E.sort(key=lambda v: [round(x, 5) for x in v])
for i in range(len(basis_kernel_E)):
    for j in range(len(basis_kernel_E[0])):
        assert are_almost_equal(basis_kernel_E[i][j], expected_basis_kernel_E[i][j]),
            \
            f"Base noyau de E - Erreur à ({i},{j}): Attendu {expected_basis_kernel_E[i][j]}, Obtenu {basis_kernel_E[i][j]}"
print("Base du noyau de E : OK")
assert get_nullity(rref_E) == 3
print("Nullité de E : OK")
for vec_k in basis_kernel_E:
    result = mat_vec_mul(E, vec_k)
    assert is_zero_vector(result), f"Vecteur du noyau de E non annulé: {vec_k}, Résultat: {result}"
print("Vérification des vecteurs du noyau de E : OK")

# --- Test du théorème du rang ---
print("\n# Tests du Théorème du Rang #")
# dim_domain = nombre de colonnes de la matrice originale
# rang + nullité = dim_domain

# Matrice A (3x4)
dim_domain_A = len(A[0])
rank_A = get_rank(rref_A)
nullity_A = get_nullity(rref_A)
assert rank_A + nullity_A == dim_domain_A, f"Théorème du rang échoué pour A: {rank_A} + {nullity_A} != {dim_domain_A}"
print(f"Théorème du rang pour A (rang={rank_A}, nullité={nullity_A}, dim_domain={dim_domain_A}): OK")

# Matrice B (3x3)
dim_domain_B = len(B[0])
rank_B = get_rank(rref_B)
nullity_B = get_nullity(rref_B)
assert rank_B + nullity_B == dim_domain_B, f"Théorème du rang échoué pour B: {rank_B} + {nullity_B} != {dim_domain_B}"
print(f"Théorème du rang pour B (rang={rank_B}, nullité={nullity_B}, dim_domain={dim_domain_B}): OK")

# Matrice C (3x3)
dim_domain_C = len(C[0])
rank_C = get_rank(rref_C)
nullity_C = get_nullity(rref_C)
assert rank_C + nullity_C == dim_domain_C, f"Théorème du rang échoué pour C: {rank_C} + {nullity_C} != {dim_domain_C}"
print(f"Théorème du rang pour C (rang={rank_C}, nullité={nullity_C}, dim_domain={dim_domain_C}): OK")

# Matrice D (3x3)
dim_domain_D = len(D[0])
rank_D = get_rank(rref_D)
nullity_D = get_nullity(rref_D)
assert rank_D + nullity_D == dim_domain_D, f"Théorème du rang échoué pour D: {rank_D} + {nullity_D} != {dim_domain_D}"
print(f"Théorème du rang pour D (rang={rank_D}, nullité={nullity_D}, dim_domain={dim_domain_D}): OK")

# Matrice E (2x3)
dim_domain_E = len(E[0])

```



```
rank_E = get_rank(rref_E)
nullity_E = get_nullity(rref_E)
assert rank_E + nullity_E == dim_domain_E, f"Théorème du rang échoué pour E: {rank_E}
    + {nullity_E} != {dim_domain_E}"
print(f"Théorème du rang pour E (rang={rank_E}, nullité={nullity_E}, dim_domaine={
    dim_domain_E}): OK")

print("\n--- Tous les tests sont passés avec succès ! ---")
```

\{\chapter*{TP 3 : Transformations Linéaires, Noyau et Image "From Scratch" en Python

\{\section*{Objectifs du TP

Ce troisième TP vise à implémenter des concepts fondamentaux de l'algèbre linéaire, essentiels à la compréhension et à la conception de nombreux algorithmes d'Intelligence Artificielle. L'implémentation sera réalisée \{\textbf{entièrement "from scratch" en Python pur, sans l'aide de bibliothèques de calcul scientifique de haut niveau (comme NumPy, SciPy, etc.) pour les opérations d'algèbre linéaire.

\{\subsection*{Objectifs Mathématiques * \{\textbf{Comprendre la représentation numérique des vecteurs et matrices. * \{\textbf{Implémenter les opérations de base sur les vecteurs et matrices (addition, multiplication scalaire, produit scalaire, multiplication matrice-vecteur, multiplication matrice-matrice). * \{\textbf{Maîtriser l'algorithme d'élimination de Gauss-Jordan pour réduire une matrice en forme échelonnée réduite (RREF). * \{\textbf{Calculer le rang d'une matrice. * \{\textbf{Déterminer une base du noyau (espace nul) d'une application linéaire. * \{\textbf{Déterminer une base de l'image (espace colonne) d'une application linéaire. * \{\textbf{Vérifier le Théorème du Rang ($\dim(\text{Ker}(A)) + \dim(\text{Im}(A)) = \dim(\text{domaine})$).

\{\subsection*{Objectifs IA * \{\textbf{Fondations des Réseaux de Neurones : Comprendre comment les multiplications matrice-vecteur et matrice-matrice sont au cœur des couches denses (fully connected layers) sans fonction d'activation. * \{\textbf{Réduction de Dimensionnalité et Compression : Le noyau et l'image d'une matrice linéaire sont des outils clés pour comprendre les transformations de l'espace des caractéristiques, la perte ou la conservation d'information, et les principes derrière des techniques comme l'Analyse en Composantes Principales (ACP/PCA). * \{\textbf{Interprétabilité et Robustesse des Modèles : L'analyse du noyau et de l'image permet d'identifier quelles combinaisons d'entrées produisent des sorties nulles ou quelles sont les sorties possibles d'un modèle, ce qui est crucial pour l'interprétabilité et la robustesse des systèmes d'IA. * \{\textbf{Algorithmes d'Optimisation : La résolution de systèmes linéaires (via l'élimination de Gauss) est une brique de base pour de nombreux problèmes d'optimisation en ML.

\{\section*{Implémentation Python "From Scratch"

Nous allons représenter les vecteurs comme des listes de nombres (`list[float]`) et les matrices comme des listes de listes de nombres (`list[list[float]]`).

```
import math

# Constante pour gérer la précision des calculs flottants
EPSILON = 1e-9

def is_zero(val):
    """Vérifie si une valeur est considérée comme nulle."""
    return abs(val) < EPSILON

def is_zero_vector(v):
    """Vérifie si tous les éléments d'un vecteur sont considérés comme nuls."""
    return all(is_zero(x) for x in v)

def are_vectors_equal(v1, v2):
    """Vérifie si deux vecteurs sont égaux avec une tolérance EPSILON."""
    if len(v1) != len(v2):
        return False
    return all(is_zero(v1[i] - v2[i]) for i in range(len(v1)))

def clone_matrix(matrix):
    """Crée une copie profonde d'une matrice."""
    return [row[:] for row in matrix]

def print_matrix(matrix, name="Matrix"):
    """Affiche une matrice de manière formatée."""
    print(f"{name}:")
    for row in matrix:
        print([f"{x:.4f}" for x in row])
    print("-" * 30)

# --- 1. Opérations de base sur les vecteurs et matrices ---

def vector_add(v1, v2):
    """Ajoute deux vecteurs."""
    assert len(v1) == len(v2), "Les vecteurs doivent avoir la même dimension."
    return [v1[i] + v2[i] for i in range(len(v1))]
```

```

def scalar_vector_mul(s, v):
    """Multiplie un vecteur par un scalaire."""
    return [s * x for x in v]

def dot_product(v1, v2):
    """Calcule le produit scalaire de deux vecteurs."""
    assert len(v1) == len(v2), "Les vecteurs doivent avoir la même dimension pour le produit scalaire."
    return sum(v1[i] * v2[i] for i in range(len(v1)))

def matrix_vector_mul(matrix, vector):
    """Multiplie une matrice par un vecteur."""
    num_rows = len(matrix)
    num_cols = len(matrix[0]) if num_rows > 0 else 0
    assert num_cols == len(vector), "Le nombre de colonnes de la matrice doit correspondre à la dimension du vecteur."

    result = [0.0] * num_rows
    for i in range(num_rows):
        result[i] = dot_product(matrix[i], vector)
    return result

def matrix_mul(m1, m2):
    """Multiplie deux matrices."""
    rows1, cols1 = len(m1), len(m1[0]) if len(m1) > 0 else 0
    rows2, cols2 = len(m2), len(m2[0]) if len(m2) > 0 else 0

    assert cols1 == rows2, "Le nombre de colonnes de la première matrice doit correspondre au nombre de lignes de la seconde."

    result = [[0.0 for _ in range(cols2)] for _ in range(rows1)]
    for i in range(rows1):
        for j in range(cols2):
            for k in range(cols1):
                result[i][j] += m1[i][k] * m2[k][j]
    return result

def transpose(matrix):
    """Transposer une matrice."""
    rows, cols = len(matrix), len(matrix[0]) if len(matrix) > 0 else 0
    if rows == 0 or cols == 0:
        return []
    transposed = [[0.0 for _ in range(rows)] for _ in range(cols)]
    for i in range(rows):
        for j in range(cols):
            transposed[j][i] = matrix[i][j]
    return transposed

# --- 2. Élimination de Gauss-Jordan pour la Forme Échelonnée Réduite (RREF) ---

def gaussian_elimination(matrix):
    """
    Transforme une matrice en sa Forme Échelonnée Réduite (RREF)
    en utilisant l'élimination de Gauss-Jordan.
    Retourne la matrice en RREF et une liste des indices des colonnes pivots.
    """
    mat = clone_matrix(matrix)
    rows = len(mat)
    cols = len(mat[0]) if rows > 0 else 0

    if rows == 0 or cols == 0:
        return [], []

    pivot_cols_indices = []
    lead = 0 # Indice de la colonne pivot actuelle

```

```

for r in range(rows):
    if lead >= cols:
        break

    i = r
    # Trouver une ligne non nulle pour le pivot dans la colonne 'lead'
    while i < rows and is_zero(mat[i][lead]):
        i += 1

    if i < rows: # Si un pivot est trouvé
        # Échanger la ligne courante (r) avec la ligne du pivot (i)
        mat[r], mat[i] = mat[i], mat[r]

        # Normaliser la ligne pivot pour que l'élément pivot soit 1
        lv = mat[r][lead]
        for j in range(cols):
            mat[r][j] /= lv

        # Éliminer les autres éléments de la colonne 'lead' dans les autres
        # lignes
        for i_other_row in range(rows):
            if i_other_row != r:
                lv_other_row = mat[i_other_row][lead]
                for j in range(cols):
                    mat[i_other_row][j] -= lv_other_row * mat[r][j]

        pivot_cols_indices.append(lead)
        lead += 1 # Passer à la colonne pivot suivante
    else: # Si aucun pivot n'est trouvé dans cette colonne, passer à la colonne
    suivante
        lead += 1
        # Redo the current row, as it was skipped.
        # This is important for sparse matrices where 'lead' might advance
        # several times without finding a pivot.
        # Example: [[0,1],[0,0]] -> lead should jump to col 1.
        # The current loop structure 'for r in range(rows)' naturally tries to
        # find a pivot for row 'r'.
        # If no pivot is found in 'lead' for row 'r', we advance 'lead' and will
        # try to find a pivot
        # for row 'r' in the *next* 'lead' column. So 'r' shouldn't increment.
        # This is done by decrementing 'r' here.
        r -= 1 # Re-evaluate the current row 'r' with the new 'lead'

    # Arrondir les valeurs très proches de zéro à zéro absolu
    for r in range(rows):
        for c in range(cols):
            if is_zero(mat[r][c]):
                mat[r][c] = 0.0

    return mat, pivot_cols_indices

# --- 3. Fonctions d'algèbre linéaire avancées ---

def get_rank(matrix):
    """
    Calcule le rang d'une matrice (la dimension de son espace colonne).
    Le rang est le nombre de pivots dans la forme échelonnée réduite.
    """
    if not matrix or not matrix[0]:
        return 0
    _, pivot_cols_indices = gaussian_elimination(matrix)
    return len(pivot_cols_indices)

def get_kernel_basis(matrix):
    """

```

```

    Trouve une base pour le noyau (espace nul) d'une matrice.
    """
    m, n = len(matrix), len(matrix[0]) if len(matrix) > 0 else 0
    if n == 0: return []
    if m == 0: return [[1.0 if i == k else 0.0 for i in range(n)] for k in range(n)]
    # Trivial case for 0-row matrix, kernel is  $\mathbb{R}^n$ 

    rref_mat, pivot_cols_indices = gaussian_elimination(matrix)

    pivot_vars_set = set(pivot_cols_indices)
    free_vars = sorted(list(set(range(n)) - pivot_vars_set))

    kernel_basis = []

    # Si toutes les colonnes sont des colonnes pivots, le noyau est trivial {0}
    if not free_vars:
        return []

    # Pour chaque variable libre, construire un vecteur de base
    for free_idx in free_vars:
        sol_vec = [0.0] * n
        sol_vec[free_idx] = 1.0 # Définir cette variable libre à 1

        # Utiliser la RREF pour exprimer les variables pivots en fonction des
        # variables libres
        # On itère sur les lignes qui contiennent un pivot
        for r_pivot_row in range(len(pivot_cols_indices)):
            pivot_col = pivot_cols_indices[r_pivot_row]

            # La relation dans la RREF est:  $x_{\text{pivot\_col}} + \sum(c_j * x_j) = 0$ 
            # où  $x_j$  sont d'autres variables, et  $c_j$  sont les coefficients de
            # rref_mat.
            # Puisque nous avons fixé les variables libres (sol_vec[k] est 0 ou 1)
            # et que les autres variables pivots sont 0 dans la ligne du pivot,
            # cela simplifie à:  $x_{\text{pivot\_col}} = -(\sum(\text{rref\_mat}[\text{r\_pivot\_row}][k] * \text{sol\_vec}[k])$ 
            # pour  $k \neq \text{pivot\_col}$ )

            val_for_pivot = 0.0
            for k in range(n):
                if k != pivot_col:
                    val_for_pivot += rref_mat[r_pivot_row][k] * sol_vec[k]
            sol_vec[pivot_col] = -val_for_pivot

        # Nettoyer les petites valeurs flottantes
        for i in range(n):
            if is_zero(sol_vec[i]):
                sol_vec[i] = 0.0

        kernel_basis.append(sol_vec)

    return kernel_basis

def get_image_basis(original_matrix):
    """
    Trouve une base pour l'image (espace colonne) d'une matrice.
    La base est constituée des colonnes originales de la matrice
    qui correspondent aux colonnes pivots de sa forme échelonnée réduite.
    """
    m, n = len(original_matrix), len(original_matrix[0]) if len(original_matrix) > 0
    else 0
    if m == 0 or n == 0:
        return []

    _, pivot_cols_indices = gaussian_elimination(original_matrix)

    image_basis = []

```

```

# Collecter les colonnes originales correspondant aux indices des colonnes pivots
for col_idx in pivot_cols_indices:
    column_vector = [original_matrix[row][col_idx] for row in range(m)]
    image_basis.append(column_vector)

return image_basis

# --- 4. Vérification du Théorème du Rang ---

def verify_rank_nullity_theorem(matrix):
    """
    Vérifie le Théorème du Rang pour une matrice donnée :
     $\dim(\text{Ker}(A)) + \dim(\text{Im}(A)) = \dim(\text{domaine})$ 
    où  $\dim(\text{domaine})$  est le nombre de colonnes de A.
    Retourne True si le théorème est vérifié, False sinon.
    """
    m, n = len(matrix), len(matrix[0]) if len(matrix) > 0 else 0
    if n == 0: # If no columns, domain dimension is 0. Rank=0, Nullity=0. 0+0=0.
        return True

    rank_A = get_rank(matrix)
    kernel_basis = get_kernel_basis(matrix)
    nullity_A = len(kernel_basis) # Dimension du noyau

    # Le théorème est  $\text{rank} + \text{nullity} = \text{nombre de colonnes (dimension du domaine)}$ 
    return abs((rank_A + nullity_A) - n) < EPSILON

# --- 5. Cas de test et Assertions de validation ---

print("--- Début des tests ---")

# --- Test 1: Matrice carrée de plein rang ---
M1 = [
    [1.0, 2.0],
    [3.0, 4.0]
]
print_matrix(M1, "M1")
assert get_rank(M1) == 2, "Test 1: Le rang de M1 devrait être 2."
kernel_M1 = get_kernel_basis(M1)
assert len(kernel_M1) == 0, "Test 1: Le noyau de M1 devrait être trivial (dimension 0)."
image_M1 = get_image_basis(M1)
assert len(image_M1) == 2, "Test 1: L'image de M1 devrait être de dimension 2."
assert are_vectors_equal(image_M1[0], [1.0, 3.0])
assert are_vectors_equal(image_M1[1], [2.0, 4.0])
assert verify_rank_nullity_theorem(M1), "Test 1: Le Théorème du Rang devrait être vérifié pour M1."

# --- Test 2: Matrice rectangulaire avec dépendance linéaire ---
M2 = [
    [1.0, 2.0, 3.0],
    [2.0, 4.0, 6.0]
]
print_matrix(M2, "M2")
assert get_rank(M2) == 1, "Test 2: Le rang de M2 devrait être 1."
kernel_M2 = get_kernel_basis(M2)
assert len(kernel_M2) == 2, "Test 2: Le noyau de M2 devrait être de dimension 2."
# Vérifier que les vecteurs du noyau sont corrects (Ax=0)
assert is_zero_vector(matrix_vector_mul(M2, kernel_M2[0])), "Test 2: Le premier vecteur du noyau ne produit pas le vecteur nul."
assert is_zero_vector(matrix_vector_mul(M2, kernel_M2[1])), "Test 2: Le second vecteur du noyau ne produit pas le vecteur nul."
# Exemple de base attendue pour M2:  $\begin{bmatrix} -2 \\ 1 \\ 0 \end{bmatrix}$ ,  $\begin{bmatrix} -3 \\ 0 \\ 1 \end{bmatrix}$  ou des multiples
assert are_vectors_equal(kernel_M2[0], [-2.0, 1.0, 0.0]) or are_vectors_equal(
    kernel_M2[0], [-3.0, 0.0, 1.0])
assert are_vectors_equal(kernel_M2[1], [-3.0, 0.0, 1.0]) or are_vectors_equal(

```

```

    kernel_M2[1], [-2.0, 1.0, 0.0])

image_M2 = get_image_basis(M2)
assert len(image_M2) == 1, "Test 2: L'image de M2 devrait être de dimension 1."
assert are_vectors_equal(image_M2[0], [1.0, 2.0]), "Test 2: La base de l'image de M2
    est incorrecte."
assert verify_rank_nullity_theorem(M2), "Test 2: Le Théorème du Rang devrait être vé
    rifié pour M2."

# --- Test 3: Matrice identité ---
M3 = [
    [1.0, 0.0, 0.0],
    [0.0, 1.0, 0.0],
    [0.0, 0.0, 1.0]
]
print_matrix(M3, "M3")
assert get_rank(M3) == 3, "Test 3: Le rang de M3 devrait être 3."
kernel_M3 = get_kernel_basis(M3)
assert len(kernel_M3) == 0, "Test 3: Le noyau de M3 devrait être trivial."
image_M3 = get_image_basis(M3)
assert len(image_M3) == 3, "Test 3: L'image de M3 devrait être de dimension 3."
assert verify_rank_nullity_theorem(M3), "Test 3: Le Théorème du Rang devrait être vé
    rifié pour M3."

# --- Test 4: Matrice de zéros ---
M4 = [
    [0.0, 0.0, 0.0],
    [0.0, 0.0, 0.0]
]
print_matrix(M4, "M4")
assert get_rank(M4) == 0, "Test 4: Le rang de M4 devrait être 0."
kernel_M4 = get_kernel_basis(M4)
assert len(kernel_M4) == 3, "Test 4: Le noyau de M4 devrait être de dimension 3 ( $\mathbb{R}^3$ )
    ."
assert is_zero_vector(matrix_vector_mul(M4, kernel_M4[0]))
assert is_zero_vector(matrix_vector_mul(M4, kernel_M4[1]))
assert is_zero_vector(matrix_vector_mul(M4, kernel_M4[2]))
image_M4 = get_image_basis(M4)
assert len(image_M4) == 0, "Test 4: L'image de M4 devrait être de dimension 0 (espace
    nul)."
assert verify_rank_nullity_theorem(M4), "Test 4: Le Théorème du Rang devrait être vé
    rifié pour M4."

# --- Test 5: Matrice colonne ---
M5 = [
    [1.0],
    [2.0],
    [3.0]
]
print_matrix(M5, "M5")
assert get_rank(M5) == 1, "Test 5: Le rang de M5 devrait être 1."
kernel_M5 = get_kernel_basis(M5)
assert len(kernel_M5) == 0, "Test 5: Le noyau de M5 devrait être trivial."
image_M5 = get_image_basis(M5)
assert len(image_M5) == 1, "Test 5: L'image de M5 devrait être de dimension 1."
assert are_vectors_equal(image_M5[0], [1.0, 2.0, 3.0])
assert verify_rank_nullity_theorem(M5), "Test 5: Le Théorème du Rang devrait être vé
    rifié pour M5."

# --- Test 6: Matrice ligne ---
M6 = [[1.0, 2.0, 3.0]]
print_matrix(M6, "M6")
assert get_rank(M6) == 1, "Test 6: Le rang de M6 devrait être 1."
kernel_M6 = get_kernel_basis(M6)
assert len(kernel_M6) == 2, "Test 6: Le noyau de M6 devrait être de dimension 2."
assert is_zero_vector(matrix_vector_mul(M6, kernel_M6[0]))

```

```

assert is_zero_vector(matrix_vector_mul(M6, kernel_M6[1]))
image_M6 = get_image_basis(M6)
assert len(image_M6) == 1, "Test 6: L'image de M6 devrait être de dimension 1."
assert are_vectors_equal(image_M6[0], [1.0]) # La base de l'image est la première
    colonne originale qui est pivot
assert verify_rank_nullity_theorem(M6), "Test 6: Le Théorème du Rang devrait être vé
    rifié pour M6."

# --- Test 7: Matrice non carrée avec plus de lignes que de colonnes ---
M7 = [
    [1.0, 0.0],
    [0.0, 1.0],
    [0.0, 0.0]
]
print_matrix(M7, "M7")
assert get_rank(M7) == 2, "Test 7: Le rang de M7 devrait être 2."
kernel_M7 = get_kernel_basis(M7)
assert len(kernel_M7) == 0, "Test 7: Le noyau de M7 devrait être de dimension 0."
image_M7 = get_image_basis(M7)
assert len(image_M7) == 2, "Test 7: L'image de M7 devrait être de dimension 2."
assert are_vectors_equal(image_M7[0], [1.0, 0.0, 0.0])
assert are_vectors_equal(image_M7[1], [0.0, 1.0, 0.0])
assert verify_rank_nullity_theorem(M7), "Test 7: Le Théorème du Rang devrait être vé
    rifié pour M7."

# --- Test 8: Matrice avec des fractions et rang non trivial ---
M8 = [
    [2.0, 1.0, 3.0, 0.0],
    [4.0, 2.0, 6.0, 0.0],
    [1.0, 0.5, 1.5, 0.0]
]
print_matrix(M8, "M8")
assert get_rank(M8) == 1, "Test 8: Le rang de M8 devrait être 1."
kernel_M8 = get_kernel_basis(M8)
assert len(kernel_M8) == 3, "Test 8: Le noyau de M8 devrait être de dimension 3."
for vec in kernel_M8:
    assert is_zero_vector(matrix_vector_mul(M8, vec)), f"Test 8: Le vecteur {vec} du
        noyau ne produit pas le vecteur nul."
assert verify_rank_nullity_theorem(M8), "Test 8: Le Théorème du Rang devrait être vé
    rifié pour M8."

# --- Test 9: Matrice vide ou 0xN/Nx0 ---
M9_empty_rows = []
assert get_rank(M9_empty_rows) == 0, "Test 9: Le rang d'une matrice vide (0 lignes)
    doit être 0."
assert len(get_kernel_basis(M9_empty_rows)) == 0 # depends on interpretation of N
assert len(get_image_basis(M9_empty_rows)) == 0

M9_empty_cols = [[], []]
assert get_rank(M9_empty_cols) == 0, "Test 9: Le rang d'une matrice vide (0 colonnes)
    doit être 0."
assert len(get_kernel_basis(M9_empty_cols)) == 0 # Kernel is always empty if no
    columns
assert len(get_image_basis(M9_empty_cols)) == 0

print("\n--- Tous les tests ont été passés avec succès ! ---")

```


\{chapter*{TP 4 : Applications Linéaires, Noyau et Image pour l'Analyse de Caractéristiques

\{section*{Objectif Mathématique et IA du TP

Ce TP a pour objectif de vous familiariser avec les concepts fondamentaux d'algèbre linéaire que sont les applications linéaires, leur noyau (kernel) et leur image (image, ou espace colonne). Ces notions sont au cœur de nombreuses techniques d'apprentissage automatique, notamment pour la transformation de caractéristiques, la réduction de dimensionnalité, l'analyse des espaces de données et la compréhension des modèles.

\{textbf{Objectifs Mathématiques : 1. \{textbf{Implémenter des opérations matricielles de base : Multiplication matrice-vecteur et manipulation de matrices. 2. \{textbf{Comprendre et implémenter la forme échelonnée réduite (RREF) : Une étape cruciale pour l'analyse des matrices. 3. \{textbf{Calculer l'image d'un vecteur par une application linéaire : Représenté par la multiplication d'une matrice par ce vecteur. 4. \{textbf{Déterminer une base du noyau d'une application linéaire : L'ensemble des vecteurs qui sont projetés sur le vecteur nul. 5. \{textbf{Déterminer une base de l'image d'une application linéaire : L'ensemble de tous les vecteurs qui peuvent être atteints par la transformation. 6. \{textbf{Vérifier le théorème du rang : Comprendre la relation entre la dimension du noyau (nullité) et la dimension de l'image (rang) d'une matrice.

\{textbf{Objectifs pour l'IA : 1. \{textbf{Transformation de Caractéristiques (Feature Engineering) : Les applications linéaires sont utilisées pour projeter des données d'un espace de caractéristiques vers un autre, par exemple pour créer de nouvelles caractéristiques ou pour adapter les données à un modèle. 2. \{textbf{Filtrage et Invariance (Noyau) : Le noyau peut être interprété comme l'ensemble des "modifications" ou des "variations" dans les données qui sont ignorées ou "filtrées" par la transformation. Par exemple, si une application linéaire représente une compression d'image, son noyau contiendrait les informations d'image qui sont perdues (non essentielles) lors de la compression. Pour l'apprentissage profond, le noyau des couches linéaires peut révéler des invariances apprises. 3. \{textbf{Réduction de Dimensionnalité et Représentation (Image) : L'image d'une application linéaire définit le sous-espace dans lequel les données transformées résident. C'est la base de techniques comme la PCA (Analyse en Composantes Principales), où l'on cherche une projection linéaire des données sur un sous-espace de dimension inférieure qui capture le maximum de variance. Le rang de la matrice donne la dimension de l'espace de caractéristiques transformées atteignables. 4. \{textbf{Comprendre les Limites des Modèles Linéaires : L'étude du noyau et de l'image aide à comprendre ce qu'un modèle linéaire peut et ne peut pas représenter ou distinguer.

\{section*{Implémentation en Python Pur

Toutes les opérations d'algèbre linéaire doivent être implémentées "from scratch", sans l'aide de bibliothèques comme `numpy`. Nous utiliserons des listes de listes pour représenter les matrices et des listes pour les vecteurs.

```
import math

# Constante pour gérer les erreurs de virgule flottante
EPSILON = 1e-9

def are_approximately_equal(a, b, epsilon=EPSILON):
    """Vérifie si deux nombres flottants sont approximativement égaux."""
    return abs(a - b) < epsilon

def is_zero(val, epsilon=EPSILON):
    """Vérifie si un nombre est approximativement zéro."""
    return abs(val) < epsilon

def create_matrix(rows, cols, value=0.0):
    """Crée une matrice de taille rows x cols initialisée avec 'value'."""
    return [[value for _ in range(cols)] for _ in range(rows)]

def get_matrix_dimensions(matrix):
    """Retourne (rows, cols) d'une matrice."""
    rows = len(matrix)
    if rows == 0:
        return 0, 0
    cols = len(matrix[0])
    return rows, cols

def matrix_scalar_multiply(matrix, scalar):
    """Multiplie une matrice par un scalaire."""
    rows, cols = get_matrix_dimensions(matrix)
    result = create_matrix(rows, cols)
    for r in range(rows):
        for c in range(cols):
```

```

        result[r][c] = matrix[r][c] * scalar
    return result

def matrix_vector_multiply(matrix, vector):
    """
    Multiplie une matrice par un vecteur.
    matrix: M x N
    vector: N x 1
    Retourne: M x 1
    """
    rows, cols = get_matrix_dimensions(matrix)
    if cols != len(vector):
        raise ValueError("Le nombre de colonnes de la matrice doit être égal à la
                           longueur du vecteur.")

    result = [0.0] * rows
    for r in range(rows):
        current_sum = 0.0
        for c in range(cols):
            current_sum += matrix[r][c] * vector[c]
        result[r] = current_sum
    return result

def transpose_matrix(matrix):
    """Calcule la transposée d'une matrice."""
    rows, cols = get_matrix_dimensions(matrix)
    result = create_matrix(cols, rows)
    for r in range(rows):
        for c in range(cols):
            result[c][r] = matrix[r][c]
    return result

def _swap_rows(matrix, r1, r2):
    """Échange deux lignes d'une matrice."""
    matrix[r1], matrix[r2] = matrix[r2], matrix[r1]

def _scale_row(matrix, r, scalar):
    """Multiplie une ligne d'une matrice par un scalaire."""
    for c in range(len(matrix[r])):
        matrix[r][c] *= scalar

def _add_multiple_of_row(matrix, target_r, source_r, scalar):
    """Ajoute un multiple d'une ligne à une autre ligne."""
    for c in range(len(matrix[target_r])):
        matrix[target_r][c] += matrix[source_r][c] * scalar

def rref(matrix):
    """
    Convertit une matrice en sa forme échelonnée réduite (RREF).
    Implémente l'algorithme d'élimination de Gauss-Jordan.
    """
    mat = [row[:] for row in matrix] # Crée une copie pour ne pas modifier l'
                                     # originale
    rows, cols = get_matrix_dimensions(mat)
    if rows == 0 or cols == 0:
        return mat

    lead = 0 # Colonne pivot actuelle
    for r in range(rows):
        if lead >= cols:
            break

        # Trouver la ligne avec le plus grand élément absolu dans la colonne 'lead'
        # pour la stabilité numérique.
        i = r
        while i < rows and is_zero(mat[i][lead]):

```

```

        i += 1

    if i == rows: # Pas de pivot dans cette colonne, passer à la suivante
        lead += 1
        continue

    _swap_rows(mat, r, i)

    # Mettre l'élément pivot à 1
    lv = mat[r][lead]
    _scale_row(mat, r, 1.0 / lv)

    # Éliminer les autres éléments de la colonne 'lead'
    for i_row in range(rows):
        if i_row != r:
            lv = mat[i_row][lead]
            _add_multiple_of_row(mat, i_row, r, -lv)

    lead += 1

return mat

def get_pivot_columns(rref_matrix):
    """
    Identifie les indices des colonnes pivots dans une matrice RREF.
    Une colonne est pivot si elle contient un 1 qui est le premier élément non nul de
    sa ligne.
    """
    rows, cols = get_matrix_dimensions(rref_matrix)
    pivot_cols = []
    for r in range(rows):
        for c in range(cols):
            if is_zero(rref_matrix[r][c]):
                continue
            if are_approximately_equal(rref_matrix[r][c], 1.0):
                # Vérifier si c'est le premier non-zéro de la ligne
                is_pivot = True
                for prev_c in range(c):
                    if not is_zero(rref_matrix[r][prev_c]):
                        is_pivot = False
                        break
                if is_pivot:
                    pivot_cols.append(c)
                    break # Passer à la ligne suivante
    return pivot_cols

def find_image_basis(matrix):
    """
    Trouve une base pour l'image (espace colonne) de la matrice.
    La base est formée par les colonnes de la MATRICE ORIGINALE
    correspondant aux colonnes pivots de sa RREF.
    """
    rows, cols = get_matrix_dimensions(matrix)
    if rows == 0 or cols == 0:
        return []

    rref_mat = rref(matrix)
    pivot_col_indices = get_pivot_columns(rref_mat)

    image_basis = []
    for col_idx in pivot_col_indices:
        column_vector = [matrix[r][col_idx] for r in range(rows)]
        image_basis.append(column_vector)

    return image_basis

```

```

def find_kernel_basis(matrix):
    """
    Trouve une base pour le noyau (null space) de la matrice.
    Ceci est fait en résolvant  $Ax = 0$  en utilisant la RREF de A.
    """
    rows, cols = get_matrix_dimensions(matrix)
    if rows == 0 or cols == 0:
        return []

    rref_mat = rref(matrix)
    pivot_col_indices = get_pivot_columns(rref_mat)

    free_var_indices = [c for c in range(cols) if c not in pivot_col_indices]
    kernel_basis = []

    for free_idx in free_var_indices:
        # Créer un vecteur solution où la variable libre courante est 1
        # et les autres variables libres sont 0.
        solution_vec = [0.0] * cols
        solution_vec[free_idx] = 1.0

        # Résoudre pour les variables pivots en utilisant les équations de la RREF
        for r_idx, p_idx in enumerate(pivot_col_indices):
            # p_idx est l'index de la colonne pivot dans la matrice originale
            # r_idx est l'index de la ligne correspondante dans la RREF (si la
            # matrice n'est pas nulle)
            # Les lignes non nulles de la RREF correspondent aux lignes de la matrice
            # originale
            # C'est-à-dire, si rref_mat[r_idx][p_idx] est un pivot 1, alors cette
            # ligne
            # définit x_p_idx en fonction des variables libres.
            # L'équation est x_p_idx + sum(coeff * x_free) = 0
            # donc x_p_idx = - sum(coeff * x_free)

            # Il faut s'assurer que r_idx reste dans les limites des lignes de
            # rref_mat
            if r_idx < rows:
                val = 0.0
                for f_idx in free_var_indices:
                    val += rref_mat[r_idx][f_idx] * solution_vec[f_idx]
                solution_vec[p_idx] = -val
            kernel_basis.append(solution_vec)

    return kernel_basis

def rank(matrix):
    """Calcule le rang d'une matrice (dimension de l'image)."""
    image_basis = find_image_basis(matrix)
    return len(image_basis)

def nullity(matrix):
    """Calcule la nullité d'une matrice (dimension du noyau)."""
    kernel_basis = find_kernel_basis(matrix)
    return len(kernel_basis)

def verify_rank_nullity_theorem(matrix):
    """
    Vérifie le théorème du rang:  $\text{rank}(A) + \text{nullity}(A) = \text{nombre de colonnes de } A$ .
    """
    rows, cols = get_matrix_dimensions(matrix)
    if cols == 0: # Cas trivial pour les matrices sans colonnes
        return True

    matrix_rank = rank(matrix)
    matrix_nullity = nullity(matrix)

```

```

    return matrix_rank + matrix_nullity == cols

def print_matrix(matrix, name="Matrix"):
    """Affiche une matrice."""
    print(f"{name}:")
    for row in matrix:
        print([f"{x:.4f}" for x in row])
    print()

def print_vector(vector, name="Vector"):
    """Affiche un vecteur."""
    print(f"{name}:")
    print([f"{x:.4f}" for x in vector])
    print()

# --- Tests de validation mathématique ---

print("--- Démarrage des tests de validation ---")

# Matrice simple 2x2
A = [[1.0, 2.0],
      [3.0, 4.0]]
v = [1.0, 0.0]

# Test 1: Multiplication Matrice-Vecteur
Av = matrix_vector_multiply(A, v)
assert are_approximately_equal(Av[0], 1.0) and are_approximately_equal(Av[1], 3.0), \
    f"Test 1 failed: matrix_vector_multiply. Expected [1.0, 3.0], got {Av}"
print("Test 1 (Matrice-Vecteur) passé.")

# Test 2: RREF - Matrice Identité
I = [[1.0, 0.0], [0.0, 1.0]]
rref_I = rref(I)
assert all(are_approximately_equal(rref_I[r][c], I[r][c]) for r in range(2) for c in
    range(2)), \
    f"Test 2 failed: RREF Identity. Expected {I}, got {rref_I}"
print("Test 2 (RREF Identité) passé.")

# Test 3: RREF - Matrice avec élimination
M1 = [[1.0, 2.0, -1.0],
      [2.0, 4.0, -2.0],
      [-1.0, -2.0, 1.0]]
rref_M1_expected = [[1.0, 2.0, -1.0],
                    [0.0, 0.0, 0.0],
                    [0.0, 0.0, 0.0]]
rref_M1 = rref(M1)
assert all(are_approximately_equal(rref_M1[r][c], rref_M1_expected[r][c]) for r in
    range(3) for c in range(3)), \
    f"Test 3 failed: RREF M1. Expected {rref_M1_expected}, got {rref_M1}"
print("Test 3 (RREF avec élimination) passé.")

# Test 4: RREF - Matrice plus complexe
M2 = [[1.0, 2.0, 3.0, 4.0],
      [4.0, 5.0, 6.0, 7.0],
      [7.0, 8.0, 9.0, 1.0]]
rref_M2_expected = [[1.0, 0.0, -1.0, 0.0],
                    [0.0, 1.0, 2.0, 0.0],
                    [0.0, 0.0, 0.0, 1.0]] # Calculé manuellement pour vérifier
rref_M2 = rref(M2)
# Comparaison avec une tolérance pour les flottants
for r in range(3):
    for c in range(4):
        assert are_approximately_equal(rref_M2[r][c], rref_M2_expected[r][c]), \
            f"Test 4 failed at ({r},{c}): RREF M2. Expected {rref_M2_expected[r][c]}, \
                got {rref_M2[r][c]}"
print("Test 4 (RREF plus complexe) passé.")

```

```

# Test 5: Noyau - Matrice nulle
Z = [[0.0, 0.0], [0.0, 0.0]]
kernel_Z = find_kernel_basis(Z)
# Pour une matrice 0, le noyau est l'espace entier ( $\mathbb{R}^2$ ), base canonique  $\begin{bmatrix} 1, 0 \\ 0, 1 \end{bmatrix}$ 
assert len(kernel_Z) == 2 and \
    all(are_approximately_equal(kernel_Z[0][c], [1.0, 0.0][c]) for c in range(2))
    and \
    all(are_approximately_equal(kernel_Z[1][c], [0.0, 1.0][c]) for c in range(2)),
    f"Test 5 failed: Kernel of zero matrix. Expected  $\begin{bmatrix} 1, 0 \\ 0, 1 \end{bmatrix}$ , got {
        kernel_Z}"
print("Test 5 (Noyau de matrice nulle) passé.")

# Test 6: Noyau - Matrice avec un noyau non trivial (M1)
# M1 RREF est  $\begin{bmatrix} 1, 2, -1 \\ 0, 0, 0 \\ 0, 0, 0 \end{bmatrix}$ 
# Pivot en colonne 0 (x), libres en colonnes 1 (y), 2 (z)
#  $x + 2y - z = 0 \Rightarrow x = -2y + z$ 
# Base du noyau:
#  $y=1, z=0 \Rightarrow x=-2$ . Vector:  $[-2, 1, 0]$ 
#  $y=0, z=1 \Rightarrow x=1$ . Vector:  $[1, 0, 1]$ 
kernel_M1 = find_kernel_basis(M1)
assert len(kernel_M1) == 2, f"Test 6 failed: Kernel M1 size. Expected 2, got {len(
    kernel_M1)}"
# Vérifier que les vecteurs du noyau donnent 0 quand multipliés par M1
for kv in kernel_M1:
    res = matrix_vector_multiply(M1, kv)
    assert all(is_zero(x) for x in res), f"Test 6 failed: Kernel M1 vector {kv} not
        in kernel. Result: {res}"
print("Test 6 (Noyau de matrice M1) passé.")

# Test 7: Image - Matrice M1
# M1 RREF est  $\begin{bmatrix} 1, 2, -1 \\ 0, 0, 0 \\ 0, 0, 0 \end{bmatrix}$ 
# Colonne pivot est la 0ème. Donc la 0ème colonne de M1 est la base de l'image.
# Base attendue:  $\begin{bmatrix} 1, 2, -1 \end{bmatrix}$ 
image_M1 = find_image_basis(M1)
assert len(image_M1) == 1 and \
    all(are_approximately_equal(image_M1[0][r], M1[r][0]) for r in range(3)), \
    f"Test 7 failed: Image M1. Expected  $\begin{bmatrix} 1, 2, -1 \end{bmatrix}$ , got {image_M1}"
print("Test 7 (Image de matrice M1) passé.")

# Test 8: Image - Matrice pleine de rang
# M2 RREF:  $\begin{bmatrix} 1, 0, 0, -1, 0 \\ 0, 1, 0, 2, 0 \\ 0, 0, 0, 0, 1 \end{bmatrix}$ 
# Colonnes pivots: 0, 1, 3.
# Base de l'image attendue: Colonnes 0, 1, 3 de M2
image_M2_expected_basis =  $\begin{bmatrix} 1, 4, 7 \\ 2, 5, 8 \\ 4, 7, 1 \end{bmatrix}$ 
image_M2 = find_image_basis(M2)
assert len(image_M2) == 3, f"Test 8 failed: Image M2 size. Expected 3, got {len(
    image_M2)}"
for i in range(3):
    for j in range(3):
        assert are_approximately_equal(image_M2[i][j], image_M2_expected_basis[i][j])
        , \
        f"Test 8 failed: Image M2. Expected {image_M2_expected_basis[i][j]}, got
            {image_M2[i][j]}"
print("Test 8 (Image de matrice M2) passé.")

# Test 9: Rang
assert rank(M1) == 1, f"Test 9 failed: Rank M1. Expected 1, got {rank(M1)}"
assert rank(M2) == 3, f"Test 9 failed: Rank M2. Expected 3, got {rank(M2)}"

```

```

print("Test 9 (Calcul du rang) passé.")

# Test 10: Nullité
assert nullity(M1) == 2, f"Test 10 failed: Nullity M1. Expected 2, got {nullity(M1)}"
assert nullity(M2) == 1, f"Test 10 failed: Nullity M2. Expected 1, got {nullity(M2)}"
# 4 cols - 3 rank = 1
print("Test 10 (Calcul de la nullité) passé.")

# Test 11: Théorème du Rang
assert verify_rank_nullity_theorem(M1), f"Test 11 failed: Rank-Nullity M1. {rank(M1)}"
+ {nullity(M1)} != {get_matrix_dimensions(M1)[1]}"
assert verify_rank_nullity_theorem(M2), f"Test 11 failed: Rank-Nullity M2. {rank(M2)}"
+ {nullity(M2)} != {get_matrix_dimensions(M2)[1]}"
print("Test 11 (Théorème du Rang) passé.")

# Test 12: Matrice non carrée (plus de lignes que de colonnes)
M3 = [[1.0, 0.0],
      [0.0, 1.0],
      [0.0, 0.0]]
# RREF is M3 itself
# Pivot cols: 0, 1
# Image basis: [[1,0,0], [0,1,0]] (cols 0 and 1 of M3)
# Kernel basis: [] (no free variables)
assert rank(M3) == 2, f"Test 12 failed: Rank M3. Expected 2, got {rank(M3)}"
assert nullity(M3) == 0, f"Test 12 failed: Nullity M3. Expected 0, got {nullity(M3)}"
assert verify_rank_nullity_theorem(M3), f"Test 12 failed: Rank-Nullity M3. {rank(M3)}"
+ {nullity(M3)} != {get_matrix_dimensions(M3)[1]}"
print("Test 12 (Matrice non carrée, plus de lignes) passé.")

# Test 13: Matrice non carrée (plus de colonnes que de lignes)
M4 = [[1.0, 2.0, 3.0],
      [4.0, 5.0, 6.0]]
# RREF_M4: [[1, 0, -1], [0, 1, 2]] (x + 2y + 3z -> x + 0y -z = 0, 0x + y + 2z = 0)
# Pivot cols: 0, 1
# Free var: 2 (z)
# Image basis: [[1,4], [2,5]] (cols 0 and 1 of M4)
# Kernel basis: z=1 -> x=1, y=-2. Vector: [1, -2, 1]
assert rank(M4) == 2, f"Test 13 failed: Rank M4. Expected 2, got {rank(M4)}"
assert nullity(M4) == 1, f"Test 13 failed: Nullity M4. Expected 1, got {nullity(M4)}"
assert verify_rank_nullity_theorem(M4), f"Test 13 failed: Rank-Nullity M4. {rank(M4)}"
+ {nullity(M4)} != {get_matrix_dimensions(M4)[1]}"
kernel_M4 = find_kernel_basis(M4)
assert len(kernel_M4) == 1 and all(is_zero(x) for x in matrix_vector_multiply(M4,
    kernel_M4[0])), \
    f"Test 13 failed: Kernel M4 vector {kernel_M4[0]} not in kernel."
print("Test 13 (Matrice non carrée, plus de colonnes) passé.")

print("\n--- Tous les tests de validation passés avec succès ! ---")

# --- Exemple d'application IA : Transformation de Caractéristiques ---
print("\n--- Exemple d'application IA : Transformation de Caractéristiques ---")

# Supposons que nous avons des données de clients avec 3 caractéristiques:
# [Revenu annuel, Âge, Score de fidélité]
client_data = [100000.0, 35.0, 8.0]
print_vector(client_data, "Données client originales (Revenu, Âge, Fidélité)")

# Application linéaire 1: Normalisation et combinaison de caractéristiques
# Nous voulons créer de nouvelles caractéristiques:
# f1 = Revenu / 10000 (normalisation)
# f2 = Âge - (Revenu / 5000) (âge ajusté par revenu, une forme de maturité financière)
# f3 = Score de fidélité * Âge (engagement pondéré par l'âge)

```

```

# Matrice de transformation T1 (3x3)
T1 = [[1/10000, 0.0,      0.0],
      [-1/5000, 1.0,      0.0],
      [0.0,      8.0,      1.0]] # Note: le 8.0 sur la 2ème ligne de la dernière
                                # colonne est une erreur, devrait être 0.0.
                                # Pour cet exemple, le calcul serait plutôt un 'âge *
                                # score'
                                # On va simplifier pour montrer l'idée:
                                # f3 = 0.1 * âge + 0.9 * fidélité

# Correction de T1 pour un exemple plus clair
T1 = [[1/10000, 0.0,      0.0], # f1 = Revenu / 10000
      [-1/5000, 1.0,      0.0], # f2 = Âge - Revenu/5000
      [0.0,      0.1,      0.9]] # f3 = 0.1*Âge + 0.9*Fidélité

print_matrix(T1, "Matrice de Transformation T1 (Normalisation/Combinaison)")
transformed_data_t1 = matrix_vector_multiply(T1, client_data)
print_vector(transformed_data_t1, "Caractéristiques transformées par T1")
# Attendu:
# f1 = 100000 / 10000 = 10.0
# f2 = 35 - (100000 / 5000) = 35 - 20 = 15.0
# f3 = 0.1 * 35 + 0.9 * 8 = 3.5 + 7.2 = 10.7
assert are_approximately_equal(transformed_data_t1[0], 10.0)
assert are_approximately_equal(transformed_data_t1[1], 15.0)
assert are_approximately_equal(transformed_data_t1[2], 10.7)
print("Transformation T1 vérifiée.")

# Application linéaire 2: Réduction de dimensionnalité (projection)
# Nous voulons projeter nos données dans un espace de 2 dimensions,
# où la première dimension représente le "potentiel financier"
# et la seconde le "niveau d'engagement".
# potentiel financier = 0.7 * f1 + 0.3 * f2 (pondération des nouvelles caracté
# ristiques)
# engagement = f3 (la même caractéristique)

# Matrice de transformation T2 (2x3)
T2 = [[0.7, 0.3, 0.0],
      [0.0, 0.0, 1.0]]

print_matrix(T2, "Matrice de Transformation T2 (Réduction de Dimensionnalité)")
reduced_data = matrix_vector_multiply(T2, transformed_data_t1)
print_vector(reduced_data, "Caractéristiques réduites par T2 (Potentiel Financier,
Engagement)")
# Attendu:
# potentiel financier = 0.7 * 10.0 + 0.3 * 15.0 = 7.0 + 4.5 = 11.5
# engagement = 10.7
assert are_approximately_equal(reduced_data[0], 11.5)
assert are_approximately_equal(reduced_data[1], 10.7)
print("Réduction de dimensionnalité T2 vérifiée.")

print("\n--- Analyse du Noyau et de l'Image pour T2 (Matrice de Réduction) ---")
print_matrix(T2, "Matrice T2")
print(f"Rang de T2 (dimension de l'image, espace des caractéristiques atteignables) :
{rank(T2)}")
print(f"Nullité de T2 (dimension du noyau, ce qui est 'ignoré') : {nullity(T2)}")

# Théorème du Rang pour T2: Rang (2) + Nullité (1) = Nombre de colonnes (3)
assert verify_rank_nullity_theorem(T2)
print(f"Vérification du théorème du rang pour T2 : {verify_rank_nullity_theorem(T2)}")

kernel_T2_basis = find_kernel_basis(T2)
print("Base du Noyau de T2 (vecteurs ignorés par la transformation) :")
for i, v in enumerate(kernel_T2_basis):
    print_vector(v, f" Vecteur {i+1}")

```



```

# Pour T2 = [[0.7, 0.3, 0.0], [0.0, 0.0, 1.0]]
# RREF est déjà cette matrice.
# Colonnes pivots: 0, 2. Variable libre: 1 (le deuxième élément du vecteur d'entrée)
# Les équations:
# 0.7 * x0 + 0.3 * x1 = 0
# x2 = 0
# Si x1 = 1 (variable libre)
# 0.7 * x0 + 0.3 * 1 = 0 => x0 = -0.3 / 0.7 = -3/7
# x2 = 0
# Vecteur du noyau : [-3/7, 1, 0] approx [-0.4286, 1.0, 0.0]
expected_kernel_T2 = [-3/7, 1.0, 0.0]
assert len(kernel_T2_basis) == 1
assert all(are_approximately_equal(kernel_T2_basis[0][i], expected_kernel_T2[i]) for
i in range(3))
print("Base du Noyau de T2 vérifiée.")

print("\nInterprétation pour T2:")
print(f"- Le rang de {rank(T2)} signifie que les données transformées résident dans
un espace de {rank(T2)} dimensions, malgré l'entrée en {get_matrix_dimensions(T2)
[1]} dimensions.")
print(f"- La nullité de {nullity(T2)} signifie qu'il y a {nullity(T2)} dimension(s)
dans l'espace des caractéristiques d'entrée qui ne contribuent pas à la sortie (
sont 'filtrées' ou mappées à zéro).")
print(f"- Le vecteur de base du noyau [-0.4286, 1.0, 0.0] indique que si les caracté
ristiques transformées par T1 varient proportionnellement à ce vecteur (c'est-à-
dire si le second terme (f2) augmente de 1 et le premier (f1) diminue de 0.4286,
et le troisième (f3) ne change pas), cela n'affectera pas la projection sur l'axe
du 'potentiel financier' dans l'espace image de T2. Plus précisément, cela
affecterait *uniquement* la première équation. L'effet de la variation de f1 et
f2 est annulé pour l'axe 'potentiel financier'. La valeur de 'engagement' (f3)
est indépendante.")

image_T2_basis = find_image_basis(T2)
print("Base de l'Image de T2 (espace des caractéristiques atteignables) :")
for i, v in enumerate(image_T2_basis):
    print_vector(v, f" Vecteur {i+1}")
# Les vecteurs de la base de l'image sont les colonnes pivots de T2:
# Colonne 0: [0.7, 0.0]
# Colonne 2: [0.0, 1.0]
expected_image_T2_basis = [[0.7, 0.0], [0.0, 1.0]]
assert len(image_T2_basis) == 2
for i in range(2):
    for j in range(2):
        assert are_approximately_equal(image_T2_basis[i][j], expected_image_T2_basis[
i][j])
print("Base de l'Image de T2 vérifiée.")

print("\nInterprétation des bases de l'Image:")
print("- La base de l'image nous donne les 'directions' ou 'composantes principales'
qui définissent l'espace de sortie. Dans ce cas, elles correspondent aux deux
dimensions que nous avons définies : 'potentiel financier' et 'engagement'.")

```

\{\chapter*{TP 5 : Géométrie des Transformations Linéaires : Noyau, Image et Rang "from scratch"

\{\section*{Objectif Pédagogique (Mathématique et IA)

Ce TP a pour objectif de solidifier la compréhension et l'implémentation des concepts fondamentaux d'algèbre linéaire que sont les applications linéaires, leur noyau (kernel), leur image, et le théorème du rang. En codant ces concepts "from scratch" en Python, sans l'aide de bibliothèques de haut niveau comme NumPy, nous allons approfondir la mécanique sous-jacente des transformations de données.

\{\textbf{Objectifs Mathématiques : * Implémenter la représentation matricielle d'une application linéaire. * Maîtriser l'algorithme d'élimination de Gauss-Jordan pour transformer une matrice en forme échelonnée réduite (RREF). * Dédire le noyau (espace nul) d'une matrice à partir de sa RREF. * Dédire l'image (espace colonne) d'une matrice à partir de sa forme échelonnée. * Calculer le rang et la nullité d'une matrice. * Vérifier le théorème du rang : $\dim(\text{Ker}(A)) + \dim(\text{Im}(A)) = \dim(\text{domaine})$.

\{\textbf{Objectifs IA et Machine Learning : * \{\textbf{Compréhension des Transformations de Données : Les applications linéaires sont au cœur des modèles d'IA, notamment les réseaux de neurones (où les couches sont des transformations linéaires). Comprendre le noyau et l'image permet de saisir comment les informations sont transformées, perdues ou préservées. * \{\textbf{Réduction de Dimensionnalité : Le concept d'image est directement lié à des techniques comme l'Analyse en Composantes Principales (PCA), où l'on projette des données sur un sous-espace d'intérêt (l'image d'une matrice de projection) pour réduire la complexité tout en maximisant la variance. * \{\textbf{Redondance et Dépendance Linéaire : Le noyau révèle les combinaisons de caractéristiques d'entrée qui n'apportent aucune information utile après une transformation, car elles sont mappées au vecteur nul. Cela aide à identifier la redondance dans les données ou les caractéristiques. * \{\textbf{Capacité des Modèles : Le rang d'une matrice de poids dans un réseau de neurones peut donner une idée de la capacité expressive du modèle ou de la complexité des transformations qu'il peut effectuer. Une matrice de faible rang peut indiquer un modèle sous-paramétré ou une transformation restrictive. * \{\textbf{Stabilité Numérique et Inversibilité : Bien que non directement implémentée, la notion de noyau est cruciale pour comprendre l'inversibilité des matrices, un concept fondamental en optimisation et dans la résolution de systèmes linéaires.

Ce TP met l'accent sur l'importance de construire une intuition géométrique et algébrique solide, qui est essentielle pour la conception, l'analyse et le débogage de systèmes d'IA complexes.

\{\section*{Code Source Python "from scratch"

```
import math

# --- Configuration pour la précision des flottants ---
EPSILON = 1e-9 # Petite valeur pour comparer les flottants à zéro

def is_zero(value):
    """Vérifie si une valeur flottante est proche de zéro."""
    return abs(value) < EPSILON

def format_vector(v):
    """Formate un vecteur pour un affichage lisible."""
    return "[" + ", ".join(f"{x:.4f}" for x in v) + "]"

def format_matrix(m):
    """Formate une matrice pour un affichage lisible."""
    if not m: return "[]"
    return "[\n" + ",\n".join(" " + format_vector(row) for row in m) + "\n]"

# --- Opérations de base sur les vecteurs et matrices ---

def add_vectors(v1, v2):
    """Additionne deux vecteurs."""
    if len(v1) != len(v2):
        raise ValueError("Les vecteurs doivent avoir la même dimension pour être additionnés.")
    return [v1[i] + v2[i] for i in range(len(v1))]

def scalar_mult_vector(scalar, v):
    """Multiplie un vecteur par un scalaire."""
    return [scalar * x for x in v]

def dot_product(v1, v2):
    """Calcule le produit scalaire de deux vecteurs."""
    if len(v1) != len(v2):
        raise ValueError("Les vecteurs doivent avoir la même dimension pour le
```

```

        produit scalaire.")
    return sum(v1[i] * v2[i] for i in range(len(v1)))

def matrix_vector_mult(matrix, vector):
    """Multiplie une matrice par un vecteur."""
    num_rows = len(matrix)
    num_cols = len(matrix[0]) if num_rows > 0 else 0

    if num_cols != len(vector):
        raise ValueError("Le nombre de colonnes de la matrice doit être égal à la dimension du vecteur.")

    result = [0.0] * num_rows
    for i in range(num_rows):
        result[i] = dot_product(matrix[i], vector)
    return result

def transpose_matrix(matrix):
    """Calcule la transposée d'une matrice."""
    if not matrix or not matrix[0]:
        return []
    num_rows = len(matrix)
    num_cols = len(matrix[0])
    transposed = [[0.0 for _ in range(num_rows)] for _ in range(num_cols)]
    for i in range(num_rows):
        for j in range(num_cols):
            transposed[j][i] = matrix[i][j]
    return transposed

def matrix_mult(m1, m2):
    """Multiplie deux matrices."""
    if not m1 or not m1[0] or not m2 or not m2[0]:
        raise ValueError("Les matrices ne peuvent pas être vides.")

    rows_m1 = len(m1)
    cols_m1 = len(m1[0])
    rows_m2 = len(m2)
    cols_m2 = len(m2[0])

    if cols_m1 != rows_m2:
        raise ValueError("Le nombre de colonnes de la première matrice doit être égal au nombre de lignes de la seconde.")

    m2_t = transpose_matrix(m2) # Transposer m2 pour un accès structuré aux colonnes
    result = [[0.0 for _ in range(cols_m2)] for _ in range(rows_m1)]

    for i in range(rows_m1):
        for j in range(cols_m2):
            result[i][j] = dot_product(m1[i], m2_t[j])
    return result

# --- Élimination de Gauss-Jordan (RREF) ---

def find_pivot_row(matrix, col, start_row):
    """Trouve la première ligne non nulle (pivot) dans une colonne à partir de start_row."""
    num_rows = len(matrix)
    for i in range(start_row, num_rows):
        if not is_zero(matrix[i][col]):
            return i
    return -1

def swap_rows(matrix, r1, r2):
    """Échange deux lignes dans une matrice."""
    matrix[r1], matrix[r2] = matrix[r2], matrix[r1]

```

```

def scale_row(matrix, row, scalar):
    """Multiplie une ligne par un scalaire."""
    for j in range(len(matrix[row])):
        matrix[row][j] *= scalar

def add_scaled_row(matrix, target_row, source_row, scalar):
    """Ajoute un multiple scalaire d'une ligne à une autre ligne."""
    for j in range(len(matrix[target_row])):
        matrix[target_row][j] += scalar * matrix[source_row][j]

def row_echelon_form(matrix_original):
    """
    Convertit une matrice en Forme Échelonnée (REF).
    Retourne une nouvelle matrice pour ne pas modifier l'originale.
    """
    matrix = [row[:] for row in matrix_original] # Copie la matrice pour ne pas
        modifier l'originale
    num_rows = len(matrix)
    if num_rows == 0: return []
    num_cols = len(matrix[0])
    if num_cols == 0: return [[] for _ in range(num_rows)]

    lead = 0 # Indice de colonne du pivot
    for r in range(num_rows):
        if lead >= num_cols:
            break
        i = find_pivot_row(matrix, lead, r)
        if i != -1: # Si un pivot est trouvé
            swap_rows(matrix, r, i)
            # Normaliser la ligne pour que le pivot soit 1
            lv = matrix[r][lead]
            scale_row(matrix, r, 1.0 / lv)
            # Éliminer les éléments en dessous du pivot
            for i in range(num_rows):
                if i != r:
                    lv_below = matrix[i][lead]
                    add_scaled_row(matrix, i, r, -lv_below)
            lead += 1
        else: # Pas de pivot dans cette colonne, passer à la suivante
            lead += 1

    # Nettoyage des petites valeurs pour éviter des erreurs de flottants
    for r in range(num_rows):
        for c in range(num_cols):
            if is_zero(matrix[r][c]):
                matrix[r][c] = 0.0 # Mettre à zéro les valeurs très proches de zéro

    return matrix

def reduced_row_echelon_form(matrix_original):
    """
    Convertit une matrice en Forme Échelonnée Réduite (RREF).
    Retourne une nouvelle matrice pour ne pas modifier l'originale.
    """
    matrix = [row[:] for row in matrix_original] # Copie la matrice pour ne pas
        modifier l'originale
    num_rows = len(matrix)
    if num_rows == 0: return []
    num_cols = len(matrix[0])
    if num_cols == 0: return [[] for _ in range(num_rows)]

    lead = 0
    for r in range(num_rows):
        if lead >= num_cols:
            break

```

```

    i = find_pivot_row(matrix, lead, r)
    if i != -1:
        swap_rows(matrix, r, i)
        # Normaliser la ligne pour que le pivot soit 1
        lv = matrix[r][lead]
        scale_row(matrix, r, 1.0 / lv)
        # Éliminer les éléments au-dessus et en dessous du pivot
        for i in range(num_rows):
            if i != r:
                lv_other = matrix[i][lead]
                add_scaled_row(matrix, i, r, -lv_other)
        lead += 1
    else: # Pas de pivot dans cette colonne, passer à la suivante
        lead += 1

# Nettoyage des petites valeurs pour éviter des erreurs de flottants
for r in range(num_rows):
    for c in range(num_cols):
        if is_zero(matrix[r][c]):
            matrix[r][c] = 0.0

return matrix

# --- Fonctions pour Noyau, Image, Rang ---

def get_rank(matrix):
    """
    Calcule le rang d'une matrice (dimension de l'image).
    Le rang est le nombre de lignes non nulles dans la forme échelonnée réduite.
    """
    rref = reduced_row_echelon_form(matrix)
    rank = 0
    for row in rref:
        if any(not is_zero(x) for x in row):
            rank += 1
    return rank

def find_image_basis(matrix):
    """
    Trouve une base pour l'image (espace colonne) d'une matrice.
    Les colonnes de la matrice originale correspondant aux colonnes pivot de la RREF
    forment une base pour l'image.
    """
    num_rows = len(matrix)
    if num_rows == 0: return []
    num_cols = len(matrix[0])
    if num_cols == 0: return []

    rref = reduced_row_echelon_form(matrix)

    pivot_columns_indices = []
    current_pivot_row = 0
    for j in range(num_cols):
        if current_pivot_row < num_rows and not is_zero(rref[current_pivot_row][j]):
            pivot_columns_indices.append(j)
            current_pivot_row += 1

    # Extraire les colonnes originales correspondant aux pivots
    image_basis = []
    for col_idx in pivot_columns_indices:
        column_vector = [matrix[i][col_idx] for i in range(num_rows)]
        image_basis.append(column_vector)

    return image_basis

def find_kernel_basis(matrix):

```

```

"""
Trouve une base pour le noyau (espace nul) d'une matrice.
Utilise la RREF pour identifier les variables libres et pivots.
"""
num_rows = len(matrix)
if num_rows == 0: return []
num_cols = len(matrix[0])
if num_cols == 0: return [[]] # Vecteur nul de la bonne dimension si l'espace est
    vide

rref = reduced_row_echelon_form(matrix)

pivot_cols = []
free_cols = []
pivot_row_for_col = [-1] * num_cols # Associe chaque colonne pivot à sa ligne

current_pivot_row = 0
for j in range(num_cols):
    if current_pivot_row < num_rows and not is_zero(rref[current_pivot_row][j]):
        pivot_cols.append(j)
        pivot_row_for_col[j] = current_pivot_row
        current_pivot_row += 1
    else:
        free_cols.append(j)

kernel_basis = []

for free_col_idx in free_cols:
    basis_vector = [0.0] * num_cols
    basis_vector[free_col_idx] = 1.0 # La variable libre correspondante est 1

    # Pour chaque variable pivot, exprimer en fonction de cette variable libre
    for pivot_col_idx in pivot_cols:
        r = pivot_row_for_col[pivot_col_idx]
        # Si le terme correspondant à la variable libre est non nul dans l'é
        # quation du pivot
        basis_vector[pivot_col_idx] = -rref[r][free_col_idx]

    kernel_basis.append(basis_vector)

# Si toutes les colonnes sont pivots et que le rang est plein (matrice carrée
# inversible), le noyau est le vecteur nul.
# Dans ce cas, free_cols sera vide, et kernel_basis sera vide.
# On ajoute le vecteur nul [0,0,...,0] si l'espace du noyau est juste {0}
if not free_cols and get_rank(matrix) == num_cols:
    return [] # Noyau trivial, seul le vecteur nul est dedans. Pas de base non
        triviale.

return kernel_basis

def get_nullity(matrix):
    """
    Calcule la nullité d'une matrice (dimension du noyau).
    """
    return len(find_kernel_basis(matrix))

def check_rank_nullity_theorem(matrix):
    """
    Vérifie le théorème du rang:  $\dim(\text{Ker}(A)) + \dim(\text{Im}(A)) = \dim(\text{domaine})$ .
     $\dim(\text{domaine})$  est le nombre de colonnes de la matrice.
    """
    if not matrix or not matrix[0]:
        # Cas des matrices vides ou à une seule ligne/colonne vide
        num_cols = len(matrix[0]) if matrix else 0
        return get_nullity(matrix) + get_rank(matrix) == num_cols

```

```

    num_cols = len(matrix[0])
    rank = get_rank(matrix)
    nullity = get_nullity(matrix)

    return rank + nullity == num_cols

# --- Assertions de Validation Mathématique ---

print("--- Début des tests ---")

# Matrice identité 3x3
I3 = [[1.0, 0.0, 0.0],
      [0.0, 1.0, 0.0],
      [0.0, 0.0, 1.0]]

# Matrice nulle 2x3
Z23 = [[0.0, 0.0, 0.0],
       [0.0, 0.0, 0.0]]

# Matrice A (full rank 2x2)
A = [[1.0, 2.0],
     [3.0, 4.0]]

# Matrice B (non-carrée)
B = [[1.0, 2.0, 3.0],
     [4.0, 5.0, 6.0]]

# Matrice C (avec dépendance linéaire)
C = [[1.0, 2.0, 3.0],
     [2.0, 4.0, 6.0],
     [7.0, 8.0, 9.0]]

# Matrice D (avec dépendance linéaire plus complexe)
D = [[1.0, 1.0, 2.0],
     [2.0, 2.0, 4.0],
     [0.0, 1.0, 1.0]]

# Matrice E (pour un noyau non trivial simple)
E = [[1.0, 1.0, 1.0],
     [0.0, 1.0, 1.0],
     [0.0, 0.0, 0.0]]

# Matrice F (pour un noyau 2D)
F = [[1.0, 2.0, 3.0, 4.0],
     [5.0, 6.0, 7.0, 8.0],
     [9.0, 10.0, 11.0, 12.0]]

print("\n--- Test 1: Opérations de base ---")
v1 = [1.0, 2.0]
v2 = [3.0, 4.0]
assert add_vectors(v1, v2) == [4.0, 6.0]
assert scalar_mult_vector(2.0, v1) == [2.0, 4.0]
assert dot_product(v1, v2) == 1.0*3.0 + 2.0*4.0 # 3 + 8 = 11
assert matrix_vector_mult(A, v1) == [1.0*1.0 + 2.0*2.0, 3.0*1.0 + 4.0*2.0] # [5, 11]
assert matrix_vector_mult(A, v1) == [5.0, 11.0]

assert matrix_mult(A, I3[:2]) == A # Test A * I_2x2 (partie de I3)
# Need a 2x2 identity matrix for the above. Let's define one.
I2 = [[1.0, 0.0], [0.0, 1.0]]
assert matrix_mult(A, I2) == A
print("Test 1: Opérations de base PASSED")

print("\n--- Test 2: Réduction RREF ---")
# RREF de I3 doit être I3

```

```

assert reduced_row_echelon_form(I3) == I3

# RREF de Z23 doit être Z23
assert reduced_row_echelon_form(Z23) == Z23

# RREF de A (matrice inversible)
A_rref = reduced_row_echelon_form(A)
expected_A_rref = [[1.0, 0.0], [0.0, 1.0]]
assert all(all(is_zero(A_rref[i][j] - expected_A_rref[i][j]) for j in range(len(
    A_rref[0]))) for i in range(len(A_rref)))

# RREF de C (dépendance linéaire)
C_rref = reduced_row_echelon_form(C)
expected_C_rref = [[1.0, 0.0, -1.0],
                   [0.0, 1.0, 2.0],
                   [0.0, 0.0, 0.0]]
for i in range(len(C_rref)):
    for j in range(len(C_rref[0])):
        assert is_zero(C_rref[i][j] - expected_C_rref[i][j]), f"C_rref[{i}][{j}] = {
            C_rref[i][j]}, Expected: {expected_C_rref[i][j]}"
print("Test 2: Réduction RREF PASSED")

print("\n--- Test 3: Rang de la matrice ---")
assert get_rank(I3) == 3
assert get_rank(Z23) == 0
assert get_rank(A) == 2 # Matrice 2x2 full rank
assert get_rank(B) == 2 # Matrice 2x3 full row rank
assert get_rank(C) == 2 # Matrice 3x3, rang 2
assert get_rank(D) == 2 # Matrice 3x3, rang 2
assert get_rank(E) == 2 # Matrice 3x3, rang 2
assert get_rank(F) == 2 # Matrice 3x4, rang 2
print("Test 3: Rang de la matrice PASSED")

print("\n--- Test 4: Base de l'Image (espace colonne) ---")
# Image de I3 est  $\mathbb{R}^3$ , base est les vecteurs canoniques
image_I3_basis = find_image_basis(I3)
assert len(image_I3_basis) == 3
assert all(any(is_zero(x - expected_x) for x, expected_x in zip(v, [1.0, 0.0, 0.0]))
           for v in image_I3_basis)
assert all(any(is_zero(x - expected_x) for x, expected_x in zip(v, [0.0, 1.0, 0.0]))
           for v in image_I3_basis)
assert all(any(is_zero(x - expected_x) for x, expected_x in zip(v, [0.0, 0.0, 1.0]))
           for v in image_I3_basis)

# Image de A (matrice 2x2 full rank) est  $\mathbb{R}^2$ , base est les colonnes de A
image_A_basis = find_image_basis(A)
assert len(image_A_basis) == 2
assert image_A_basis == [[1.0, 3.0], [2.0, 4.0]] # colonnes originales de A

# Image de C (rang 2)
image_C_basis = find_image_basis(C)
# Les colonnes pivot de C sont la 1ère et la 2ème (indices 0 et 1)
assert len(image_C_basis) == 2
assert image_C_basis[0] == [1.0, 2.0, 7.0]
assert image_C_basis[1] == [2.0, 4.0, 8.0]

# Image de F (rang 2)
image_F_basis = find_image_basis(F)
assert len(image_F_basis) == 2
assert image_F_basis[0] == [1.0, 5.0, 9.0]
assert image_F_basis[1] == [2.0, 6.0, 10.0]
print("Test 4: Base de l'Image (espace colonne) PASSED")

```



```

print("\n--- Test 5: Base du Noyau (espace nul) ---")
# Noyau de I3 (matrice inversible) est {0}
kernel_I3_basis = find_kernel_basis(I3)
assert len(kernel_I3_basis) == 0 # Noyau trivial

# Noyau de A (matrice 2x2 inversible) est {0}
kernel_A_basis = find_kernel_basis(A)
assert len(kernel_A_basis) == 0

# Noyau de C (rang 2, 3 colonnes, nullité 1)
kernel_C_basis = find_kernel_basis(C)
assert len(kernel_C_basis) == 1
# De la RREF de C:  $x_1 - x_3 = 0 \Rightarrow x_1 = x_3$ ,  $x_2 + 2x_3 = 0 \Rightarrow x_2 = -2x_3$ 
# Variable libre  $x_3$ . Si  $x_3 = 1$ , alors  $x_1 = 1$ ,  $x_2 = -2$ . Vecteur:  $[1, -2, 1]$ 
expected_kernel_C_vec = [1.0, -2.0, 1.0]
# Assurez-vous que le vecteur du noyau trouvé est un multiple de l'attendu.
# matrix_vector_mult(C, expected_kernel_C_vec) should be 0 vector
res_C_kernel = matrix_vector_mult(C, kernel_C_basis[0])
assert all(is_zero(x) for x in res_C_kernel)

# Noyau de E (rang 2, 3 colonnes, nullité 1)
kernel_E_basis = find_kernel_basis(E)
assert len(kernel_E_basis) == 1
# RREF de E:  $\begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 1 \\ 0 & 0 & 0 \end{bmatrix}$ 
#  $x_1 = 0$ ,  $x_2 + x_3 = 0 \Rightarrow x_2 = -x_3$ .  $x_3$  est libre. Si  $x_3=1$ ,  $x_2=-1$ ,  $x_1=0$ .
# Vecteur:  $[0, -1, 1]$ 
expected_kernel_E_vec = [0.0, -1.0, 1.0]
res_E_kernel = matrix_vector_mult(E, kernel_E_basis[0])
assert all(is_zero(x) for x in res_E_kernel)

# Noyau de F (rang 2, 4 colonnes, nullité 2)
kernel_F_basis = find_kernel_basis(F)
assert len(kernel_F_basis) == 2
# RREF de F:
#  $\begin{bmatrix} 1.0 & 0.0 & -1.0 & -2.0 \\ 0.0 & 1.0 & 2.0 & 3.0 \\ 0.0 & 0.0 & 0.0 & 0.0 \end{bmatrix}$ 
#  $x_1 - x_3 - 2x_4 = 0 \Rightarrow x_1 = x_3 + 2x_4$ 
#  $x_2 + 2x_3 + 3x_4 = 0 \Rightarrow x_2 = -2x_3 - 3x_4$ 
#  $x_3, x_4$  sont libres.
# 1)  $x_3=1, x_4=0 \Rightarrow x_1=1, x_2=-2$ . Vecteur:  $[1, -2, 1, 0]$ 
# 2)  $x_3=0, x_4=1 \Rightarrow x_1=2, x_2=-3$ . Vecteur:  $[2, -3, 0, 1]$ 
# Vérifier que les vecteurs générés sont bien dans le noyau
for v_basis in kernel_F_basis:
    res_F_kernel = matrix_vector_mult(F, v_basis)
    assert all(is_zero(x) for x in res_F_kernel)
print("Test 5: Base du Noyau (espace nul) PASSED")

print("\n--- Test 6: Nullité de la matrice ---")
assert get_nullity(I3) == 0
assert get_nullity(A) == 0
assert get_nullity(C) == 1
assert get_nullity(D) == 1 # rang 2, nullité 1 (pour D =  $\begin{bmatrix} 1 & 1 & 2 \\ 2 & 2 & 4 \\ 0 & 1 & 1 \end{bmatrix}$ )
assert get_nullity(E) == 1
assert get_nullity(F) == 2
print("Test 6: Nullité de la matrice PASSED")

print("\n--- Test 7: Théorème du Rang ---")
# dim(domaine) est le nombre de colonnes
assert check_rank_nullity_theorem(I3) == True # rank=3, nullity=0, cols=3
assert check_rank_nullity_theorem(Z23) == True # rank=0, nullity=3, cols=3 (nullity = len(Z23[0]) - rank)
assert get_nullity(Z23) == 3 # le noyau de Z23 est  $\mathbb{R}^3$ , de base

```

```

    [[1,0,0],[0,1,0],[0,0,1]]
assert check_rank_nullity_theorem(A) == True # rank=2, nullity=0, cols=2
assert check_rank_nullity_theorem(B) == True # rank=2, nullity=1, cols=3
assert check_rank_nullity_theorem(C) == True # rank=2, nullity=1, cols=3
assert check_rank_nullity_theorem(D) == True # rank=2, nullity=1, cols=3
assert check_rank_nullity_theorem(E) == True # rank=2, nullity=1, cols=3
assert check_rank_nullity_theorem(F) == True # rank=2, nullity=2, cols=4
print("Test 7: Théorème du Rang PASSED")

print("\n--- Tous les tests sont passés avec succès ! ---")

```